

СОДЕРЖАНИЕ

Управление внешними устройствами: специальные файлы устройств, адресация внешних устройств и их идентификация в системе, тип `dev_t`. Система прерываний: типы прерываний и их особенности. Прерывания в последовательности ввода-вывода — обслуживание запроса процесса на ввод-вывод (диаграмма). Быстрые и медленные прерывания. Обработка аппаратных прерываний: регистрация в системе — функция и ее параметры, примеры. Такелсты — объявление, планирование (пример таб. раб).

Управление внешними устройствами: специальные файлы устройств, идентификация внешних устройств в системе (тип `dev_t`). Система прерываний: типы прерываний и их особенности. Быстрые и медленные прерывания. Обработка аппаратных прерываний: регистрация в системе. Линия IRQ (простейшая концептуальная схема — три шины). Верхние и нижние `softirq` — перечисления и функции ядра для выполнения `softirq`, такелсты и очереди работ. К `softirq` — схема обслуживания входящих сетевых пакетов. `Softirq` и `tasklet`: объявление, инициализация, планирование такелстов (примеры из таб. раб).

Сокеты как средство взаимодействия параллельных процессов. Сокеты семейства `AF_INET` и `AF_INET6`. Сокеты в пространстве файловых ячеек. Сокеты `AF_INET`, сетевой стек. Классификация моделей ввода-вывода. Мультиплексирование при взаимодействии процессов в распределённых системах по модели клиент-сервер. Ввод-вывод с мультиплексированием: мультиплексор `epoll` — особенности (типы и алгоритм работы). Пример из лабораторной работы — сервер производное-потребление с мультиплексированием.

Файловая подсистема `/proc` — назначение, особенности, файлы, поддиректории, ссылка `self`, информация об окружении, состоянии процесса, прерываниях. Структура `proc_dir_entry` функции для работы с элементами `/proc`. Использование структуры `file_operations` для регистрации функций работы с файлами. Передача данных из адресного пространства пользователя в адресное пространство ядра и обратно (таб. раб). Обоснование необходимости использования специальных функций для передачи данных из пространства пользователя в ядро и из ядра в пространство пользователя.

2 / 257

1 Управление внешними устройствами: специальные файлы устройств, адресация внешних устройств и их идентификация в системе, тип `dev_t`. Система прерываний: типы прерываний и их особенности. Прерывания в последовательности ввода-вывода — обслуживание запроса процесса на ввод-вывод (диаграмма). Быстрые и медленные прерывания. Обработка аппаратных прерываний: регистрация в системе — функция и ее параметры, примеры. Такелсты — объявление, планирование (пример таб. раб).

1.1 Управление внешними устройствами: специальные файлы устройств, адресация и идентификация, тип `dev_t`

Существует два типа внешних устройств:

- Байт-ориентированные (или символьные) устройства передают данные посылками, как непрерывный поток байтов (например, мышь, клавиатура).
- Блок-ориентированные (или блочные) устройства (например, жёсткий диск) передают данные блоками. Все байтоориентированные устройства относятся к блочным.

Чтобы управлять всем разнообразием внешних устройств, в Unix/Linux реализована концепция драйверов. Она заключается в том, что устройства должны быть зарегистрированы в системе и в системе существует набор драйверов, которые можно использовать для управления новыми устройствами. Драйвер — это часть кода ядра, которая предназначена для управления конкретным внешним устройством.

В Linux используется парадигма «всё — файл», поэтому устройство — тоже файл. Но в полном объёме это невозможно реализовать как с обычными файлами. Устройства в системе должны идентифицироваться. Для этого в системе определён специальный код. Файлы внешних устройств находятся в директории `/dev`. В ней с помощью `ls -l` видим внешние устройства.

```
1 ls -l /dev
2 crw-rw-rw- 1 root root 1, 3 Apr 11 2002 null
3 crw-rw-rw- 1 root root 88, 1 Apr 11 2002 ram
4 crw-rw-rw- 1 root root 4, 1 Oct 28 03:04 ttyL
5 crw-rw-rw- 1 root tty 4, 64 Apr 11 2002 ttyS0
6 crw-rw-rw- 1 root uucp 4, 65 Apr 11 2002 ttyS1
7 crw-rw-rw- 1 vcsa tty 7, 1 Apr 11 2002 vcsL
8 crw-rw-rw- 1 vcsa tty 7, 129 Apr 11 2002 vcsA
9 crw-rw-rw- 1 root root 1, 5 Apr 11 2002 zero
```

Старший номер идентифицирует драйвер, связанный с устройством. Например, `/dev/tty` и `/dev/zero` управляются драйвером 1, виртуальные терминалы — драйвером 4. Младший номер используется для различия конкретных устройств в рамках одного драйвера (например, жёсткий диск имеет один старший номер, а его разделы — разные машины).

7 / 257

Управление устройствами: абстракция устройств, типы устройств и идентификация в ядре Unix/Linux. Управление устройствами: драйверы. USB-шина: особенности, хост и хабы, конечные точки и каналы, 4 типа передачи данных. Структура USB-драйвера (`struct usb_driver`), таблица `id_table`, основные точки входа драйвера USB. Регистрация `usb_driver` в системе.

Файловая система, задачи файловой системы и иерархическая организация ФС. Файловая подсистема `LINUX`, поддержка большого числа файловых систем и структура, описывающая файловую систему. VFS: четыре основные структуры файловой системы и связь между ними. Раздел жёсткого диска и суперблок. Структура `struct super_operations`. Мониторинг файловой системы, команда `mount` и функции монтирования и их параметры, точка монтирования — корневой каталог и `inode`. Пример (таб. раб).

Виртуальная файловая система `/proc`: структура `proc_dir_entry`, создание файла `proc`, директории и символические ссылки: точки входа и их регистрация, интерфейс передачи данных из ядра в пространство пользователя — реализация и особенности. Примеры из лабораторных работ.

Средства взаимодействия процессов — сокеты Беркли. Создание сокета — семейство, тип, протокол. Системный вызов `sys_socket` и `inet_socket`. Состояния сокета. Адресация сокета и ее особенности для разных типов сокетов. Модель клиент-сервер. Сетевые сокеты — сетевой стек, аппаратный и сетевой порядок байтов. Примеры реализации взаимодействия процессов по модели клиент-сервер с использованием сокетов и мультиплексирования (таб. раб).

Файловая система: процесс и файловые структуры связанные с процессом. Файлы и открытые файлы, связь структур, представляющих открытые файлы на разных уровнях. Системный вызов `open` и библиотечная функция `fopen`: параметры и флаги, определённые на функции `open`. Реализация системного вызова `open` в ядре Linux. Пример: файл открывается два раза системным вызовом `open` для записи и в него последовательно записывается строка «аааааааааа» по первому дескриптору и затем строка «ввв» по второму дескриптору, затем файл закрывается два раза. Показано, что будет записано в файл и пояснён результат.

3 / 257

Стандарт POSIX-1 оговаривает существование типа `dev_t` — 32-разрядное число, в котором 12 бит отведены под старший номер, 20 — под младший. Код не должен делать предположений об организации номеров внутри числа. Для работы с `dev_t` используются макросы:

```
1 MAJOR(dev_t dev); // получить старший номер
2 MINOR(dev_t dev); // получить младший номер
```

Имея старший и младший номера, можно их объединить в значение типа `dev_t`.

```
1 MAKEDEV(major, int minor) // объединить в dev_t
```

1.2 Выделение и освобождение номеров устройств

Одно из первых действий, которые должен сделать драйвер, когда устанавливает символическое устройство — он должен получить 1 или более номеров устройств, чтобы работать с ними, чтобы идентифицировать свой `char device`. Для этого вызывается функция

```
1 int register_chrdev_region(dev_t first, unsigned int count, char *name);
```

— `first` — начальный номер устройства доступного для выделения диапазона

Часто минорная часть значения `first` равна нулю. Но этот счёт нет никаких определённых требований.

— `count` — общее количество последовательных номеров устройств, которые могут быть зарезервированы. При этом значение `count` может превышать размер запрашиваемого диапазона. В этом случае будет переход на следующий старший номер, ошибка не возникает, если запрашиваемый диапазон доступен.

— `name` — имя устройства, которое связывается с диапазоном. Это можно увидеть или в `/proc/devices` или в `sysfs (/sys)`.

При успешном выделении функция возвращает 0.

Часто неизвестно значение старшего номера устройства, поэтому разработчики ядра всё время стремятся перейти на динамическое распределение номеров устройств. Для динамического выделения используется функция

```
1 int alloc_chrdev_region(dev_t *dev, unsigned int firstminor, unsigned int count, char *name);
```

— `dev` — значение указателя при успешном завершении будет содержать первый номер в определённом диапазоне. Обычно указывают 0.

8 / 257

Файловая система `OC Linux`: иерархическая структура каталогов, имена файлов и идентификация файлов. `Inode` и `dentry`: пример доступа к файлу `mbox`. Книш `inode` и `dentry`. Назначение суперблока в интерфейсе VFS: монтирование файловой системы — суперблок и связанные с ним структуры: структура `itree`, `super_block` и корневой каталог. Структуры `struct super_operations`. Структуры `struct inode`, `struct inode_operations`. Функции `simple` и `generic`. Точка монтирования. Функции монтирования. Пример создания файловой системы, её регистрации и монтирование (таб. раб.).

Аппаратные прерывания в Linux: запрос прерывания и линии IRQ. Простейшая схема аппаратной поддержки прерываний (концептуальная трёхшинная архитектура системы). Быстрые и медленные прерывания, флаги, пример быстрого прерывания. Нижняя и верхняя половины обработчиков прерывания, функция регистрации прерывания и её параметры. Нижняя половина `softirq`, определение, массив `softirq_vec`, типы `soft_irq`, функции `open_softirq`, `raise_softirq`. Поток ядра `ksoftirqd`. Пример обработки приходящих пакетов `NET_RX_SOFTIRQ`.

Создание виртуальных файловых систем. Структура, описывающая файловую систему. Регистрация и прекращение регистрации файловой системы в `OC Linux`. Мониторинг файловой системы. Точка монтирования. Функции монтирования. Команда монтирования файловой системы. Суперблок, назначение суперблока. `Inode` и адресация очень больших файлов (`fs_cach`). Структура `dentry`. Пример доступа к файлу `mbox`. Мониторинг в системе: `dentry` кэш и `kni` `inode`. Книш `SLAB`, функции для работы с кешем `SLAB`. Функции, определённые в файлах и их регистрация (`struct file_operations`). Пример из лабораторной работы.

Файловая подсистема: особенности файловой подсистемы `Unix/Linux`: иерархическая структура файловой подсистемы. Виртуальная файловая система `VFS` в `Linux`. Четыре структуры `VFS` — `super_block`, `inode`, `dentry`, `file` — их назначение. Адресация файлов большого размера в файловой системе `extX` и пример, показывающий доступ к файлу `/usr/src/mbox`. Книрование `dentry` и `inode`. `Slab` кэш. Мониторинг файловой системы. Команда `mount` и функции монтирования, пример из таб. раб.

4 / 257

Остальные параметры аналогичны рассмотренной ранее функции.

Номера освобождаются функцией

```
1 void unregister_chrdev_region(dev_t first, unsigned int count);
```

Все эти функции вызываются в модуле в функции `init`, а последняя функция — в функции `exit`.

1.3 Система прерываний: типы прерываний и их особенности. Три типа событий, переводящих систему в режим ядра:

- Системные вызовы (программные прерывания) — процесс переключается в режим ядра и выполняет реентерабельный код ядра, после переключается в режим пользователя.
- Исключения — синхронные. Два типа: исправные (странное прерывание, после обработки программа продолжает выполнение) и неисправные (программа завершается аварийно).
- Аппаратные прерывания — асинхронные. Три типа:
 - от системного таймера (единственное периодическое прерывание в системе);
 - от устройств ввода/вывода. Прерывание информирует процесс о завершении операции ввода/вывода. При этом, каждая такая операция инициирует сам процессор.
 - от действий оператора (использование горячих клавиш).

Синхронные — возникают в процессе выполнения программы; Асинхронные — не зависят ни от каких действий в системе.

Файловая подсистема `/proc` — назначение, особенности, файлы, поддиректории, ссылка `self`, информация об окружении, состоянии процесса, прерываниях. Структура `proc_dir_entry`: функции для работы с элементами `/proc`. Структура, перечисляющая функции, определённые на файлах. Использование структуры `file_operations` для регистрации собственных функций работы с файлами. Передача данных из пространства пользователя в пространство ядра и из ядра в пространство пользователя. Обоснование необходимости этих функций. Функция `printk` — назначение и особенности. Пример программы «Фортуник» из таб. работы.

Загружаемые модули ядра. Структура загружаемых модулей. Информация о процессах, доступная в ядре: `prio`, `priority`, `state`, `flags`. Пример вывода информации о запущенных процессах, символ `cuser` (таб. раб.). Взаимодействие загружаемых модулей в ядре. Экспорт данных. Пример взаимодействия модулей (таб. раб.). Функция `printk` — назначение и особенности. Виртуальная файловая система

процесс: назначение, примеры директорий и файлов `/proc`, файлы `mapr`, `rapemap`, `tasks`. Регистрация функций работы с файлами `proc`. Пример проинициализированных полей структуры `proc_ops`. Передача данных из пространства ядра в пространство пользователя. Примеры из лабораторной работ.

Открытые флаги: системный вызов `open`: `int open(const char *pathname, int flags); mode_t mode`; посылать сматс параметров. Основные флаги: `O_CREAT`. Реализация системного вызова `open` в системе — действия в ядре: `SYSCALL_DEFINE3(open,...) → ksys_open(filename, flags, mode) → do_sys_open(...) → do_sys_openat2(...) ...` найти наименьший файловый дескриптор... Действия, если флаг `O_CREATE` установлен?

5 / 257

1.4 Прерывания в последовательности ввода-вывода.

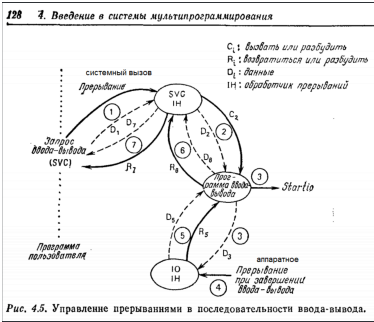


Рис. 4.5. Управление прерываниями в последовательности ввода-вывода.

Схематически показано, что приложение запрашивает данные. Данные приложение может получить не сразу. Здесь включаются различные дополнительные действия, например, ввод данных с клавиатуры или с мыши (мышь тоже устройство ввода/вывода). Для однозначности рассуждений будем рассматривать клавиатуру. Клавиатура — интерактивное внешнее устройство, у него есть контроллер. Клавиатура как большинство внешних устройств подключается к шине через порт клавиатуры, то есть взаимодействие с процессором выполняется по модели `input/output mapping`. При этом выполняется вся последовательность действий, характерных для работы с внешним устройством.

Есть драйвер. Когда приложение запрашивает ввод данных с клавиатуры, ядро системы выполняет целую последовательность действий.

10 / 257

Когда введённые пользователем данные поступят в буфер клавиатуры, контроллер клавиатуры отправит сигнал прерывания на контроллер прерывания. С контроллера этот сигнал по шине управления поступит на ножку процессора и процессор перейдёт на выполнение обработчика прерывания. В результате процесс, запросивший ввод, будет разблокирован. Вся последовательность действий выполняется, когда процесс блокирован на запросе ввода данных. Это блокирующий ввод/вывод, в основном мы имеем дело именно с ним.

Поскольку схема рассматривает ситуацию в самом общем виде, на ней показан `supervisor call interrupt handler` — функция, которая вызывается в режиме ядра в ответ на системный вызов. В общем виде последовательность блокирующего ввода/вывода:

- Запрос ввода/вывода — это системный вызов; при переходе в режим ядра он блокирует приложение, то есть его состояние будет изменено на `D (uninterruptable sleep)`.
- Начиная выполняться код ядра будет вызвана точка входа драйвера устройства, которое идентифицируется запросом. В результате работы кода драйвера будет сформирована команда для устройства, которая по шине данных будет передана контроллеру устройства.
- Отправив команду, процессор отключается от управления устройством — это принцип распараллеливания функций: процессор не управляет работой внешних устройств, им управляют контроллеры устройств. Этот принцип привёл к необходимости включения в систему прерываний прерываний от внешних устройств: они информируют процессор о завершении операции ввода/вывода.
- Когда данные готовы, будет вызван обработчик соответствующего аппаратного прерывания: его задача — считать данные из буфера устройства в буфер ядра и инициализировать следующие действия по обработке полученных данных.
- Обработчик вызовов соответствующего драйвера-функция драйвера, передаст полученные данные, и в результате этих действий приложение будет разблокировано: в буфер приложения (переменной, массива, поле структуры и так далее) будут записаны запрошенные данные.

Важно

В этой модели важно именно то, что процесс блокируется на запросе и разблокируется только тогда, когда данные могут быть процессом получены. Дальше нужно скопировать, но это выполняет обработчик прерывания. Данные готовы — они поступили в буфер ядра.

11 / 257

Важно

Процессор не управляет внешним устройством.

Визу справа – аппаратное прерывание, которое формируется, когда данные в буфере устройства. Из этого буфера данные пройдут несколько этапов прежде чем окажутся в буфере приложения, и приложение сможет продолжить своё выполнение.

Важно

Приложение не может продолжить своё выполнение, пока оно не получит запрошенные данные.

1.5 Быстрые и медленные прерывания

В ОС Linux различают быстрые и медленные прерывания.

1) Быстрые:

Обработка быстрых прерываний выполняется от начала до конца. В современных системах осталось единственное быстрое прерывание – прерывание от таймера, вытеснить его невозможно. При этом выполняется на процессоре 0 и условно этот процессор считается главным.

2) Медленные

Все остальные аппаратные прерывания, то есть прерывания от устройств ввода/вывода (внешних устройств) делится на две части:

2.1) top half (верхняя половина):

2.1.1) interrupt (hardware – аппаратное)

2.2) bottom half (нижняя половина):

2.2.1) softirq

2.2.2) tasklet

2.2.3) workqueue

Главные задачи обработчика прерывания от таймера:

1) Инициализировать планировщик

Станет в очередь на выполнение важные коды:

– Планировщик

– Диспетчер

– swapper

2) Инициализировать диспетчеризацию

12 / 257

Динамическое объявление:

```
1 void tasklet_init(struct tasklet_struct *, void (*)(void)(unsigned long),
2 unsigned long data)
3 {
4 t->next = NULL;
5 t->state = 0;
6 atomic_set(&t->count, 0);
7 t->func = func;
8 t->data = data;
9 }
```

Листинг 3. Динамическая инициализация tasklet

Прежде чем инициализировать, нужно выделить память.

```
1 my_tasklet = (struct tasklet_struct *)kmallo(sizeof(struct tasklet_struct),
2 GFP_KERNEL);
```

В exit нужно:

1) free irq

Нужно освободить линию прерывания от нашего обработчика

2) tasklet_kill

Передаём для данного действия my_tasklet

Планирование Для того чтобы запланировать выполнение tasklet, ядро представляет две функции:

```
1 extern void __tasklet_schedule(struct tasklet_struct *t);
2 static inline void tasklet_schedule(struct tasklet_struct *t)
3 {
4 if ((test_and_set_bit(TASKLET_STATE_SCHED, &t->state))
5 __tasklet_schedule(t);
6 }
7
8 extern void __tasklet_hi_schedule(struct tasklet_struct *t);
9 static inline void tasklet_hi_schedule(struct tasklet_struct *t)
10 {
11 if ((test_and_set_bit(TASKLET_STATE_SCHED, &t->state))
12 __tasklet_hi_schedule(t);
13 }
```

Пример из лабораторной работы (обработчик прерывания клавиатуры с taskletом):

```
1 static struct tasklet_struct *tasklet = NULL;
```

17 / 257

Он не может выполняться долго. Если он выполняется долго, это приведет к снижению отзывчивости системы

1.6 Top half

Это аппаратные прерывания, выполняются на высоких уровнях приоритета. Системы, с которыми мы работаем не поддерживают вложенных прерываний, которые были написаны для IBM/360. Для выполнения аппаратных прерываний установлены следующие ограничения:

Если на процессоре выполняется обработчик аппаратного прерывания, то на этом процессоре запрещены все прерывания. На остальных процессорах запрещены прерывания по линии IRQ, на которой установлен выполняющийся обработчик прерывания. В результате если такой обработчик написан неправильно, выполнятся большой объем кода, это приведет к снижению отзывчивости системы.

Поэтому обработчик аппаратного прерывания должен выполнять минимальный объем действий. Например, если это обработчик прерывания от клавиатуры, то он должен записывать код нажатой клавиши, который поступает из буфера контроллера в буфер ядра, для последующей обработки должен установить задачу для последующего выполнения нижней половины, то есть отложенного действия.

1.7 Обработчики аппаратных прерываний: регистрация, функция и её параметры

Чтобы зарегистрировать в системе наш обработчик прерывания, в ядре имеется функция

```
1 static inline int __must_check request_irq(unsigned int irq, irq_handler_t handler,
2 unsigned long flags, const char *name, void *dev)
3 {
4 return request_threaded_irq(irq, handler, NULL, flags | IRQF_COND_ONESHOT, name,
5 dev);
6 }
```

Параметры:

– irq: номер прерывания. IRQ0 – прерывание от таймера, IRQ1 – от клавиатуры.

– handler: наш обработчик прерывания

– flags: флаги Interrupt ReQuest Flags (IRQF)

– name: имя, которое увидим в /proc/interrupts

Это функция обертка над request_threaded_irq, это базовая функция. При ее вызове в параметр irq_handler_t thread_fn равен NULL.

Флаги:

```
1 #define IRQF_SHARED 0x00000000
2 #define IRQF_PROBE_SHARED 0x00000100
3 #define __IRQF_TIMER 0x00000200
4 #define IRQF_TIMER 0x00000400
5 #define IRQF_NOBALLOON 0x00000800
6 #define IRQF_NO_POLL 0x00001000
7 #define IRQF_ONESHOT 0x00002000
8 #define IRQF_NO_SUSPEND 0x00004000
9 #define IRQF_FORCE_RESUME 0x00008000
10 #define IRQF_NO_THREAD 0x00010000
11 #define IRQF_EARLY_RESUME 0x00020000
12 #define IRQF_COND_SUSPEND 0x00040000
13 #define IRQF_NO_AUTOTEN 0x00080000
14 #define IRQF_NO_DEBUG 0x00100000
15 #define IRQF_COND_ONESHOT 0x00200000
16
17 #define IRQF_TIMER (IRQF_TIMER | IRQF_NO_SUSPEND | IRQF_NO_THREAD)
```

– IRQF_SHARED – Этот флаг как раз и позволяет разделить линию прерывания несколькими обработчиками.

– IRQF_PROBE_SHARED – когда мы «не уверены»?

– IRQF_TIMER – от системного таймера.

– IRQF_NO_THREAD – не может быть параллельно.

Структура, описывающая обработчик:

```
1 struct irqaction {
2 irq_handler_t handler;
3 void *dev_id;
4 void __percpu *percpu_dev_id;
5 struct irqaction *next;
6 irq_handler_t thread_fn;
7 struct task_struct *thread;
8 struct irqaction *secondary;
9 unsigned int irq;
10 unsigned int flags;
11 unsigned long thread_flags;
12 unsigned long thread_mask;
13 const char *name;
14 struct proc_dir_entry *dir;
15 };
```

Поле dir заполняется автоматически на основе параметров, переданных request_irq

14 / 257

dir – pointer to the proc/irq/NN/name entry.

Возвращаемые значения обработчика:

```
1 enum irqreturn {
2 IRQ_NONE = 0 << 0, // прерывания не было или не обработано
3 IRQ_HANDLED = 1 << 0, // прерывание обработано
4 IRQ_WAKE_THREAD = 1 << 1, // запрос пробуждения потока мейндера
5 };
6
7 typedef enum irqreturn irqreturn_t;
```

Пример из JIP:

```
1 static irqreturn_t irq_handler(int irq, void *dev_id)
2 {
3 int code;
4 int is_release;
5 int press_code;
6
7 if (irq == IRQ_NUM) {
8 code = inb(0x60);
9 is_release = (code & 0x80) ? 1 : 0;
10 press_code = code & 0x7f;
11
12 if (is_release && press_code < 84 && press_code != 28) {
13 irq_time = ktime_get();
14 tasklet->data = (unsigned long)code;
15 tasklet_schedule(tasklet);
16 return IRQ_HANDLED;
17 }
18 }
19 return IRQ_NONE;
20 }
```

Когда драйвер загружается, необходимо отменить регистрацию соответствующего обработчика прерывания. Для этого существует функция:

```
1 void free_irq(unsigned int irq, void *dev_id);
```

Если указанная линия прерывания не является разделяемой, то функция free_irq() удалит обработчик и отключит линию. Если линия прерывания разделяется, то обработчик, определённый dev_id удалится, но линия прерывания будет отключена только при удалении последнего обработчика.

15 / 257

1.8 Tasklety: объявление, планирование (пример из лаб. раб.)

Tasklety – частный случай softirq. softirq одного типа может выполняться параллельно, поэтому требуются средства взаимного исключения. Tasklet одного типа может выполняться только на одном процессоре – средства взаимного исключения не требуются. Разные tasklety могут выполняться параллельно. Tasklet – компромисс между производительностью и простотой использования.

softirq написаны таким образом, что они могут выполняться реально параллельно. Это freebty, чтобы код был регенерируемым и в них реализовывался взаимное исключение.

```
1 struct tasklet_struct {
2 struct tasklet_struct *next; // указатель на следующий tasklet в списке
3 unsigned long state;
4 atomic_t count;
5 bool use_callback;
6 union {
7 void (*func)(unsigned long data); // функция-обработчик taskleta
8 void (*callback)(struct tasklet_struct *t);
9 };
10 unsigned long data; // аргумент функции-обработчика
11 };
12
13 enum {
14 TASKLET_STATE_SCHED, /* Tasklet is scheduled for execution */
15 TASKLET_STATE_RUN, /* Tasklet is running (SMP only) */
16 };
17
18 Листинг 1. Структура tasklet_struct
```

Состоянии: 0, TASKLET_STATE_SCHED, TASKLET_STATE_RUN, TASKLET_STATE_RUN имеют смысл только в SMP-архитектуре. Tasklet может выполняться только если count == 0.

Статическое объявление:

```
1 DECLARE_TASKLET(name, func, /* старшее, сойказ 3-го параметра нет */ data);
2
3 #define DECLARE_TASKLET(name, _callback) \
4 struct tasklet_struct name = { \
5 .count = ATOMIC_INIT(0), \
6 .callback = _callback, \
7 .use_callback = true, \
8 };
```

Листинг 2. Статическая инициализация tasklet

16 / 257

2 Управление внешними устройствами: специальные файлы устройств, идентификация внешних устройств в системе (тип dev_t). Система прерываний: типы прерываний и их особенности. Быстрые и медленные прерывания. Обработка аппаратных прерываний: регистрация в системе. Линия IRQ (простейшая компьютерная схема – три шина). Верхние и нижние softirq – перечисления и функция ядра для выполнения softirq, tasklety и очереди работ. К softirq – схема обслуживания входящих сетевых пакетов. Softirq и tasklet: объявление, инициализация, планирование taskleton (примеры из лаб раб).

Существует два типа внешних устройств:

– Байт-ориентированные (или символьные) устройства передают данные посылкой, как непрерывный поток байтов (например, мышь, клавиатура).

– Блок-ориентированные (или блочные) устройства (например, жёсткий диск) передают данные блоками. Все запоминающие устройства относятся к блочным.

Чтобы управлять всем разнообразием внешних устройств, в Unix/Linux реализована концепция драйверов. Она заключается в том, что устройства должны быть зарегистрированы в системе и в системе существует набор драйверов, которые можно использовать для управления новыми устройствами. Драйвер – Это часть кода ядра, которая предназначена для управления конкретным внешним устройством.

1) В Linux используется парадигма «все файл», поэтому устройство – тоже файл. Но в полном объеме невозможно взаимодействовать как с обычными файлами. Устройства в системе должны идентифицироваться. Для этого в системе определен специальный подход. Файлы внешних устройств находятся в директории /dev. В ней с помощью ls -l видим внешние устройства.

```
1 $ ls -l /dev
2 crw-rw-rw- 1 root root 1, 3 Apr 11 2002 null
3 crw-rw-rw- 1 root root 10, 1 Apr 11 2002 psmu
4 crw-rw-rw- 1 root root 4, 1 Oct 28 03:04 tty1
5 crw-rw-rw- 1 root tty 4, 64 Apr 11 2002 ttyS0
6 crw-rw-rw- 1 root uucp 4, 60 Apr 11 2002 ttySL
7 crw-rw-rw- 1 vcsa tty 7, 1 Apr 11 2002 vcsa1
8 crw-rw-rw- 1 vcsa tty 7, 129 Apr 11 2002 vcsaL
9 crw-rw-rw- 1 root root 1, 5 Apr 11 2002 zero
```

Старший номер идентифицирует драйвер, связанный с устройством. Например, /dev/null и /dev/zero управляются драйвером 1, виртуальные терминалы – драйвером 4. Младший номер используется для различия конкретных устройств в рамках одного драйвера (например, жёсткий диск имеет один старший номер, а его разделы – разные младшие).

21 / 257

Стандарт POSIX-1 оговаривает существование типа `dev_t` – 32-разрядное число, в котором 12 бит отведены под старший номер, 20 – под младший. Код не должен делать предположений об организации номеров внутри числа. Для работы с `dev_t` используются макросы:

```
1 MAJOR(dev_t dev) // получить старший номер
2 MINOR(dev_t dev) // получить младший номер
```

Имея старший и младший номера, можно их объединить в значение типа `dev_t`.

```
1 MAKEDEV(major, int minor) // объединить в dev_t
```

2.1 Выделение и освобождение номеров устройств

Одно из первых действий, которые должен сделать драйвер, когда устанавливает символическое устройство, он должен получить 1 или более номеров устройств, чтобы работать с ними, чтобы идентифицировать свой `char device`. Для этого вызывается функция

```
1 int register_chrdev_region(dev_t first, unsigned int count, char *name);
```

– `first` – начальный номер устройства доступного для выделения диапазона

Часто минорная часть значения `first` равна нулю. Но этот счёт не никаких определённых требований.

– `count` – общее количество последовательных номеров устройств, которые могут быть запрошены. При этом значение `count` может превышать размер запрашиваемого диапазона. В этом случае будет переход на следующий старший номер, ошибка не возникает, если запрашиваемый диапазон доступен.

– `name` – имя устройства, которое связывается с диапазоном. Это можно увидеть или в `/proc/devices` или в `sysfs (/sys)`.

При успешном выделении функция возвращает 0.

Часто неизвестно значение старшего номера устройства, поэтому разработчики ядра всё время стремятся перейти на динамическое распределение номеров устройств. Для динамического выделения используется функция

```
1 int alloc_chrdev_region(dev_t *dev, unsigned int firstminor, unsigned int count, char *name);
```

– `dev` – значение указателя при успешном завершении будет содержать первый номер в определённом диапазоне. Обычно указывают 0

```
5 };
6
7 typedef enum irqreturn irqreturn_t;
```

Когда драйвер выражается, необходимо отменить регистрацию соответствующего обработчика прерывания. Для этого существует функция:

```
1 void free_irq(unsigned int irq, void *dev_id);
```

Если указанная линия прерывания не является разделёмой, то функция `free_irq()` удаляет обработчик и отключает линию. Если линия прерывания разделёная, то обработчик, определённый `dev_id` удаётся. Если линия прерывания будет отключена только при удалении последнего обработчика.

2.7 softirq (гибкие прерывания)

`softirq` (гибкие прерывания) – отложенные действия, определяются статически при компиляции. `softirq` инициализируется при загрузке системы. Если надо написать свой собственный `softirq`, то необходимо перекомпилировать ядро (помнить `enum`). Из версии ядра в версию ядра (как `выключение`) разработчики переименовуют: «пожалуйста, откажитесь от выделения нового `softirq`, если действительно не нужна высокая частота `threaded job scheduling`».

В настоящее время в системе определено 10 `softirq`.

```
1 enum
2 {
3     HI_SOFTIRQ = 0,
4     TIMER_SOFTIRQ,
5     NET_TX_SOFTIRQ,
6     NET_RX_SOFTIRQ,
7     BLOCK_SOFTIRQ,
8     IRQ_POLL_SOFTIRQ,
9     TASKLET_SOFTIRQ,
10    SCHED_SOFTIRQ,
11    HRTIMER_SOFTIRQ,
12    RCU_SOFTIRQ, /* Preferable RCU should always be the last softirq */
13    NR_SOFTIRQS
14 };
```

Индекс	Приоритет	Назначение
HI_SOFTIRQ	0	Высокоприоритетные <code>softirq</code>
TIMER_SOFTIRQ	1	Таймер
NET_TX_SOFTIRQ	2	Отправка сетевых пакетов

Остальные параметры аналогичны рассмотренной ранее функции.

Номера освобождается функцией

```
1 void unregister_chrdev_region(dev_t first, unsigned int count);
```

Все эти функции вызываются в модуле в функции `init`, а последняя функция – в функции `exit`.

2.2 Система прерываний: типы прерываний и их особенности.

Три типа событий, передающих систему в режим ядра:

- 1) Системные вызовы (программные прерывания) – процесс переключается в режим ядра и выполняет реентерабельный код ядра, после переключается в режим пользователя.
- 2) Исключения – синхронные. Два типа: исправимые (страничное прерывание, после обработки программа продолжает выполнение) и неисправимые (программа завершается аварийно).
- 3) Аппаратные прерывания – асинхронные. Три типа:
 - 3.1) от системного таймера (единственное периодическое прерывание в системе);
 - 3.2) от устройств ввода/вывода. Прерывание информирует процесс о завершении операции ввода/вывода. При этом, каждую такую операцию инициирует сам процессор.
 - 3.3) от действий оператора (использование горячих клавиш).

Синхронные – возникают в процессе выполнения программы; Асинхронные – не зависят ни от каких действий в системе.

2.3 Быстрые и медленные прерывания

В ОС Linux различают быстрые и медленные прерывания:

- 1) **Быстрые:** Обработка быстрых прерываний выполняется от начала до конца. В современных системах осталось единственное быстрое прерывание – прерывание от таймера, вытеснить его невозможно. При этом выполняется на процессоре 0 и условно этот процессор считается главным.
- 2) **Медленные:** Все остальные аппаратные прерывания, то есть прерывания от устройств ввода/вывода (внешних устройств), делятся на две части:
 - 2.1) top half (верхняя половина)
 - 2.1.1) interrupt (hardware – аппаратно)
 - 2.2) bottom half (нижняя половина):
 - 2.2.1) `softirq`

Индекс	Приоритет	Назначение
NET_RX_SOFTIRQ	3	Принём сетевых пакетов
BLOCK_SOFTIRQ	4	Блоковые устройства
IRQ_POLL_SOFTIRQ	5	Опрос блочного устройства
TASKLET_SOFTIRQ	6	Тасклет
SCHED_SOFTIRQ	7	Планировщик
HRTIMER_SOFTIRQ	8	Не используется
RCU_SOFTIRQ	9	Обязательно должен быть последним

Таблица 1. Типы `softirq`

```
1 const char * const softirq_to_name[NR_SOFTIRQS] = {
2     "HI", "TIMER", "NET_TX", "NET_RX", "BLOCK", "IRQ_POLL",
3     "TASKLET", "SCHED", "HRTIMER", "RCU"
4 };
```

Размер вектора 32, но реально определено 10. Меньшие номера имеют более высокий приоритет.

`softirq_NET_TX_SOFTIRQ` и `softirq_NET_RX_SOFTIRQ` относятся к сетевой подсистеме. За выполнение `softirq` отвечает `softirqd`.

Когда возникает необходимость выполнения `softirq` – соответствующее аппаратное прерывание отключает `bottom half pending`, `ksftirqd` проверяет эту маску и если в ней была занесена соответствующая единица, запускает на выполнение соответствующий `softirq` (do `softirq`).

`softirq` одного типа может выполняться параллельно на разных процессорах одновременно (сервализовано). Это сделано с целью ускорения обслуживания соответствующих аппаратных подсистем – ярким примером является сетевая подсистема. Поэтому в коде `softirq` должно быть реализовано взаимное исключение в полном объёме, код должен быть реентерабельным.



`softirq` никогда не вытеснит другой `softirq`. Единственное событие, которое может вытеснить `softirq` – это аппаратное прерывание.

Функции для работы с `softirq`:

```
1 void open_softirq(int nr, void (*action)(void)); // регистрация обработчика
2 void raise_softirq(unsigned int nr); // планирование softirq на текущем CPU
```

2.2.2 tasklet 2.2.3 workqueue

Пример быстрого прерывания – прерывание от системного таймера. Главные задачи его обработчика:

- 1) Инициализировать планировщик.
- 2) Ставить в очередь на выполнение очень важные коды:
 - Диспетчер
 - `swapper`
- 2) Инициализировать диспетчеризацию.

Он не может выполняться долго. Если он выполняется долго, это приведёт к снижению отзывчивости системы.

2.4 Прерывания в последовательности ввода-вывода.

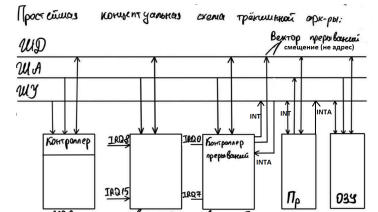


Рис. 2. Простейшая концептуальная схема

В компьютерах передаются сигналы трёх типов: данные (данные и команды), адреса, сигналы управления – отсюда три шины: шина данных, шина адреса, шина управления.

Обработка запроса прерывания по этой схеме:

- 1) На линию IRQ приходит сигнал от внешнего устройства.

- 2) Когда на контроллер прерываний приходит прерывание, он формирует сигнал INT, который по шине управления поступает на выделенную ножку процессора.
- 3) В конце цикла выполнения команды процессор проверяет наличие сигнала на выделенной ножке.
- 4) Если сигнал пришёл – процессор посылает контроллеру сигнал INTA (подтверждение).
- 5) Получив INTA, контроллер по шине данных выставляет вектор прерывания (базовый вектор + номер линии IRQ).
- 6) Процессор считывает вектор прерывания, переводит его смещение по таблице и считает адрес обработчика.

2.5 Регистрации аппаратных прерываний в системе

2.6 Обработчики аппаратных прерываний: регистрация, функция и её параметры

Чтобы зарегистрировать в системе наш обработчик прерывания, в ядре имеется функция

```
1 static inline int __must_check request_irq(unsigned int irq, irq_handler_t handler,
2 unsigned long flags, const char *name, void *dev)
3 {
4     return request_threaded_irq(irq, handler, NULL, flags | IRQF_COND_ONESHOT, name,
5 dev);
6 }
```

Параметры:
– `irq` номер прерывания. IRQ0 – прерывание от таймера, IRQ1 – от клавиатуры.
– `handler`: наш обработчик прерывания
– `flags`: флаги Interrupt ReQuest Flags (IRQF)
– `name`: имя, которое увидим в `/proc/interrupts`

Эта функция обёртка над `request_threaded_irq`, это базовая функция. При её вызове в параметр `irq_handler_t thread_fn` равен NULL.

Флаги:

```
1 #define IRQ_SHARED 0x00000000
2 #define IRQ_PROBE_SHARED 0x00000000
3 #define IRQ_TIMER 0x00000200
4 #define IRQ_PERCPU 0x00000400
5 #define IRQ_NOBALANCE 0x00000800
6 #define IRQ_NOPOLL 0x00001000
7 #define IRQ_ONESHOT 0x00002000
8 #define IRQ_NO_SUSPEND 0x00004000
```

```
9 #define IRQ_FORCE_RESUME 0x00008000
10 #define IRQ_NO_THREAD 0x00010000
11 #define IRQ_EARLY_RESUME 0x00020000
12 #define IRQ_COND_SUSPEND 0x00040000
13 #define IRQ_NO_AUTEN 0x00080000
14 #define IRQ_NO_FLAGS 0x00100000
15 #define IRQ_COND_ONESHOT 0x00200000
16
17 #define IRQ_TIMER (IRQ_TIMER | IRQF_NO_SUSPEND | IRQF_NO_THREAD)
```

– `IRQ_SHARED` – Этот флаг как раз и позволяет разделить линию прерывания несколькими обработчиками.
– `IRQ_PROBE_SHARED` – когда мы «не уверены»?
– `__IRQ_TIMER` – от системного таймера.
– `IRQF_NO_THREAD` – не может быть распараллелен.

Структура, описывающая обработчик:

```
1 struct irqaction {
2     irq_handler_t handler;
3     void *dev_id;
4     void (*percpu) *percpu_dev_id;
5     struct irqaction *next;
6     irq_handler_t thread_fn;
7     struct task_struct *thread;
8     struct irqaction *secondary;
9     unsigned int irq;
10    unsigned int flags;
11    unsigned long thread_flags;
12    unsigned long thread_mask;
13    const char *name;
14    struct proc_dir_entry *dir;
15 };
```

Поле `dir` заполняется автоматически на основе параметров, переданных `request_irq`.

`dir` – pointer to the `proc/irq/NN/name` entry.

Возвращаемые значения обработчика:

```
1 enum irqreturn {
2     IRQ_NONE = 0, // прерывание не было или не обработано
3     IRQ_HANDLED = 1, // прерывание обработано
4     IRQ_WAKE_THREAD = 2, // запрос пробуждения потока кедрера
5 }
```

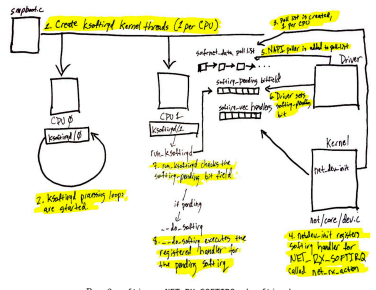


Рис. 3. `softirq` – `NET_RX_SOFTIRQ` и `ksoftirqd`

2.8 Схемы обслуживания входящих сетевых пакетов (NET_RX_SOFTIRQ)

- НАРІ – New API. Сети были описаны очень давно, никто не мог предполагать современное состояние сети, когда передаются огромные потоки данных. В частности, протокол TCP стал проблемой сетей: возникли DDoS-атаки, они работают из-за особенностей протокола TCP. Возможна ситуация, когда на определённый хост поступает большое количество пакетов с большой частотой. Каждый приход пакета – обработка прерывания, что является затратным действием: выполняется на самом высоком уровне привилегий, и, обрабатывая пакеты, система может потерять работоспособность. Поэтому был написан НАРІ – он переводит сетевую систему в режим опроса (poll). Основная цель НАРІ – сократить число прерываний при приёме пакетов.
- Схема обслуживания входящих сетевых пакетов (NET_RX_SOFTIRQ):
- 1) При загрузке системы (`smnboot`) создаётся по одному потоку ядра `ksoftirqd` на каждый CPU.
 - 2) Запускаются `processing loop` и `ksoftirqd` – потоки входят в цикл обработки.
 - 3) Создается `softnet_data poll list` – по одному на каждый CPU.
 - 4) `net_dev_init (net/core/dev.c)` регистрирует обработчик `softirq` для `NET_RX_SOFTIRQ` – функцию `net_rx_action`.
 - 5) Когда подключается сетевое устройство, его NAPI poller добавляется в `poll list`.
 - 6) Приходит пакет → драйвер устанавливает бит в `softirq_pending` bitfield.
 - 7) `run_ksoftirqd` проверяет `softirq_pending` bitfield – если бит выставлен (if pending), передаёт управление `_do_softirq`.
 - 8) `_do_softirq` выполняет зарегистрированный обработчик для pending `softirq` – то есть вызывает `net_rx_action`, которая обходит `poll list` и опрашивает драйверы на наличие пакетов.

Пояснение от Рязановой: Для каждого процессора имеется свой `softirqd`. Чтобы он начал работу вызывается функция `run_ksoftirqd` («Дать возможность»). Если соответствующее битовое поле включено для соответствующего `softirq`, то вызывается функция `do_softirq`. Она выполняет зарегистрированный хендлер для добавленного `softirq`. Эту информацию задал драйвер.

2.9 Тасклеты

Тасклеты не сервализуются: один тип тасклеты может выполняться только на одном процессоре одновременно, т.е. средства взаимного исключения не требуются. Разные тасклеты могут выполняться параллельно. Тасклет – компромисс между производительностью и простотой использования.

```
1 struct tasklet_struct {
2     struct tasklet_struct *next; // указать на следующий тасклет в списке
3     unsigned long state;
4     atomic_t count;
5     bool use_callback;
6     union {
7         void (*func)(unsigned long data); // функция-обработчик тасклета
8         void (*callback)(struct tasklet_struct *t);
9     };
10    unsigned long data; // аргумент функции-обработчика
11 };
12
13 enum {
14     TASKLET_STATE_SCHED, /* Tasklet is scheduled for execution */
15     TASKLET_STATE_RUN /* Tasklet is running (SMP only) */
16 };
```

Листинг 5. Структура `tasklet_struct`

Состояния: 0, TASKLET_STATE_SCHED, TASKLET_STATE_RUN. `TASKLET_STATE_RUN` имеет смысл только в SMP-архитектуре. Тасклет может выполняться только если `count == 0`.

Статическое объявление:

```
1 DECLARE_TASKLET(name, func, /* устаревшее, сейчас 3-го параметра нет */ data);
2
3 #define DECLARE_TASKLET(name, callback) \
4 struct tasklet_struct name = { \
5     .count = ATOMIC_INIT(0), \
6     .callback = callback, \
7     .use_callback = true, \
8 }
```

8)

Листинг 6. Статическая инициализация tasklet

Динамическое объявление:

```
1 void tasklet_init(struct tasklet_struct *t, void (*func)(unsigned long),
2 unsigned long data)
3 {
4     t->next = NULL;
5     t->state = 0;
6     atomic_get(&count, 0);
7     t->func = func;
8     t->data = data;
9 }
```

Листинг 7. Динамическая инициализация tasklet

Прежде чем инициализировать, нужно выделить память.

```
1 my_tasklet = (struct tasklet_struct *)kmalloc(sizeof(struct tasklet_struct),
2 GFP_KERNEL);
```

В exit нужно:

- 1) free irq
 - 2) tasklet_kill
- Перед тем для данного действия my_tasklet

Планирование Для того чтобы запланировать выполнение tasklet'a, ядро предоставляет две функции:

```
1 extern void __tasklet_schedule(struct tasklet_struct *t);
2
3 static inline void tasklet_schedule(struct tasklet_struct *t)
4 {
5     if (!test_and_set_bit(TASKLET_STATE_SCHED, &t->state))
6         __tasklet_schedule(t);
7 }
8
9 extern void __tasklet_hi_schedule(struct tasklet_struct *t);
10
11 static inline void tasklet_hi_schedule(struct tasklet_struct *t)
12 {
13     if (!test_and_set_bit(TASKLET_STATE_SCHED, &t->state))
14         __tasklet_hi_schedule(t);
15 }
```

32 / 257

Пример из лабораторной работы (обработка прерывания клавиатуры с taskletом):

```
1 static struct tasklet_struct *tasklet = NULL;
2 static irqreturn_t interrupt_handler(int irq, void *dev_id) {
3     if (irq == IRQ_NO) {
4         start_time = ktime_get();
5         tasklet_schedule(tasklet);
6         return IRQ_HANDLED;
7     }
8     return IRQ_NONE;
9 }
10
11 static int __init my_init(void) {
12     int ret = request_irq(IRQ_NO, interrupt_handler, IRQ_SHARED,
13         "interrupt_handler_tasklet", (void *)(&interrupt_handler));
14     if (ret) return ret;
15     tasklet = kmalloc(sizeof(struct tasklet_struct), GFP_KERNEL);
16     if (!tasklet) {
17         free_irq(IRQ_NO, (void *)(&interrupt_handler));
18         return -ENOMEM;
19     }
20     tasklet_init(tasklet, my_tasklet_fun, 0);
21     return 0;
22 }
23
24 static void __exit my_exit(void) {
25     tasklet_kill(tasklet);
26     kfree(tasklet);
27     free_irq(IRQ_NO, (void *)(&interrupt_handler));
28 }
```

Пример из лабораторной работы (обработка прерывания клавиатуры с workqueue):

```
1 static irqreturn_t irq_handler(int irq, void *dev_id) {
2     if (irq == IRQ_NO) {
3         start_time = ktime_get();
4         tasklet_schedule(tasklet);
5         return IRQ_HANDLED;
6     }
7     return IRQ_NONE;
8 }
9
10
```

37 / 257

2.10 Очереди работ

Альтернатива tasklet'ам. Tasklet выполняется в контексте прерывания, код tasklet'a неидеален. Работа выполняется в контексте специального потока ядра и может быть заблокирована. Tasklet всегда выполняется на процессоре, на котором завершилась обработка прерывания. Работу можно выполнить на другом процессоре и отложить на некоторое время.

Основные понятия все между собой связаны. Везде чаще встречаются сокращения имен.

- 1) **work**
Для work определяется функция, которая будет выполнять соответствующие действия.

- 2) **workqueue**
work необходимо поставить в очередь работ, которую мы для work создаём.

- 3) **worker (work_thread)**
Поток ядра.

- 4) **worker_pool**
отношение worker_pool и worker — 1 к 1

- 5) **pool_workqueue (pq)**
Устанавливает отношение между workqueue и worker_pool, причём — workqueue и pq — один ко многим
— pq и worker_pool — один к одному

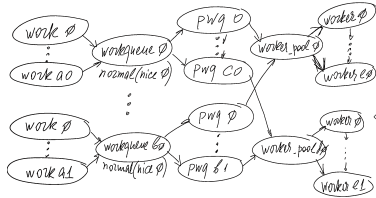


Рис. 4. Очереди работ

— work ставится в workqueue

2.11 worker

worker — поток ядра.

Все задействованные worker'и рано или поздно будут обслужены

Пузы worker'ов — per CPU. Мы это видели в первом модуле. Мы видели, что их определено много для каждого CPU. (Не писать, но может спросить, а думаю)

Смысл картинки: всё per CPU, потому определены эти пузы, сосредоточение worker'ов. Главное, что work ставится в workqueue, а он в итоге будет выполняться worker'ом, который принадлежит worker_pool

```
1 struct worker {
2     ...
3     struct work_struct *current_work; /* K: work being processed and its */
4     work_func_t current_func; /* K: function */
5     struct pool_workqueue *current_pq; /* K: pq */
6     ...
7     int sleeping; /* S: is worker sleeping? */
8     ...
9     struct list_head scheduled; /* L: scheduled works */
10    struct task_struct *task; /* L: worker task */
11    struct worker_pool *pool; /* A: the associated pool */
12    ...
13 }
```

34 / 257

```
55 kfree(work2);
56 printk(KERN_INFO "irq_workqueue: unloaded\n");
57 }
```

39 / 257

```
13 unsigned int flags; /* L: flags */
14 int id; /* I: worker id */
15 ...
16 struct workqueue_struct *rescue_wq; /* I: the workqueue to rescue */
17 };
```

Листинг 9. Структура worker

— Работа, которая будет выполнена рабочим потоком. Смысл этого потока в том, что она планируется планировщиком ядра. Мы эту информацию получали, когда выводили task_struct

```
1 struct worker_pool {
2     ...
3     int cpu; /* I: the associated cpu */
4     ...
5     int id; /* I: pool id */
6     unsigned int flags; /* L: flags */
7     ...
8     int nr_running;
9     int nr_workers; /* L: total number of workers */
10    int nr_idle; /* L: currently idle workers */
11
12    struct list_head idle_list; /* L: list of idle workers */
13    struct timer_list idle_timer; /* L: worker idle timeout */
14    struct work_struct idle_call_work; /* L: worker idle cleanup */
15
16    struct timer_list mayday_timer; /* L: SOS timer for workers */
17
18    /* A workers is either on busy_hash or idle_list, or the manager */
19    DECLARE_HASHTABLE(busy_hash, BUSY_WORKER_HASH_ORDER);
20    /* L: hash of busy workers */
21
22    struct worker *manager; /* L: purely informational */
23    struct list_head workers; /* A: attached workers */
24
25    struct ida worker_ids; /* K: worker IDs for task name */
26
27    struct workqueue_attrs *attrs; /* I: worker attributes */
28    struct hlist_node hash_node; /* PL: unbound_pool_hash node */
29    int refcnt; /* PL: refcnt for unbound pools */
30    #ifdef CONFIG_PREEMPT_RT
31    spinlock_t cb_lock; /* BH worker cancel lock */
32    #endif
33 };
```

35 / 257

```
34 * destruction of pool is RCU protected to allow dereferences
35 * from get_work_pool().
36 */
37 struct rcu_head rcu;
38 };
```

Листинг 10. Структура worker_pool

— Раз это пул, значит, самое главное — списки.

— nr_workers, nr_idle
Нужны для работы планировщика. Планировщик должен предоставить всем процессам примерно одинаковое процессорное время, то есть дать возможность выполняться всем процессам.

```
1 // статическая инициализация
2 DECLARE_WORK(name, void (*func)(void *));
3
4 // динамическая инициализация
5 INIT_WORK(struct work_struct *work, void (*func)(void *), void *data);
6
7 // поставлена в очередь
8 int queue_work(struct workqueue_struct *wq, struct work_struct *work);
9 int queue_work_on(int cpu, struct workqueue_struct *wq, struct work_struct *work);
10 int queue_delayed_work(struct workqueue_struct *wq, struct work_struct *work,
11     unsigned long delay);
12
13 void destroy_workqueue(struct workqueue_struct *wq);
```

Критерий сравнения	tasklet	workqueue
Контекст выполнения	Контекст прерывания того обработчика, который поставил его на выполнение. Обычно tasklet выполняется сразу после первой половины.	Специальный поток ядра. Планирует его стандартный обработчик.
Возможность блокироваться	—	+
Процессор, на котором выполняется	Только на том, который и поставил tasklet на выполнения	По умолчанию как у tasklet, но это можно настроить с помощью флага WQ_UNBOUND

Таблица 2. Сравнение очередей работ в tasklets

36 / 257

3 Сокеты как средство взаимодействия параллельных процессов. Сокеты семейства AF_UNIX и AF_INET. Сокеты в пространстве файловых имен. Сокеты AF_INET, сетевой стек. Классификация моделей ввода-вывода. Мультиплексирование при взаимодействии процессов в распределённых системах по модели клиент сервер. Ввод-вывод с мультиплексированием: мультиплексор epoll — особенности (типы и алгоритм работы). Пример из лабораторной работы — сервер производства-потребление с мультиплексированием.

3.1 Сокеты как средство взаимодействия параллельных процессов

Сокет — абстракция точки соединения. Являются универсальным средством меж-процессного взаимодействия: могут служить как для взаимодействия процессов на отдельной стоящей машине, так и для взаимодействия в распределённых системах (по сети). Были разработаны Unix BSD, поэтому часто называются сокетами Беркли или сокетами BSD.

Определение 3.1 (Сокет)

Сокет — абстракция точки соединения.

Абстракция от особенностей взаимодействия. Средство универсальное: на конкретной машине особенности определяются тем, что это процессор и память, а в распределённой системе — тем, что память распределённая.

В компьютере обращение к данным всегда выполняется по адресу, но на отдельно стоящей машине один способ обращения, а в распределённых системах — другой, поскольку нужно иметь адреса хостов.

Разработаны парные сокеты — отдельный системный вызов socketpair(). Парные сокеты сделаны как альтернатива pipe, но в отличие от последних, которые обеспечивают симметричную связь, сокеты имеют асимметричную связь.

Все остальные сокеты создаются системным вызовом socket().

```
1 int socket(int family, int type, int protocol);
```

Системный вызов socket() создаёт точку соединения и возвращает файловый дескриптор, который ссылается на эту точку. При успешном вызове возвращает файловый дескриптор с наименьшим номером дескриптора файла, который в данный момент еще не открыт для процесса.

3.2 Сокеты семейства AF_UNIX и AF_INET

Параметры системного вызова socket().

41 / 257

- 1) family – пространство (семейство) адресов.
- 1.1) AF_UNIX – сокеты для взаимодействия параллельных процессов на отдельных стоящих машинах в пространстве файловых имён.
- 1.2) AF_INET – сокеты семейства протоколов TCP/IP основанные на взаимодействии процессов в сети интернет IPv4
- 1.3) AF_INET6 – IPv6
- 1.4) AF_IPX
- 1.5) AF_UNSPEC

- 2) type – тип сокета. В зависимости от того, где используется сокет, у него будет соответствующий тип.
- 2.1) SOCK_STREAM – определяет ориентированное на потоки надёжное упорядоченное полудуплексное логическое соединение двух сокетов (между двумя socketами).
- 2.2) SOCK_DGRAM – определяет ненадёжную службу доставки (datagram) без установления логического соединения, где пакеты могут передаваться без сохранения порядка. Это широкосетчатая передача данных.
- 2.3) SOCK_RAW – прямой (сырой, низкоуровневый) сокет.

- 3) protocol – определяет конкретный протокол, который будет использоваться с сокетом. Обычно параметр устанавливается равным 0, тогда протокол задаётся по умолчанию. Протоколы обозначаются символическими константами с префиксами IPPROTO_*, например IPPROTO_TCP.
- По умолчанию для сокетов AF_INET типа SOCK_STREAM будет установлен протокол TCP, для этих же сокетов типа SOCK_DGRAM – протокол UDP.

Таким образом, сокеты AF_UNIX используются для взаимодействия параллельных процессов на одной стоящей машине, а сокеты AF_INET – для взаимодействия процессов в распределённой системе (в сети).

3.3 Сокеты в пространстве файловых имён

Сокеты BSD поддерживают множество протоколов, поэтому на сокетах была определена общая структура struct sockaddr:

```
1 struct sockaddr
2 {
3     sa_family_t sa_family; // AF_XXX
4     char sa_data[14];
5 };
```

Листинг 11. Структура sockaddr

42 / 257

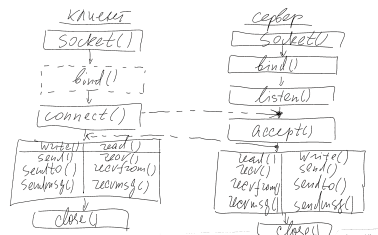


Рис. 6. Клиент-сервер на сокетах

Системный вызов socket() создаёт сокет и возвращает файловый дескриптор сокета. Затем на стороне сервера в обязательном порядке вызывается системный вызов bind(), который связывает файловый дескриптор сокета с адресом, то есть до bind() необходимо инициализировать поля структуры struct sockaddr_in. После этого на стороне сервера вызывается системный вызов listen(), который переводит серверный процесс в режим пассивного ожидания соединения. Когда соединение произошло, то есть клиент на своей стороне выполнил системный вызов connect(), ядро выполняет предварительную регистрацию соединения и выполняется системный вызов accept(). После этого начинается обмен данными.

```
1 #include <sys/socket.h>
2 int bind(int sockfd, struct sockaddr *addr, int addrlen);
3 int listen(int sockfd, int backlog);
4 int connect(int sockfd, struct sockaddr *serv_addr, int addrlen);
5 int accept(int sockfd, void *addr, int *addrlen);
```

Листинг 16. Объявления функций

Системный вызов bind(). Назначает сокету sockfd адрес, указанный addr; параметр addrlen – размер в байтах структуры адреса, на которую указывает addr. Для сетевых сокетов адрес состоит из IP-адреса сетевого интерфейса локальной

47 / 257

машины и номера порта. Клиенты могут не вызывать bind(), так как их адрес часто никакой роли не играет – в таком случае адрес назначается им автоматически.

Системный вызов listen(). На стороне сервера переводит сервер в режим пассивного прослушивания. Сервер вызывает listen() для того, чтобы проинформировать операционную систему, что на сокет нужно принять соединение. Это имеет смысл только для ориентированных на соединение протоколов (TCP). Параметр backlog определяет максимальную длину, до которой может вырасти очередь ожидающих соединений для sockfd.

Системный вызов connect(). Вызывается на стороне клиента, чтобы создать активное соединение. Клиент должен знать адрес сервера (serv_addr – адрес сервера, нужно обратить на это внимание). Для протокола TCP активное соединение устанавливается по указанному адресу. Для протокола UDP connect() может использоваться для указания адреса назначения для всех передаваемых впоследствии пакетов.

Системный вызов accept(). Выполняется на стороне сервера для принятия соединения при условии, что ранее поступил запрос на соединение. В противном случае accept() будет блокирован до поступления запроса на соединение.

Системный вызов accept(). Возвращает файловый дескриптор созданной копии сокета, то есть нового сокета. Такое дублирование сокетов в ходе принятия соединений даёт серверу возможность продолжать принимать новые соединения без необходимости предварительно закрывать предыдущие.

Сервер блокируется на системном вызове accept() до тех пор, пока не поступит запрос на соединение от клиента.

Для протокола TCP определено тройное рукопожатие (cint: SYN, srv: SYN-ACK, clnt: ACK): сначала клиент обращается к серверу с запросом на установление соединения; получив первое ответное сообщение, клиент так же отправляет подтверждение получения, и только после этого соединение устанавливается. В настоящее время тройное рукопожатие стало огромной проблемой – DDOS-атаки.

Начего не конкретизировано, просто 14 байт. Структура определена в таком виде, потому что разработчики понимали, что в разных системах нужна разная адресация.

Для сокетов AF_UNIX в поле sa_data помещается имя файла, поскольку эти сокеты взаимодействуют через пространство файловых имён

В системе сокет определён как открытый файл – системный вызов socket() возвращает файловый дескриптор. struct file описывает открытый файл, в самой структуре есть указатель на struct inode, которая описывает физический файл. struct inode в Unix описывает большое количество разных типов файлов – в том числе сокеты в пространстве файловых имён, программные каналы и т.д.

Сокет сервера можно показать в файловой подсистеме: командой ls -la выводятся файл сокета с его inode, тип файла обозначается буквой s.

3.4 Сокеты AF_INET, сетевой стек

Для сетей адрес в виде структуры struct sockaddr не подходит, мы должны указать сетевой адрес и номер порта, поэтому объявлена структура struct sockaddr_in для IPv4. Для IPv6 также объявлена структура struct sockaddr_in6.

Листинг 12. Структура sockaddr_in

В наших системах 2 адресных пространства: системная память и порты ввода/вывода. Порт – это адрес, но он локальный.

Когда создавался интернет, был изменён порядок байтов. Аппаратный и сетевой порядок байтов:

- big_endian – network byte order (сетевой порядок);
- little_endian – host byte order (порядок хоста).

Функции для работы с порядком байтов:

43 / 257

Смысл этих функций в том, что существует разная последовательность байтов, фактически меняется формат передаваемых значений. Эти функции нужно вызывать в обязательном порядке, даже если порядок байтов в сети и на хосте совпадает: не все аппаратные платформы имеют прямой порядок байтов. Если порядок байтов совпадает, функция не выполняется.

Смысл этих функций в том, что существует разная последовательность байтов, фактически меняется формат передаваемых значений. Эти функции нужно вызывать в обязательном порядке, даже если порядок байтов в сети и на хосте совпадает: не все аппаратные платформы имеют прямой порядок байтов. Если порядок байтов совпадает, функция не выполняется.

Смысл этих функций в том, что существует разная последовательность байтов, фактически меняется формат передаваемых значений. Эти функции нужно вызывать в обязательном порядке, даже если порядок байтов в сети и на хосте совпадает: не все аппаратные платформы имеют прямой порядок байтов. Если порядок байтов совпадает, функция не выполняется.

Смысл этих функций в том, что существует разная последовательность байтов, фактически меняется формат передаваемых значений. Эти функции нужно вызывать в обязательном порядке, даже если порядок байтов в сети и на хосте совпадает: не все аппаратные платформы имеют прямой порядок байтов. Если порядок байтов совпадает, функция не выполняется.

Смысл этих функций в том, что существует разная последовательность байтов, фактически меняется формат передаваемых значений. Эти функции нужно вызывать в обязательном порядке, даже если порядок байтов в сети и на хосте совпадает: не все аппаратные платформы имеют прямой порядок байтов. Если порядок байтов совпадает, функция не выполняется.

Смысл этих функций в том, что существует разная последовательность байтов, фактически меняется формат передаваемых значений. Эти функции нужно вызывать в обязательном порядке, даже если порядок байтов в сети и на хосте совпадает: не все аппаратные платформы имеют прямой порядок байтов. Если порядок байтов совпадает, функция не выполняется.

Смысл этих функций в том, что существует разная последовательность байтов, фактически меняется формат передаваемых значений. Эти функции нужно вызывать в обязательном порядке, даже если порядок байтов в сети и на хосте совпадает: не все аппаратные платформы имеют прямой порядок байтов. Если порядок байтов совпадает, функция не выполняется.

Смысл этих функций в том, что существует разная последовательность байтов, фактически меняется формат передаваемых значений. Эти функции нужно вызывать в обязательном порядке, даже если порядок байтов в сети и на хосте совпадает: не все аппаратные платформы имеют прямой порядок байтов. Если порядок байтов совпадает, функция не выполняется.

48 / 257

- 1) htons() – host to network short:

```
1 #include <arpa/inet.h>
2 uint16_t htons(uint16_t hostint16);
3 uint32_t htonl(uint32_t hostint32);
```

- 2) ntohs() – network to host:

```
1 #include <arpa/inet.h>
2 uint16_t ntohs(uint16_t netint16);
3 uint32_t ntohl(uint32_t netint32);
```

Смысл этих функций в том, что существует разная последовательность байтов, фактически меняется формат передаваемых значений. Эти функции нужно вызывать в обязательном порядке, даже если порядок байтов в сети и на хосте совпадает: не все аппаратные платформы имеют прямой порядок байтов. Если порядок байтов совпадает, функция не выполняется.

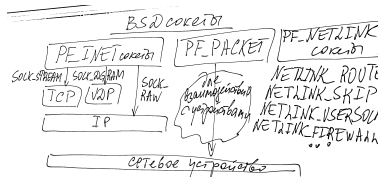


Рис. 5. BSD сокеты

Пакеты приходят на сетевую карту и с неё же уходит.

Для работы с сокетами ядро Linux предоставляет системный вызов sys_socketcall. Этот вызов является общей точкой входа для различных операций с сокетами.

```
1 asmlinkage long sys_socketcall(int call, unsigned long *args)
2 {
3     int err;
4     if (copy_from_user(&args, args, call))
5         return -EFAULT;
```

44 / 257

3.5 Классификация моделей ввода-вывода

3.5.1 Блокирующий ввод/вывод (blocking I/O)

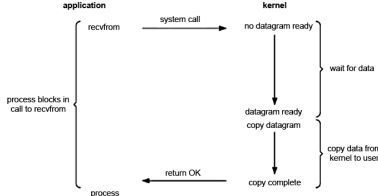


Рис. 7. Блокирующий ввод/вывод

Процесс, выполнивший системный вызов ввода/вывода, будет ожидать получения данных: система вернёт либо код ошибки либо значение, соответствующее успешному выполнению системного вызова.

Когда приложение запрашивает ввод данных с клавиатуры (в примере на картинке – receive_from), процесс блокируется до тех пор, пока не получит данные и ядро системы выполнит цепую последовательность действий:

- 1) Когда введённые пользователем данные поступят в буфер клавиатуры, контроллер клавиатуры отправит сигнал прерывания на контроллер прерываний.
- 2) С контроллера этот сигнал по шине управления поступит на новую процессора, и процессор перейдёт на выполнение обработчика прерывания.
- 3) В результате процесс, запросивший ввод, будет разблокирован.

Все последовательности действий выполняются, пока процесс блокирован на запросе ввода данных. Это выполняется ввод/вывод, в основном мы имеем дело именно с ним.

49 / 257

```
6
7 ab = a[0];
8 al = a[1];
9
10 switch (call)
11 {
12     case SYS_SOCKET:
13         err = sys_socket(ab, al, a[2]);
14         break;
15     case SYS_BIND:
16         err = sys_bind(ab, (struct sockaddr *) al, a[2]);
17         break;
18     case SYS_CONNECT:
19         err = sys_connect(ab, (struct sockaddr *) al, a[2]);
20         break;
21     ...
22 }
23
24 return err;
```

Листинг 13. Системный вызов sys_socketcall()

В качестве одного из аргументов передаётся номер операции: создание сокета, привязка адреса, установление соединения, создание соединений, приём или передача данных. Внутри ядра по этому номеру выбирается соответствующая функция, например socket, bind, connect, listen, accept, send, recv и другие.

Перед выполнением операции ядро копирует аргументы из пользовательского пространства в пространство ядра. Напрямую обратиться к памяти процесса нельзя, поэтому используются специальные функции вроде copy_from_user и copy_to_user.

sys_socketcall() приводит к вызову sys_socket():

```
1 asmlinkage long sys_socket(int family, int type, int protocol)
2 {
3     int retval;
4     struct socket *sock;
5     ...
6     retval = socket_create(family, type, protocol, &sock);
7     ...
8     return retval;
9 }
```

Листинг 14. Системный вызов sys_socket()

45 / 257

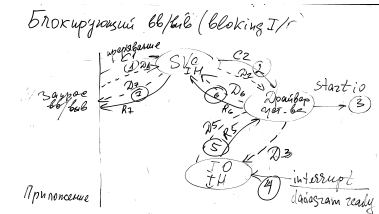


Рис. 8. Блокирующий ввод/вывод. По Шоу

Эта модель была декларирована намного раньше, чем та, которую мы берём из книги по сетям.

Важно

Разрыв справа на схеме говорит о том, что процессор не управляет внешним устройством.

Внизу справа – аппаратное прерывание, которое формируется, когда данные в буфере устройства. Из этого буфера данные пройдут несколько этапов, прежде чем окажутся в буфере приложения, и приложение сможет продолжить своё выполнение.

Важно

В этой модели важно именно то, что процесс блокируется на запросе и разблокируется только тогда, когда данные могут быть процессом получены.

Данные нужно скопировать, но это выполняет обработчик прерывания. Данные готовы – они поступили в буфер ядра.

50 / 257

В ядре сокет описывает структура struct socket:

```
1 struct socket
2 {
3     socket_state state;
4     short type;
5     unsigned long flags;
6     const struct proto_ops *ops; // protocol operations
7     struct fasync_struct *fasync_list;
8     struct file *file;
9     struct sock *sk;
10     wait_queue_head_t wait;
11 };
```

Листинг 15. Структура socket

У сокета может быть 5 состояний (state):

- 1) SS_FREE – свободный сокет.
- 2) SS_UNCONNECTED – несоединённый сокет
- 3) SS_CONNECTING – сокет находится в процессе соединения
- 4) SS_CONNECTED – соединённый сокет
- 5) SS_DISCONNECTING – сокет в процессе разъединения

Взаимодействие по модели клиент-сервер на сетевых сокетах:

46 / 257

Важно

Процессор не управляет внешним устройством.

Внизу справа – аппаратное прерывание, которое формируется, когда данные в буфере устройства. Из этого буфера данные пройдут несколько этапов, прежде чем окажутся в буфере приложения, и приложение сможет продолжить своё выполнение.

Поскольку схема рассматривает ситуацию в самом общем виде, на ней показан supervisor call interrupt handler – функция, которая вызывается в режиме ядра в ответ на системный вызов. В общем виде последовательность блокирующего ввода/вывода:

- 1) Запрос ввода/вывода – это системный вызов; при переходе в режим ядра он блокирует приложение, то есть его состояние будет изменено на D (uninterruptible sleep).
- 2) Начиная выполняться код ядра будет вызвана точка входа драйвера устройства, которое идентифицируется запросом. В результате работы кода драйвера будет сформирована команда для устройства, которая по шине данных будет передана контроллеру устройства.
- 3) Отправка команды, процессор отключается от управления устройством – это принцип разделения функций: процессор не управляет работой внешних устройств, ими управляют контроллеры устройств. Этот принцип инициализируется включением в систему прерываний прерываний от внешних устройств: они информируют процессор о завершении операции ввода/вывода.

- 4) Когда данные готовы, будет вызван обработчик соответствующего аппаратного прерывания: его задача – считать данные из буфера устройства в буфер ядра и инициализировать следующие действия по обработке полученных данных.
- 5) Обработчик вызовет соответствующую callback-функцию драйвера, передаст полученные данные, и в результате этих действий приложение будет разблокировано: в буфере приложения (переменной, массива, поле структуры и так далее) будут записаны запрошенные данные.

51 / 257

3.5.2 Неблокирующий ввод/вывод (non-blocking I/O, polling (opncos))

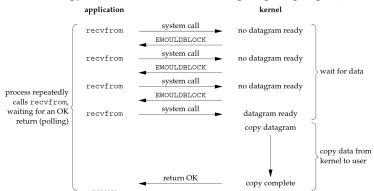


Рис. 9. Непрерывный ввод/вывод

Опрос (polling) в системах был реализован раньше прерываний от устройств ввода/вывода: процессор опрашивал внешние устройства об их готовности (когда данные поступают в буфер устройства). При этом процесс не блокируется, но процессорное время тратится на опрос, никакой другой полезной работы процессор не выполняет (непроизводительные затраты). Это неэффективно, потому что быстроедействующий процессор ждёт медленное устройство.

Советы бывают двух видов: блокирующие и неблокирующие. Благодаря многопоточности можно блокирующие сокет использовать во всех случаях: и тогда, когда нужно обрабатывать несколько сокетов параллельно, и тогда, когда требуется возможность прервать операцию на сокете. Для установки неблокирующего сокета существует макрос `O_NONBLOCK`:

```
1 {
2 ...
3 sk_fd = socket(AF_INET, SOCK_STREAM, 0);
4 fcntl(sk_fd, F_SET, O_NONBLOCK); // Установка неблокирующего режима сокета
5 ...
6 }
```

Если таким образом установлен неблокирующий режим сокета, то после любого вызова функции `read` будет возвращено управление сразу, если на входе нет

данных для чтения. При этом функцией `read` будет возвращено значение `EAGAIN` (`EWOULDBLOCK`).

3.5.3 Мультиплексирование ввода/вывода (multiplexing I/O)

Мультиплексер (коммутатор) – устройство, которое объединяет информацию, поступающую по нескольким каналам ввода, и выдает ее по одному выходному каналу.

Мультиплексирование – процесс совмещения нескольких сообщений, передаваемых одновременно, в одной физической или логической среде.

Мультиплексирование является альтернативой многопоточности. Мультиплексирование выполняет сервер, так как сервер принимает большое количество соединений. Сервер блокируется на мультиплексоре (например, `select`) до тех пор, пока не будет готова хотя бы одна точка соединения, после чего получает и обрабатывает данные.

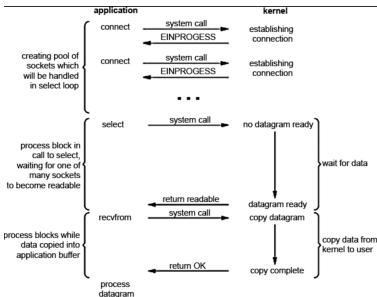


Рис. 10. Мультиплексирование

Приложение клиента вызывает системный вызов `connect`, чтобы установить соединение с сервером. Такое соединение устанавливается не мгновенно, поэтому на схеме есть `establishing connection`, при этом возможно, что соединение в данный момент не установлено – приложение повторяет попытку соединиться. В результате запросов клиентов на соединение на стороне сервера будет сформирован пул сокетов, который надо обслужить, обработать.

На стороне сервера работает селектор. Он опрашивает точки соединения, и будет обработано первое возникшее готовое соединение. Опрашивается пул точек соединения, чтобы определить готовую для соединения точку. Клиентов много, сервер обрабатывает большое количество клиентов, но не все клиенты одновременно готовы установить соединение. Клиенты устанавливают соединение по мере готовности и необходимости, а сервер опрашивает эти точки, чтобы начать обслуживать первое готовое соединение.

При этом на `select` сервер блокируется до тех пор, пока не будет готова хотя бы одна точка соединения. Если такая точка готова, значит, данные могут быть получены сервером для последующей обработки. В результате сервер сможет выполнить получение данных (вексель `from`) и копирование этих данных. Когда данные получены, они копируются в приложение сервера, и сервер начинает их обрабатывать.

Почему потоки являются альтернативой мультиплексированию?

Потому что поток, когда выполнил системный вызов `read`, заблокировался, и не участвует в планировании. Когда данные готовы, он разблокируется и продолжает выполнение.

Бытовой пример. Переключатель режимов на стиральной машине, с помощью которого мы выбираем нужную точку.

3.5.4 Ввод/вывод, управляемый сигналами (Signal-Driven I/O)

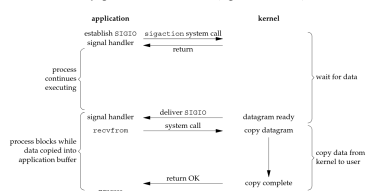


Рис. 11. Ввод/вывод, управляемый сигналами (Signal-Driven I/O)

Существует специальный сигнал `SIGIO`. Приложение должно установить обработчик сигнала – для этого вызывается системный вызов `sigaction(int signal, const struct sigaction *act, struct sigaction *oldact)`. Этот обработчик сигнала будет информировать о том, что данные готовы (`datagram ready`). Установка обработчика сигнала, процесс не блокируется в ожидании завершения ввода и продолжает выполнение. Когда процесс получает сигнал, обрабатывает обработчик, после этого процесс может получить данные: при вызове `recvfrom` выполняется копирование данных из `kernel mode` в `user mode`. Процесс блокируется только на время, пока данные копируются в буфер приложения.

Важно

В данном способе взаимодействия с внешним устройством всю работу выполняет ядро операционной системы. Во-первых, ядро отслеживает, когда данные будут готовы. Во-вторых, после того как данные готовы, посылает сигнал `SIGIO`, в результате чего будет вызван обработчик сигнала, причём этот обработчик называется функцией обратного вызова (`callback`).

Далее идёт `recvfrom` или `read`, но вызов можно выполнить или в обработчике сигнала, или в основном потоке программы. При этом ожидание может выполняться через `polling (opncos)`.

Существует ограничение: сигнал типа `SIGIO` для каждого процесса может быть только 1, следовательно обработать можно только 1 файловый дескриптор, то есть взаимодействие можно выполнять только с одним открытым файлом. На время выполнения обработки сигнала сигнал `SIGIO` блокируется. Это значит, что если во время блокировки сигнал поступит несколько раз, то эти сигналы будут потеряны.

У сигнала может быть установлена маска `NULL`. Это приведёт к тому, что другие сигналы блокироваться не будут: блокируется только такой же сигнал `SIGIO`.

3.5.5 Асинхронный ввод-вывод (Asynchronous I/O)

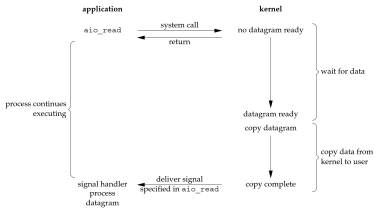


Рис. 12. Асинхронный ввод-вывод (Asynchronous I/O)

При асинхронном вводе-выводе приложение вызывает специальные функции, например `aio_read`, передавая дескриптор, адрес буфера и его размер. После вызова управление сразу возвращается процессу, потому что он не блокируется и может продолжать выполнение.

Основную работу берёт на себя ядро: оно контролирует выполнение операции и сообщает о её завершении, например с помощью связанного с АЮ сигнала. Когда данные готовы, процесс получает результат и обрабатывает их.

Главная особенность этой модели в том, что программу нужно специально организовывать: после запроса ввода-вывода процесс должен выполнять другую работу,

а не ждать завершения операции. Это позволяет ускорить выполнение за счёт отсутствия блокировки.

Проблема асинхронного ввода-вывода состоит в том, что надо получать данные синхронно. В асинхронном вводе-выводе внимание сосредотачивается на:

- 1) возможности определить, что ввод-вывод можно выполнить быстро;
- 2) завершении операции ввода-вывода, а именно: при невозможности выполнить его сразу возвращается ошибка.

3.5.6 Классификация моделей

	Блокирующий Blocking	Неблокирующий Non-blocking	
Синхронный Synchronous	read / write ①	read / write O_NONBLOCK	polling
Асинхронный Asynchronous	IO Multiplexing (select/poll/epoll)	AIO	②

Таблица 3. Классификация моделей ввода-вывода

Когда модель неблокирующая, запрос надо повторить. В любом случае это `polling`.

* Claude

В классификации Стивенса (UNIX Network Programming), из которой взяты сами модели, мультиплексирование относится к синхронному вводу-выводу: асинхронной считается только модель POSIX AIO, поскольку лишь в ней процесс не блокируется ни на ожидании готовности, ни на копировании данных. В таблице из контекста мультиплексирование помещено в строку «асинхронный».

3.6 Ввод-вывод с мультиплексированием: мультиплексор epoll – особенности (типы и алгоритм работы)

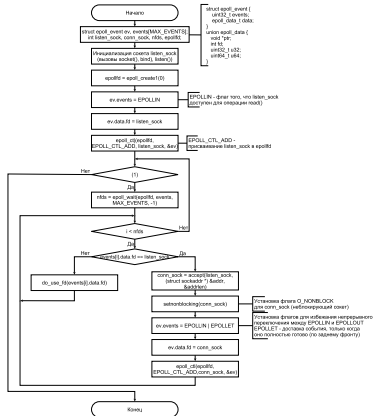


Рис. 13. Схема алгоритма epoll

Алгоритм работы (думать можно не писать, но прочтите перед счёй)

- Мультиплексер работает с массивом events: `struct epoll_event ev, events[MAX_EVENTS]`. В структуре `struct epoll_event` есть поле `data`, а в нём – поле `fd` (файловый дескриптор).
- Мультиплексер создаёт вызовом `epoll_create1(0)`, который возвращает файловый дескриптор `epollfd`.
- В структуре события указывается интересное событие: `ev.events = EPOLLIN` (готовность к чтению).
- Сокет нужно добавить в набор отслеживаемых: `epoll_ctl(epollfd, EPOLL_CTL_ADD, listen_sock, &ev)`.
- Вызовом `nfds = epoll_wait(epollfd, events, MAX_EVENTS, -1)` создаётся массив готовых событий: сервер блокируется на `epoll_wait`, пока не будет готов хотя бы один дескриптор.
- Дальше цикл по соединениям, которые мы зафиксировали: просматриваются поля массива events.
- Если это исходный (слушающий) сокет (`events[n].data.fd == listen_sock`), то вызывается `accept()`: создаётся копия исходного сокета, она переводится в неблокирующий режим (`setnonblocking`), и этот файловый дескриптор добавляется в массив дескрипторов мультиплексора с событиями `EPOLLIN` и `EPOLLET`.
- Иначе обслуживается готовый дескриптор уже принятого соединения (`do_use_fd`).

Разница в последовательности действий по сравнению с обычным сервером: мультиплексор определяет готовность к соединению обращения клиентов, то есть клиент вызывает `connect`, выполнил соответствующие действия и ждёт соединения. Фактически в этом `if` анализируется: если это необработанное соединение, то `accept` (принять). Мультиплексор подготавливает информацию для принятия соединения.

Важно

- В: Показывает на блок-решение на схеме алгоритма, для чего?
- О: Если соединение не обработано то `accept()`

3.7 Пример из лабораторной работы – сервер производство-потребление с мультиплексированием

Реализуется классическая задача производство-потребление. У нас 2 типа клиентов – клиент-производитель и клиент-потребитель. Клиент посылает серверу

принимая, кто он: P (производитель) или C (consumer, потребитель), то есть кто он хочет сделать – положить данные в кольцевой буфер или считать данные из буфера.

Сервер с мультиплексированием однопоточный: создаётся массив events, дальше прощаем по нему, обслуживаемые соединения (по схеме из предыдущего подраздела). Когда мы переходим к обработке по `else`, для нас важно, производитель это или потребитель – какую функцию вызвать (практически как у потоков в многопоточной версии).

Синхронизация:

- Не нужно обеспечивать взаимное исключение (сервер однопоточный, блокировок нет), а синхронизация нужна: считывающие семафоры вызвать не получится, иначе буфер будет переполнен. Как определить, что буфер заполнен? – Всё равно вводить счётчик.
- В обеих версиях (многопоточной и с мультиплексированием) должны быть семафоры, но в мультиплексоре не должно быть бинарного.
- Считывающие семафоры, чтобы исключить переход в режим ядра, можно было бы заменить счётчиками: переход в режим ядра – затратное действие. Но счётчики не позволяют нам выполнить блокировку.
- Если буфер заполнен (`N == max`), производитель ничего положить в буфер не может – значит, мы можем посылать ему, что он должен ждать. Если мы используем семафор, то сервер будет заблокирован на семафоре, значит, функция не отправит клиенту сообщение, что всё в порядке. Что должен получить производитель от сервера? – Он должен получить OK, что его данные положены. Пока ответ не получен, клиент блокирован на запросе `recvline from`, и только после получения он сможет выполнить следующую итерацию.

* Claude

В однопоточном сервере с мультиплексированием блокировка на считывающем семафоре остановила бы весь сервер, включая обслуживание потребителей, которые могли бы освободить место в буфере – получился бы `deadlock`. Поэтому проверку семафора в такой версии нужно делать неблокирующей (например, `sem_trywait`) и при заполненном буфере отвечать клиенту, что нужно повторить попытку, а чём и говорится в конспекте («послать, что он должен ждать»).

```
1 #define _GNU_SOURCE
2 #include <errno.h>
3 #include <netinet/in.h>
4 #include <stdio.h>
```

```
5 #include <stdio.h>
6 #include <sys/epoll.h>
7 #include <sys/ipc.h>
8 #include <sys/msg.h>
9 #include <sys/socket.h>
10 #include <sys/stat.h>
11 #include <time.h>
12 #include <unistd.h>
13
14 #define SERVER_PORT 9878
15 #define SERVER_BACKLOG 5
16 #define BUF_SIZE 52
17 #define MAX_EVENTS 64
18
19 #define OP_PRODUCER 'P'
20 #define OP_CONSUMER 'C'
21
22 #define SEM_KEY 200
23 #define SEM_STORAGE_EMPTY 0
24 #define SEM_STORAGE_FULL 1
25 #define SEM_LOCK -1
26 #define SEM_UNLOCK 1
27
28 static char ring_buffer[BUF_SIZE];
29 static int write_idx = 0;
30 static int read_idx = 0;
31
32 static int semaphores[4];
33 static int listen_fd;
34 static int epoll_fd;
35 static int should_exit = 0;
36
37 struct sembuf stop_write = {SEM_STORAGE_EMPTY, SEM_LOCK, IPC_NOWAIT | SEM_UNDO};
38 struct sembuf start_read = {SEM_STORAGE_FULL, SEM_UNLOCK, SEM_UNDO};
39 struct sembuf stop_read = {SEM_STORAGE_FULL, SEM_LOCK, IPC_NOWAIT | SEM_UNDO};
40 struct sembuf start_write = {SEM_STORAGE_EMPTY, SEM_UNLOCK, SEM_UNDO};
41
42 void producer(int fd) {
43     char letter;
44     if (recv(fd, &letter, 1, 0) == 0) {
45         return;
46     }
47     if (semop(semaphores, &stop_write, 1) == -1) {
48         send(fd, "MUT", 5, 0);
```

```
49 printf("IP %d) FULL, rejected '%c'\n", fd, letter);
50 return;
51 }
52 ring_buffer[write_idx] = letter;
53 write_idx = (write_idx + 1) % BUF_SIZE;
54 semop(semaphore_id, &start_read, 1);
55 send(fd, "OK", 3, 0);
56 printf("IP %d) put '%c'\n", fd, letter);
57 }
58 void consumer(int fd) {
59 if (semop(semaphore_id, &stop_read, 1) == -1) {
60 send(fd, "WAIT", 5, 0);
61 printf("(C %d) EMPTY\n", fd);
62 return;
63 }
64 char resp[2];
65 resp[0] = ring_buffer[read_idx];
66 resp[1] = '\0';
67 read_idx = (read_idx + 1) % BUF_SIZE;
68 semop(semaphore_id, &start_write, 1);
69 send(fd, resp, 2, 0);
70 printf("(C %d) got '%c'\n", fd, resp[0]);
71 }
72 }
73 void client_service(int fd, struct timespec time_start) {
74 char op;
75 if (recv(fd, &op, 1, 0) <= 0) {
76 close(fd);
77 return;
78 }
79 if (op == OP_PRODUCER) {
80 producer(fd);
81 } else if (op == OP_CONSUMER) {
82 consumer(fd);
83 }
84 close(fd);
85 }
86 int main(void) {
87 struct sockaddr_in server_addr = {0}, client_addr;
88 socklen_t client_len = sizeof(client_addr);
89 if (!listen(fd = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
90 perror("socket");
91 exit(1);
92 }
```

62 / 257

```
93 server_addr.sin_family = AF_INET;
94 server_addr.sin_addr.s_addr = INADDR_ANY;
95 server_addr.sin_port = htons(SERVER_PORT);
96 if (bind(listen_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) ==
-1) {
97 perror("bind");
98 exit(1);
99 }
100 if (listen(listen_fd, SERVER_BACKLOG) == -1) {
101 perror("listen"); exit(1);
102 }
103 int permissions = S_IRWXU | S_IRWXG | S_IRWXO;
104 semaphore_id = semget(SEM_KEY, 2, IPC_CREAT | permissions);
105 if (semaphore_id == -1) {
106 perror("semget"); exit(1);
107 }
108 if (semctl(semaphore_id, SEM_STORAGE_EMPTY, SETVAL, BUF_SIZE) == -1) {
109 perror("semctl"); exit(1);
110 }
111 if (semctl(semaphore_id, SEM_STORAGE_FULL, SETVAL, 0) == -1) {
112 perror("semctl"); exit(1);
113 }
114 epoll_fd = epoll_create(0);
115 if (epoll_fd == -1) {
116 perror("epoll_create"); exit(1);
117 }
118 struct epoll_event ev;
119 ev.events = EPOLLIN;
120 ev.data.fd = listen_fd;
121 if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, listen_fd, &ev) == -1) {
122 perror("epoll_ctl"); exit(1);
123 }
124 struct epoll_event events[MAX_EVENTS];
125 while (!should_exit) {
126 int nfd = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
127 if (nfd == -1) {
128 perror("epoll_wait");
129 break;
130 }
131 for (int i = 0; i < nfd; i++) {
132 if (events[i].data.fd == listen_fd) {
133 int conn_fd = accept(listen_fd, (struct sockaddr*)&client_addr,
&client_len);
134 if (conn_fd == -1) {
135 perror("accept");
```

63 / 257

```
136 continue;
137 }
138 ev.events = EPOLLIN;
139 ev.data.fd = conn_fd;
140 if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, conn_fd, &ev) == -1) {
141 perror("epoll_ctl");
142 close(conn_fd);
143 continue;
144 }
145 } else {
146 client_service(events[i].data.fd, &time_start);
147 }
148 }
149 }
150 close(epoll_fd);
151 if (semctl(semaphore_id, 0, IPC_RMID) == -1) {
152 perror("semctl");
153 exit(1);
154 }
155 return EXIT_SUCCESS;
156 }
```

Клиент в цикле подключается к серверу, отправляет признак P/C (производитель дополнительно шлёт символ данных) и обрабатывает ответ сервера: ОК – запрос выполнен, WAIT – буфер полон/пуст, попытку нужно повторить.

```
1 #include <arpa/inet.h>
2 #include <netinet/in.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <sys/socket.h>
7 #include <unistd.h>
8
9 #define SERVER_IP "127.0.0.1"
10 #define SEM_KEY 9876
11 #define MSG_SIZE 8
12 #define OP_PRODUCER 'P'
13 #define OP_CONSUMER 'C'
14
15 static int loop_flag = 1;
16 int connect_to_server(void) {
17 int fd = socket(AF_INET, SOCK_STREAM, 0);
18 if (fd == -1) {
```

64 / 257

```
19 perror("socket");
20 return -1;
21 }
22 struct sockaddr_in server_addr;
23 memset(&server_addr, 0, sizeof(server_addr));
24 server_addr.sin_family = AF_INET;
25 server_addr.sin_port = htons(SERVER_PORT);
26 int plen(AF_INET, SERVER_IP, &server_addr, sizeof(server_addr));
27 if (connect(fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) == -1) {
28 perror("connect");
29 close(fd);
30 return -1;
31 }
32 return fd;
33 }
34 int producer(int fd, char* letter) {
35 char response(MSG_SIZE) = {0};
36 if (recv(fd, response, MSG_SIZE, 0) <= 0) {
37 perror("recv"); return -1;
38 }
39 if (strcmp(response, "WAIT") == 0) {
40 printf("P: buffer full\n"); return 1;
41 }
42 printf("P: '%c' -> %s\n", *letter, response);
43 *letter = (*letter == 'z') ? 'a' : *letter + 1;
44 return 0;
45 }
46 int consumer(int fd) {
47 char response(MSG_SIZE) = {0};
48 if (recv(fd, response, MSG_SIZE, 0) <= 0) {
49 perror("recv"); return -1;
50 }
51 if (strcmp(response, "WAIT") == 0) {
52 printf("C: buffer empty\n"); return 1;
53 }
54 printf("C: got '%c'\n", response[0]);
55 return 0;
56 }
57 int main(void) {
58 char letter = 'a';
59 for (int i = 0; i < ITERATIONS && loop_flag; i++) {
60 char op = ((i % 3) == 2) ? OP_CONSUMER : OP_PRODUCER;
61 int fd = connect_to_server();
62 if (fd == -1) return EXIT_FAILURE;
```

65 / 257

```
63 int rc;
64 if (op == OP_PRODUCER) {
65 char msg[2] = {op, letter};
66 if (send(fd, msg, 2, 0) == -1) {
67 perror("send");
68 close(fd);
69 return EXIT_FAILURE;
70 }
71 rc = producer(fd, &letter);
72 } else {
73 if (send(fd, &op, 1, 0) == -1) {
74 perror("send");
75 close(fd);
76 return EXIT_FAILURE;
77 }
78 rc = consumer(fd);
79 }
80 close(fd);
81 if (rc == -1) return EXIT_FAILURE;
82 }
83 return EXIT_SUCCESS;
84 }
```

66 / 257

4

4 Файловая подсистема /proc – название, особенности, файлы, поддиректории, ссылка self, информация об окружении, состоянии процесса, прерываниях. Структура proc_dir_entry: функция для работы с элементами /proc. Использование структуры file_operations для регистрации функций работы с файлами. Передача данных из адресного пространства пользователя в адресное пространство ядра и обратно (tab. раб.). Обоснование необходимости использования специальных функций для передачи данных из пространства пользователя в ядро и из ядра в пространство пользователя.

4.1 Файловая подсистема /proc

Виртуальная файловая подсистема /proc (process information pseudo-file-system) позволяет в режиме пользователя получить большой объем информации о ресурсах процесса. Виртуальная – то есть не существующая на самом деле (не хранящаяся во вторичной памяти), а создающаяся “на лету” по мере обращения к данным. По сути интерфейс /proc позволяет получить информацию из структур ядра, не обращаясь к ним напрямую, а потому работая в режиме пользователя. Большинство файлов доступны только для чтения, однако некоторые доступны для записи, что позволяет изменять переменные ядра.

Файловая система /proc является немониторимой. Под этим подразумевается, что она не связана ни с каким внешним носителем (она видна в файловых системах, подключенных к корневому каталогу, но не связана ни с каким внешним устройством. Поэтому она и называется псевдофайловой системой).

Для каждого процесса в /proc определена директория идентификатора процесса (/proc/<pid>/.), она будет создана налету. В ней хранятся файлы, директории и символические ссылки, используя которые можно получить развернутую информацию о конкретном процессе. Вся информация доступна из шлемоды:

Элемент	Тип	Содержание
cmdline	Файл	holds the complete command line
cwd	Символическая ссылка	Содержит командную строку процесса
environ	Файл	Указывает на директорию окружения процесса
exe	Символическая ссылка	Содержит список переменных окружения процесса
		Указывает на образ процесса (на его файл)
		Символическая ссылка, содержащая фактический путь к выполняемой команде.
comm	Файл	Имя команды, связанное с процессом. Это именно то имя файла, которое приведет к выполнению нашего кода.

69 / 257

Элемент	Тип	Содержание
		Замечание 4.1 Разные процессы в одном и том же процессе могут иметь разные значения, доступные через /proc/<pid>/task/<tid>/comm. task – файл, в котором находится поддиректория потоков.
stat	Файл	Содержит информацию о процессе (vsize, start, brk и др.)
fd	Директория	Содержит ссылки на открытые процессом файлы Замечание 4.2 В системе различаются файлы и открытые файлы. Открытые файлы описываются структурой struct file. Просто файлы предоставляются в системе деревом каталогов, а описываются функцией-файлом struct file_operations. Один и тот же файл может быть открыт много раз, каждый раз разным процессом. В таблице открытых файлов на каждой такой открытой файл будет дескриптор struct file
root	soft link	Это символическая ссылка, которая указывает на корень файловой системы процесса. Замечание 4.3 Процессы, которые выполняются на нашем компьютере, могут привязываться разным файловым системам. Пока мы имя исполняемого файла не указали в командной строке, пока оно не было выполнено, он может по какой-либо причине.
maps	Файл	Это 64-х разрядное значение для каждой строки процесса.
readmap	файл	При этом нас интересует фактически несколько бит: 1) 63 бит – present (in RAM) 2) 62 бит – swap space, область свопинга 3) 61 – shared anonymous page 4) 55 бит – dirty. Он указывается в дескрипторе страницы.

70 / 257

Элемент	Тип	Содержание
		С 54 по 6: если страница присутствует в физической памяти, то эти биты являются номером фрейма, то есть номером физической страницы.
task	Директория	Содержит поддиректории потоков
interrupts	Файл	Предоставляет таблицу о количестве прерываний на каждом из процессоров в следующем виде: 1) Первая колонка: номер прерывания 2) Колонка CPUs: сичетчики прерываний на каждом из процессоров 3) Следующая колонка: вид прерывания: 3.1) IO-APIC-edge – прерывание по фронту на контроллер I/O APIC. 3.2) IO-APIC-fasteoi – прерывание по уровню на контроллер I/O APIC. 3.3) PCI-MSI-edge – MSI прерывание. 3.4) XT-PIC-XT-PIC – прерывание на PIC контроллер. 4) Последняя колонка: устройство, ассоциированное с данным прерыванием.

Таблица 4. Содержание директории /proc/<pid>

4.1.1 Ссылка self /proc/self – символическая ссылка, существующая для каждого процесса и указывающая на его собственную директорию /proc/<pid>. позволяет процессу обращаться к своим данным, не вызывая getpid().

4.1.2 Информация об окружении Окружение неёт файл environ (для текущего процесса – /proc/self/environ); в нём исходное окружение, установленное при запуске (execve()), переменные разделены нулевыми символами ('\0'), на строки файл не разбит. Изменения окружения после execve() (например, putenv()) в нём не отражаются.

4.1.3 Информация о состоянии процесса Информацию о состоянии процесса неёт файл stat – важную, подобранную разработчиками информацию о процессе: его идентификатор pid, состояние state, имя команды comm, идентификатор родителя ppid, приоритет prio, размер виртуального адресного пространства vsize, адрес начала кучи start_brk и т.д.

71 / 257

67 / 257

Стандарт POSIX-1 оговаривает существование типа `dev_t` – 32-разрядное число, в котором 12 бит отведены под старший номер, 20 – под младший. Код не должен делать предположений об организации номеров внутри числа. Для работы с `dev_t` используются макросы:

```
1 MAJOR(dev_t dev) // получить старший номер
2 MINOR(dev_t dev) // получить младший номер
```

Имея старший и младший номера, можно их объединить в значение типа `dev_t`.

```
1 MDEV(int major, int minor) // объединить в dev_t
```

В ядре определены «элементарные типы»:

```
1 typedef _kernel_dev_t dev_t;
2 typedef _kernel_mode_t mode_t;
3 typedef _kernel_inode_t inode_t;
4 typedef _kernel_off_t off_t;
5
6 typedef u32 nlink_t;
```

Элементарные типы определены специально, чтобы на этапе компиляции выполнять проверку соответствия типов, хотя это всё, конечно, int.

5.2 Выделение и освобождение номеров устройств

Одно из первых действий, которые должен сделать драйвер, когда устанавливает связываемое устройство – он должен получить 1 или более номеров устройств, чтобы работать с ними, чтобы идентифицировать свой `char device`. Для этого выданы функции

```
1 int register_chrdev_region(dev_t first, unsigned int count, char *name);
```

– **first** – начальный номер устройства доступного для выделения диапазона

Часто минорная часть значения `first` равна нулю. Но этот счёт нет никаких определённых требований.

– **count** – общее количество последовательных номеров устройств, которые могут быть запрошены. При этом значение `count` может превышать размер запрашиваемого диапазона. В этом случае будет переход на следующий старший номер, ошибка не возникает, если запрашиваемый диапазон доступен.

– **name** – имя устройства, которое связывается с диапазоном. Это можно увидеть или в `/proc/devices` или в `sysfs` (/sys).

При успешном выделении функции возвращает 0.

82 / 257

Часто неизвестно значение старшего номера устройства, поэтому разработчики ядра всё время стремятся перейти на динамическое распределение номеров устройств. Для динамического выделения используется функция

```
1 int alloc_chrdev_region(dev_t *dev, unsigned int firstminor, unsigned int count, char *name);
```

– **dev**

Значение указателя при успешном завершении будет содержать первый номер в определённом диапазоне. Обычно указывают 0.

Остальные параметры аналогичны рассмотренной ранее функции.

Номера освобождаются функцией

```
1 void unregister_chrdev_region(dev_t first, unsigned int count);
```

Все эти функции вызываются в модуле в функции `init`, а последняя функция – в функции `exit`.

5.3 Управление устройствами: драйверы

Для управления разнообразием внешних устройств в Unix/Linux реализована концепция драйверов: устройства регистрируются в системе, а в системе есть набор драйверов; когда у системы есть драйвер, новое подключаемое устройство может им управляться.

Драйвер – программа или часть кода ядра, которая предназначена для управления конкретным внешним устройством. Драйверы обычно содержат набор команд, специфичных для этого устройства: каждое устройство воспринимает только специальный набор команд, и для каждого устройства он различен – этим и объясняется необходимость драйверов.

В Linux поддерживаются 3 типа драйверов:

- 1) **Г-тип** – изначально часть кода ядра, загружаются и инициализируются при запуске ОС, поэтому их устройства сразу доступны. Нужны для поддержки, например, мониторинга корневой ФС и запуска ПК (материнская плата и т.д.).
- 2) **П-тип** – загружаемые модули ядра: функции становятся доступны после загрузки модуля. Такие драйверы пишутся, чтобы изменить или добавить функциональность устройства (звуковые и сетевые карты, SCSI адаптеры).
- 3) **И-тип** – драйверы, код которых поделён между ядром и специальной утилитой управления устройством (принтер: ядро осуществляет взаимодействие с параллельным портом, а формирование управляющих сигналов для принтера осуществляет демон печати `lpd`).

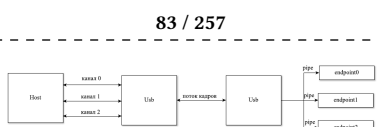


Рис. 16. Конечные точки и каналы

Чтобы выполнить операцию (отправить/получить данные, изменить настройки), драйвер создаёт или получает USB, заполняет его (тип запроса, адрес устройства и номер конечной точки, буфер, длина данных, функция обратного вызова) и отправляет в ядро USB. По завершении операции хост-контроллер вызывает функцию обратного вызова USB, уведомляя драйвер о результате.

struct urb содержит всю информацию для выполнения USB-транзакции и доставки данных и статус:

```
1 struct urb {
2     struct kref kref; // reference count of the URB
3     struct list_head urb_list; // list head for use by the urb's current owner
4     struct usb_device *dev; // (in) pointer to associated device
5     struct usb_host_endpoint *ep; // (internal) pointer to endpoint
6     unsigned int pipe; // (in) pipe information
7     ...
8     ...
9     ...
10    ...
11    ...
12    ...
13    ...
14    ...
15    ...
16    ...
17    ...
18    ...
19    ...
20    ...
21    ...
22    ...
23    ...
24    ...
25    ...
26    ...
27    ...
28    ...
29    ...
30    ...
31    ...
32    ...
33    ...
34    ...
35    ...
36    ...
37    ...
38    ...
39    ...
40    ...
41    ...
42    ...
43    ...
44    ...
45    ...
46    ...
47    ...
48    ...
49    ...
50    ...
51    ...
52    ...
53    ...
54    ...
55    ...
56    ...
57    ...
58    ...
59    ...
60    ...
61    ...
62    ...
63    ...
64    ...
65    ...
66    ...
67    ...
68    ...
69    ...
70    ...
71    ...
72    ...
73    ...
74    ...
75    ...
76    ...
77    ...
78    ...
79    ...
80    ...
81    ...
82    ...
83    ...
84    ...
85    ...
86    ...
87    ...
88    ...
89    ...
90    ...
91    ...
92    ...
93    ...
94    ...
95    ...
96    ...
97    ...
98    ...
99    ...
100   ...
101   ...
102   ...
103   ...
104   ...
105   ...
106   ...
107   ...
108   ...
109   ...
110   ...
111   ...
112   ...
113   ...
114   ...
115   ...
116   ...
117   ...
118   ...
119   ...
120   ...
121   ...
122   ...
123   ...
124   ...
125   ...
126   ...
127   ...
128   ...
129   ...
130   ...
131   ...
132   ...
133   ...
134   ...
135   ...
136   ...
137   ...
138   ...
139   ...
140   ...
141   ...
142   ...
143   ...
144   ...
145   ...
146   ...
147   ...
148   ...
149   ...
150   ...
151   ...
152   ...
153   ...
154   ...
155   ...
156   ...
157   ...
158   ...
159   ...
160   ...
161   ...
162   ...
163   ...
164   ...
165   ...
166   ...
167   ...
168   ...
169   ...
170   ...
171   ...
172   ...
173   ...
174   ...
175   ...
176   ...
177   ...
178   ...
179   ...
180   ...
181   ...
182   ...
183   ...
184   ...
185   ...
186   ...
187   ...
188   ...
189   ...
190   ...
191   ...
192   ...
193   ...
194   ...
195   ...
196   ...
197   ...
198   ...
199   ...
200   ...
201   ...
202   ...
203   ...
204   ...
205   ...
206   ...
207   ...
208   ...
209   ...
210   ...
211   ...
212   ...
213   ...
214   ...
215   ...
216   ...
217   ...
218   ...
219   ...
220   ...
221   ...
222   ...
223   ...
224   ...
225   ...
226   ...
227   ...
228   ...
229   ...
230   ...
231   ...
232   ...
233   ...
234   ...
235   ...
236   ...
237   ...
238   ...
239   ...
240   ...
241   ...
242   ...
243   ...
244   ...
245   ...
246   ...
247   ...
248   ...
249   ...
250   ...
251   ...
252   ...
253   ...
254   ...
255   ...
256   ...
257   ...
258   ...
259   ...
260   ...
261   ...
262   ...
263   ...
264   ...
265   ...
266   ...
267   ...
268   ...
269   ...
270   ...
271   ...
272   ...
273   ...
274   ...
275   ...
276   ...
277   ...
278   ...
279   ...
280   ...
281   ...
282   ...
283   ...
284   ...
285   ...
286   ...
287   ...
288   ...
289   ...
290   ...
291   ...
292   ...
293   ...
294   ...
295   ...
296   ...
297   ...
298   ...
299   ...
300   ...
301   ...
302   ...
303   ...
304   ...
305   ...
306   ...
307   ...
308   ...
309   ...
310   ...
311   ...
312   ...
313   ...
314   ...
315   ...
316   ...
317   ...
318   ...
319   ...
320   ...
321   ...
322   ...
323   ...
324   ...
325   ...
326   ...
327   ...
328   ...
329   ...
330   ...
331   ...
332   ...
333   ...
334   ...
335   ...
336   ...
337   ...
338   ...
339   ...
340   ...
341   ...
342   ...
343   ...
344   ...
345   ...
346   ...
347   ...
348   ...
349   ...
350   ...
351   ...
352   ...
353   ...
354   ...
355   ...
356   ...
357   ...
358   ...
359   ...
360   ...
361   ...
362   ...
363   ...
364   ...
365   ...
366   ...
367   ...
368   ...
369   ...
370   ...
371   ...
372   ...
373   ...
374   ...
375   ...
376   ...
377   ...
378   ...
379   ...
380   ...
381   ...
382   ...
383   ...
384   ...
385   ...
386   ...
387   ...
388   ...
389   ...
390   ...
391   ...
392   ...
393   ...
394   ...
395   ...
396   ...
397   ...
398   ...
399   ...
400   ...
401   ...
402   ...
403   ...
404   ...
405   ...
406   ...
407   ...
408   ...
409   ...
410   ...
411   ...
412   ...
413   ...
414   ...
415   ...
416   ...
417   ...
418   ...
419   ...
420   ...
421   ...
422   ...
423   ...
424   ...
425   ...
426   ...
427   ...
428   ...
429   ...
430   ...
431   ...
432   ...
433   ...
434   ...
435   ...
436   ...
437   ...
438   ...
439   ...
440   ...
441   ...
442   ...
443   ...
444   ...
445   ...
446   ...
447   ...
448   ...
449   ...
450   ...
451   ...
452   ...
453   ...
454   ...
455   ...
456   ...
457   ...
458   ...
459   ...
460   ...
461   ...
462   ...
463   ...
464   ...
465   ...
466   ...
467   ...
468   ...
469   ...
470   ...
471   ...
472   ...
473   ...
474   ...
475   ...
476   ...
477   ...
478   ...
479   ...
480   ...
481   ...
482   ...
483   ...
484   ...
485   ...
486   ...
487   ...
488   ...
489   ...
490   ...
491   ...
492   ...
493   ...
494   ...
495   ...
496   ...
497   ...
498   ...
499   ...
500   ...
501   ...
502   ...
503   ...
504   ...
505   ...
506   ...
507   ...
508   ...
509   ...
510   ...
511   ...
512   ...
513   ...
514   ...
515   ...
516   ...
517   ...
518   ...
519   ...
520   ...
521   ...
522   ...
523   ...
524   ...
525   ...
526   ...
527   ...
528   ...
529   ...
530   ...
531   ...
532   ...
533   ...
534   ...
535   ...
536   ...
537   ...
538   ...
539   ...
540   ...
541   ...
542   ...
543   ...
544   ...
545   ...
546   ...
547   ...
548   ...
549   ...
550   ...
551   ...
552   ...
553   ...
554   ...
555   ...
556   ...
557   ...
558   ...
559   ...
560   ...
561   ...
562   ...
563   ...
564   ...
565   ...
566   ...
567   ...
568   ...
569   ...
570   ...
571   ...
572   ...
573   ...
574   ...
575   ...
576   ...
577   ...
578   ...
579   ...
580   ...
581   ...
582   ...
583   ...
584   ...
585   ...
586   ...
587   ...
588   ...
589   ...
590   ...
591   ...
592   ...
593   ...
594   ...
595   ...
596   ...
597   ...
598   ...
599   ...
600   ...
601   ...
602   ...
603   ...
604   ...
605   ...
606   ...
607   ...
608   ...
609   ...
610   ...
611   ...
612   ...
613   ...
614   ...
615   ...
616   ...
617   ...
618   ...
619   ...
620   ...
621   ...
622   ...
623   ...
624   ...
625   ...
626   ...
627   ...
628   ...
629   ...
630   ...
631   ...
632   ...
633   ...
634   ...
635   ...
636   ...
637   ...
638   ...
639   ...
640   ...
641   ...
642   ...
643   ...
644   ...
645   ...
646   ...
647   ...
648   ...
649   ...
650   ...
651   ...
652   ...
653   ...
654   ...
655   ...
656   ...
657   ...
658   ...
659   ...
660   ...
661   ...
662   ...
663   ...
664   ...
665   ...
666   ...
667   ...
668   ...
669   ...
670   ...
671   ...
672   ...
673   ...
674   ...
675   ...
676   ...
677   ...
678   ...
679   ...
680   ...
681   ...
682   ...
683   ...
684   ...
685   ...
686   ...
687   ...
688   ...
689   ...
690   ...
691   ...
692   ...
693   ...
694   ...
695   ...
696   ...
697   ...
698   ...
699   ...
700   ...
701   ...
702   ...
703   ...
704   ...
705   ...
706   ...
707   ...
708   ...
709   ...
710   ...
711   ...
712   ...
713   ...
714   ...
715   ...
716   ...
717   ...
718   ...
719   ...
720   ...
721   ...
722   ...
723   ...
724   ...
725   ...
726   ...
727   ...
728   ...
729   ...
730   ...
731   ...
732   ...
733   ...
734   ...
735   ...
736   ...
737   ...
738   ...
739   ...
740   ...
741   ...
742   ...
743   ...
744   ...
745   ...
746   ...
747   ...
748   ...
749   ...
750   ...
751   ...
752   ...
753   ...
754   ...
755   ...
756   ...
757   ...
758   ...
759   ...
760   ...
761   ...
762   ...
763   ...
764   ...
765   ...
766   ...
767   ...
768   ...
769   ...
770   ...
771   ...
772   ...
773   ...
774   ...
775   ...
776   ...
777   ...
778   ...
779   ...
780   ...
781   ...
782   ...
783   ...
784   ...
785   ...
786   ...
787   ...
788   ...
789   ...
790   ...
791   ...
792   ...
793   ...
794   ...
795   ...
796   ...
797   ...
798   ...
799   ...
800   ...
801   ...
802   ...
803   ...
804   ...
805   ...
806   ...
807   ...
808   ...
809   ...
810   ...
811   ...
812   ...
813   ...
814   ...
815   ...
816   ...
817   ...
818   ...
819   ...
820   ...
821   ...
822   ...
823   ...
824   ...
825   ...
826   ...
827   ...
828   ...
829   ...
830   ...
831   ...
832   ...
833   ...
834   ...
835   ...
836   ...
837   ...
838   ...
839   ...
840   ...
841   ...
842   ...
843   ...
844   ...
845   ...
846   ...
847   ...
848   ...
849   ...
850   ...
851   ...
852   ...
853   ...
854   ...
855   ...
856   ...
857   ...
858   ...
859   ...
860   ...
861   ...
862   ...
863   ...
864   ...
865   ...
866   ...
867   ...
868   ...
869   ...
870   ...
871   ...
872   ...
873   ...
874   ...
875   ...
876   ...
877   ...
878   ...
879   ...
880   ...
881   ...
882   ...
883   ...
884   ...
885   ...
886   ...
887   ...
888   ...
889   ...
890   ...
891   ...
892   ...
893   ...
894   ...
895   ...
896   ...
897   ...
898   ...
899   ...
900   ...
901   ...
902   ...
903   ...
904   ...
905   ...
906   ...
907   ...
908   ...
909   ...
910   ...
911   ...
912   ...
913   ...
914   ...
915   ...
916   ...
917   ...
918   ...
919   ...
920   ...
921   ...
922   ...
923   ...
924   ...
925   ...
926   ...
927   ...
928   ...
929   ...
930   ...
931   ...
932   ...
933   ...
934   ...
935   ...
936   ...
937   ...
938   ...
939   ...
940   ...
941   ...
942   ...
943   ...
944   ...
945   ...
946   ...
947   ...
948   ...
949   ...
950   ...
951   ...
952   ...
953   ...
954   ...
955   ...
956   ...
957   ...
958   ...
959   ...
960   ...
961   ...
962   ...
963   ...
964   ...
965   ...
966   ...
967   ...
968   ...
969   ...
970   ...
971   ...
972   ...
973   ...
974   ...
975   ...
976   ...
977   ...
978   ...
979   ...
980   ...
981   ...
982   ...
983   ...
984   ...
985   ...
986   ...
987   ...
988   ...
989   ...
990   ...
991   ...
992   ...
993   ...
994   ...
995   ...
996   ...
997   ...
998   ...
999   ...
1000  ...
1001  ...
1002  ...
1003  ...
1004  ...
1005  ...
1006  ...
1007  ...
1008  ...
1009  ...
1010  ...
1011  ...
1012  ...
1013  ...
1014  ...
1015  ...
1016  ...
1017  ...
1018  ...
1019  ...
1020  ...
1021  ...
1022  ...
1023  ...
1024  ...
1025  ...
1026  ...
1027  ...
1028  ...
1029  ...
1030  ...
1031  ...
1032  ...
1033  ...
1034  ...
1035  ...
1036  ...
1037  ...
1038  ...
1039  ...
1040  ...
1041  ...
1042  ...
1043  ...
1044  ...
1045  ...
1046  ...
1047  ...
1048  ...
1049  ...
1050  ...
1051  ...
1052  ...
1053  ...
1054  ...
1055  ...
1056  ...
1057  ...
1058  ...
1059  ...
1060  ...
1061  ...
1062  ...
1063  ...
1064  ...
1065  ...
1066  ...
1067  ...
1068  ...
1069  ...
1070  ...
1071  ...
1072  ...
1073  ...
1074  ...
1075  ...
1076  ...
1077  ...
1078  ...
1079  ...
1080  ...
1081  ...
1082  ...
1083  ...
1084  ...
1085  ...
1086  ...
1087  ...
1088  ...
1089  ...
1090  ...
1091  ...
1092  ...
1093  ...
1094  ...
1095  ...
1096  ...
1097  ...
1098  ...
1099  ...
1100  ...
1101  ...
1102  ...
1103  ...
1104  ...
1105  ...
1106  ...
1107  ...
1108  ...
1109  ...
1110  ...
1111  ...
1112  ...
1113  ...
1114  ...
1115  ...
1116  ...
1117  ...
1118  ...
1119  ...
1120  ...
1121  ...
1122  ...
1123  ...
1124  ...
1125  ...
1126  ...
1127  ...
1128  ...
1129  ...
1130  ...
1131  ...
1132  ...
1133  ...
1134  ...
1135  ...
1136  ...
1137  ...
1138  ...
1139  ...
1140  ...
1141  ...
1142  ...
1143  ...
1144  ...
1145  ...
1146  ...
1147  ...
1148  ...
1149  ...
1150  ...
1151  ...
1152  ...
1153  ...
1154  ...
1155  ...
1156  ...
1157  ...
1158  ...
1159  ...
1160  ...
1161  ...
1162  ...
1163  ...
1164  ...
1165  ...
1166  ...
1167  ...
1168  ...
1169  ...
1170  ...
1171  ...
1172  ...
1173  ...
1174  ...
1175  ...
1176  ...
1177  ...
1178  ...
1179  ...
1180  ...
1181  ...
1182  ...
1183  ...
1184  ...
1185  ...
1186  ...
1187  ...
1188  ...
1189  ...
1190  ...
1191  ...
1192  ...
1193  ...
1194  ...
1195  ...
1196  ...
1197  ...
1198  ...
1199  ...
1200  ...
1201  ...
1202  ...
1203  ...
1204  ...
1205  ...
1206  ...
1207  ...
1208  ...
1209  ...
1210  ...
1211  ...
1212  ...
1213  ...
1214  ...
1215  ...
1216  ...
1217  ...
1218  ...
1219  ...
1220  ...
1221  ...
1222  ...
1223  ...
1224  ...
1225  ...
1226  ...
1227  ...
1228  ...
1229  ...
1230  ...
1231  ...
1232  ...
1233  ...
1234  ...
1235  ...
1236  ...
1237  ...
1238  ...
1239  ...
1240  ...
1241  ...
1242  ...
1243  ...
1244  ...
1245  ...
1246  ...
1247  ...
1248  ...
1249  ...
1250  ...
1251  ...
1252  ...
1253  ...
1254  ...
1255  ...
1256  ...
1257  ...
1258  ...
1259  ...
1260  ...
1261  ...
1262  ...
1263  ...
1264  ...
1265  ...
1266  ...
1267  ...
1268  ...
1269  ...
1270  ...
1271  ...
1272  ...
1273  ...
1274  ...
1275  ...
1276  ...
1277  ...
1278  ...
1279  ...
1280  ...
1281  ...
1282  ...
1283  ...
1284  ...
1285  ...
1286  ...
1287  ...
1288  ...
1289  ...
1290  ...
1291  ...
1292  ...
1293  ...
1294  ...
1295  ...
1296  ...
1297  ...
1298  ...
1299  ...
1300  ...
1301  ...
1302  ...
1303  ...
1304  ...
1305  ...
1306  ...
1307  ...
1308  ...
1309  ...
1310  ...
1311  ...
1312  ...
1313  ...
1314  ...
1315  ...
1316  ...
1317  ...
1318  ...
1319  ...
1320  ...
1321  ...
1322  ...
1323  ...
1324  ...
1325  ...
1326  ...
1327  ...
1328  ...
1329  ...
1330  ...
1331  ...
1332  ...
1333  ...
1334  ...
1335  ...
1336  ...
1337  ...
1338  ...
1339  ...
1340  ...
1341  ...
1342  ...
1343  ...
1344  ...
1345  ...
1346  ...
1347  ...
1348  ...
1349  ...
1350  ...
1351  ...
1352  ...
1353  ...
1354  ...
1355  ...
1356  ...
1357  ...
1358  ...
1359  ...
1360  ...
1361  ...
1362  ...
1363  ...
1364  ...
1365  ...
1366  ...
1367  ...
1368  ...
1369  ...
1370  ...
1371  ...
1372  ...
1373  ...
1374  ...
1375  ...
1376  ...
1377  ...
1378  ...
1379  ...
1380  ...
1381  ...
1382  ...
1383  ...
1384  ...
1385  ...
1386  ...
1387  ...
1388  ...
1389  ...
1390  ...
1391  ...
1392  ...
1393  ...
1394  ...
1395  ...
1396  ...
1397  ...
1398  ...
1399  ...
1400  ...
1401  ...
1402  ...
1403  ...
1404  ...
1405  ...
1406  ...
1407  ...
1408  ...
1409  ...
1410  ...
1411  ...
1412  ...
1413  ...
1414  ...
1415  ...
1416  ...
1417  ...
1418  ...
1419  ...
1420  ...
1421  ...
1422  ...
1423  ...
1424  ...
1425  ...
1426  ...
1427  ...
1428  ...
1429  ...
1430  ...
1431  ...
1432  ...
1433  ...
1434  ...
1435  ...
1436  ...
1437  ...
1438  ...
1439  ...
1440  ...
1441  ...
1442  ...
1443  ...
1444  ...
1445  ...
1446  ...
1447  ...
1448  ...
1449  ...
1450  ...
1451  ...
1452  ...
1453  ...
1454  ...
1455  ...
1456  ...
1457  ...
1458  ...
1459  ...
1460  ...
1461  ...
1462  ...
1463  ...
1464  ...
1465  ...
1466  ...
1467  ...
1468  ...
1469  ...
1470  ...
1471  ...
1472  ...
1473  ...
1474  ...
1475  ...
1476  ...
1477  ...
1478  ...
1479  ...
1480  ...
1481  ...
1482  ...
1483  ...
1484  ...
1485  ...
1486  ...
1487  ...
1488  ...
1489  ...
1490  ...
1491  ...
1492  ...
1493  ...
1494  ...
1495  ...
1496  ...
1497  ...
1498  ...
1499  ...
1500  ...
1501  ...
1502  ...
1503  ...
1504  ...
1505  ...
1506  ...
1507  ...
1508  ...
1509  ...
1510  ...
1511  ...
1512  ...
1513  ...
1514  ...
1515  ...
1516  ...
1517  ...
1518  ...
1519  ...
1520  ...
1521  ...
1522  ...
1523  ...
1524  ...
1525  ...
1526  ...
1527  ...
1528  ...
1529  ...
1530  ...
1531  ...
1532  ...
1533  ...
1534  ...
1535  ...
1536  ...
1537  ...
1538  ...
1539  ...
1540  ...
1541  ...
1542  ...
1543  ...
1544  ...
1545  ...
1546  ...
1547  ...
1548  ...
1549  ...
1550  ...
1551  ...
1552  ...
1553  ...
1554  ...
1555  ...
1556  ...
1557  ...
1558  ...
1559  ...
1560  ...
1561  ...
1562  ...
1563  ...
1564  ...
1565  ...
1566  ...
1567  ...
1568  ...
1569  ...
1570  ...
1571  ...
1572  ...
1573  ...
1574  ...
1575  ...
1576  ...
1577  ...
1578  ...
1579  ...
1580  ...
1581  ...
1582  ...
1583  ...
1584  ...
1585  ...
1586  ...
1587  ...
1588  ...
1589  ...
1590  ...
1591  ...
1592  ...
1593  ...
1594  ...

```

6. Файловая система, задачи файловой системы и иерархическая организация ФС. Файловая подсистема LINUX: поддержка большого числа файловых систем и структура, описывающая файловую систему. VFS: четыре основные структуры файловой системы и связь между ними. Раздел жесткого диска и суперблок. Структура *struct super_operations*. Мониторинг файловой системы, команда *mount* и функции мониторинга и их параметры, точка монтирования — корневой каталог и *inode*. Пример (*lab. rab.*)

6.1 Файловая система

Файл — поименованная совокупность данных, может быть бессмысленная, хранящаяся во вторичной памяти. Понятие файла связано с долговременным хранением информации.

Файловая система — это порядок, определяющий способ организации хранения, именования и доступа к данным на вторичных носителях информации.

Файловая система используется для управления хранением данных и доступа к ним. ФС должна обеспечить доступ к хранимой в системе на вторичных дисках информации.

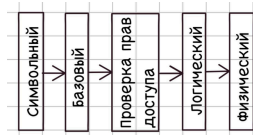
6.2 Задачи файловой системы

- 1) Долговременное хранение данных
- 2) Обеспечение доступа к хранившимся данным

6.3 Иерархическая организация ФС

Для обеспечения доступа к данным нужно предоставить пользователю достаточно удобный способ доступа. Уровни файловой системы:

- 1) Символьный
- 2) Базовый
- 3) Проверка прав доступа
- 4) Логический
- 5) Физический



- **Символьный уровень** — именование файлов: у файлов **имена** (массив символов), собранные по каталогам; корневой каталог в UNIX/Linux — /. Имя не является идентификатором файла, а лишь позволяет удобно с ним работать; идентификатором файла является *inode*; определены жесткие (*hard links*) и символические (*soft links*) ссылки.
- **Базовый уровень** — идентификация файла: это *inode*; идентификатором является номер *inode*, который описывает физический файл.
- **Проверка прав доступа** — права отдельно для пользователя, группы и остальных (команда *chmod*).
- **Логический уровень** — процесс открывает файл и выполняет над ним операции: файл считает, что начинается с нулевого смещения (*offset*), каждая операция чтения/записи смещает указатель; файл байт-ориентированный, размера не имеет. Здесь определяется адресация данных, преобразуемая в физические адреса на диске.
- **Физический уровень** — уровень внешнего ЗУ; логический и физический уровни связывает драйвер дискового устройства, переводящий смещения в физические адреса.

Файловые системы знакомых нам ОС являются иерархическими — это негласное правило, впервые определенное именно в UNIX. Для Unix характерно существование корневой каталога /, от которого ветвится древо каталогов:

```
1 /
2 | bin
3 | etc
4 | dev
5 | usr
6 | local
7 | lib
```

93 / 257

Связь структур строится относительно процесса (именно процессы открывают файлы): через *task_struct* по файловому дескриптору находится *struct file*; его поле *f_path/f_inode* ведет к *dentry* и *inode*, а из *inode* (*inode i_sb*) и со стороны монтирования — к *super_block* смонтированной ФС.

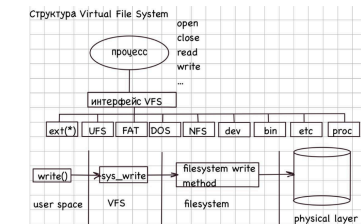


Рис. 18. Структура VFS

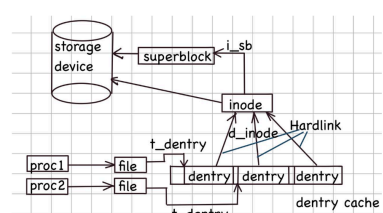


Рис. 19. Схема структур

6.6 Раздел жесткого диска и суперблок

Адресное пространство диска делится на разделы, один раздел занимает одна ФС.

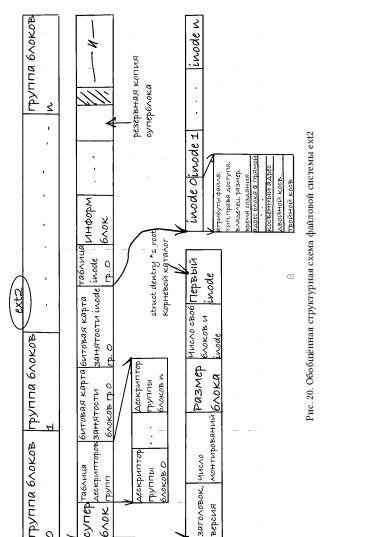


Рис. 20. Обычная структурная схема файловой системы ext2

99 / 257

6.4 Файловая подсистема LINUX: поддержка большого числа файловых систем и структура, описывающая файловую систему

6.4.1 Поддержка большого числа файловых систем

Для Unix был предложен интерфейс *VFS/inode* (*virtual file system, virtual node*). Linux отказался от *inode*, оставив только *VFS*. *VFS* — интерфейс, через который ОС получает доступ к большому числу разных ФС, сократив до минимума количество системных вызовов: любая работа с файлом любой ФС идет через один и тот же интерфейс. Основа такой работы — монтирование: прежде чем ФС станет доступна (станут видны её каталоги и файлы), она должна быть смонтирована.

6.4.2 Структура, описывающая файловую систему

Для описания возможностей и характеристик каждой поддерживаемой ФС в ядре определена структура *file_system_type*. Это тип ФС, которая будет монтироваться; инициализируя её поля, мы определяем собственный тип ФС.

```
1 struct file_system_type {
2     const char *name;
3     int fs_flags;
4     struct dentry *(*mount) (struct file_system_type *, int, const char *, void *);
5     void (*kill_sb) (struct super_block *);
6     struct module *owner;
7     struct file_system_type *next;
8     struct list_head fs_supers;
9 };
```

Листинг 27. Структура *file_system_type* (собр.)

- *name* — имя ФС.
- *fs_flags* — флаги типа ФС; для ФС, работающей с внешним ЗУ, нужен флаг *FS_REQUIRES_DEV* (от него зависит вызываемые функции монтирования). Флаг *FS_USERS_MOUNT* означает, что обязательно все выполнять в ядре.
- *mount, kill_sb* — монтирование и прекращение доступа к суперблоку.
- *owner* — модуль-владелец ФС; нужен для сечки ссылки (модуль не выгружается, пока ФС примонтирована).
- *next* — связный список *file_system_type*.
- *fs_supers* — список суперблоков.

Для каждого типа файловой системы существует только одна структура *file_system_type*, независимо от того, сколько экземпляров этой ФС смонтировано: такая ФС может быть смонтирована в системе много раз, но зарегистрирована будет

95 / 257

Суперблок представляет в VFS информацию о смонтированной ФС — структура с данными, необходимыми для монтирования и доступа к файлам ФС: метаданные для обращения к файлам, хранищимся «вразброс» (адресация участков диска). В каждой ФС ровно один суперблок, он располагается в начале раздела (есть резервная копия). При монтировании суперблок считывается в память ядра и находится там до размонтирования.

```
1 struct super_block {
2     struct list_head s_list;
3     dev_t s_dev;
4     unsigned long s_blocksize;
5     loff_t s_maxbytes; // Max file size
6     struct file_system_type *s_type;
7     const struct super_operations *s_op;
8     unsigned long s_flags;
9     unsigned long s_magic;
10    struct dentry *s_root; // точка монтирования
11    struct rw_semaphore s_mounts;
12    struct list_head s_mounts;
13    struct block_device *s_bdev;
14    struct list_head s_inodes; // all inodes
15 };
```

Листинг 28. Структура *super_block* (собр.)

- *s_list* — список подмонтированных ФС (*list_head* — двусвязный список); *s_dev*, *s_bdev* — устройство и базовое устройство ФС.
- *s_blocksize*, *s_maxbytes* — размер блока и максимальный размер файла.
- *s_type*, *s_op* — тип ФС и функции на суперблоке; *s_flags* — флаги монтирования;
- *s_magic* — магическое число ФС.
- *s_root* — точка монтирования (корневой каталог); *s_inodes* — список всех *inode*.
- *s_mount* — семафор «читатели-писатели» для монтирования; *s_mounts* — список монтированных данной ФС (одна и та же ФС может быть смонтирована много раз).

⚠ Осторожно

Вместо первого рисунка суперблока, который Ризанова не показывала (показывала), вероятно, хочет видеть второй.

100 / 257

1 раз. Для регистрации достаточно полей *owner* и *name*; зарегистрировать ФС можно вызовом *register_filesystem*, для дерегистрации — *unregister_filesystem*.

6.5 VFS: четыре основные структуры файловой системы и связь между ними

В основе интерфейса VFS лежат 4 структуры (это ядреные структуры):

- 1) *struct super_block*;
- 2) *struct inode*;
- 3) *struct dentry*;
- 4) *struct file*.



Рис. 17. Virtual File System

Все четыре структуры взаимодействуют между собой указателями, и для каждого файла создаются их экземпляры. Кратко их назначение: — *super_block* — описывает смонтированную ФС (метаданные для обращения к файлам, хранищимся «вразброс»); — *inode* — дескриптор физического файла (идентификатор файла — номер *inode*); — *dentry* — элемент пути (имя); создается «на лету», на диске не хранится; — *file* — дескриптор открытого файла, с которым работает процесс.

96 / 257

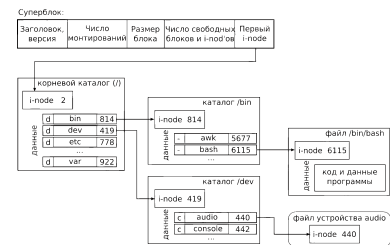


Рис. 21. Суперблок

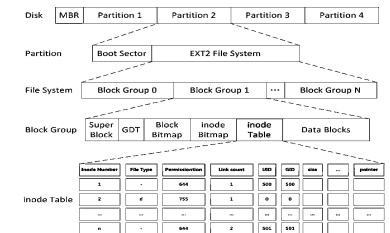


Рис. 22. Суперблок

97 / 257

98 / 257

6.7 Структура struct super_operations

Для каждой ФС определяется свой набор операций над объектами суперблока. Поскольку разработчики Linux ставили задачу обеспечить интерфейс для большого числа ФС, перечень операций на объектах суперблока широк:

```
1 struct super_operations {
2     struct inode *(*alloc_inode)(struct super_block *sb);
3     void (*destroy_inode)(struct inode *);
4     void (*free_inode)(struct inode *);
5     void (*dirty_inode)(struct inode *, int flags);
6     int (*write_inode)(struct inode *, struct writeback_control *wbc);
7     int (*drop_inode)(struct inode *);
8
9     void (*put_super)(struct super_block *);
10    ...
11    int (*freeze_fs)(struct super_block *);
12    ...
13    int (*remount_fs)(struct super_block *, int *, char *);
14    void (*umount_begin)(struct super_block *);
15    int (*show_options)(struct seq_file *, struct dentry *);
16    ...
17    int (*show_path)(struct seq_file *, struct dentry *);
18    ...
19};
```

Листинг 29. struct super_operations

Основное количество перечисляемых операций — это операции с inode:

- **alloc_inode** — создает и инициализирует новый объект inode для конкретной смонтированной ФС (выделяет адресное пространство);
- **destroy_inode** — удаляет inode из своего списка inode;
- **free_inode** — информация с метаданными (с номером inode) не удаляется, и данный inode может быть повторно использован;
- **dirty_inode** — вызывается, чтобы переместить inode в другой список, если в него были введены изменения;
- **write_inode** — записывает inode на диск (только alloc недостаточно);
- **drop_inode** — вызывает интерфейс VFS, когда количество ссылок на inode равно 0;
- **put_super** — вызывается VFS при размонтировании ФС; данный суперблок изымается из своего списка монтираний;
- **remount_fs** — вызывается VFS для монтирования суперблока с другими параметрами;

102 / 257

— **umount_begin** — вызывается VFS для прерывания монтирования; используется сетевыми ФС, например NFS (Network File System).

В перечне super_operations нет функции создания суперблока: сам суперблок создает отдельная функция alloc_super (не входит в struct super_operations), которая выделяет и инициализирует новую структуру super_block и возвращает указатель на нее.

6.8 Монтирование файловой системы, команда mount и функции монтирования и их параметры, точка монтирования — корневой каталог и inode

6.8.1 Монтирование файловой системы. Точка монтирования

Дерево файлов и каталогов в Linux формируется из сетей, соответствующих разным носителям (разным ФС). Но это одно большое дерево; некоторые каталоги могут быть точками монтирования.

Монтирование — система действий, делающая ФС (и все её файлы) доступной. Точка монтируемой ФС становится файлами и поддиректориями точки монтирования, а её прежние файлы скрываются до размонтирования (поэтому лучше использовать пустые каталоги в качестве точки монтирования). Доступ к файлам ФС возможен только после монтирования. В Unix/Linux всегда есть корневая ФС (root fs); её монтирование — часть инициализации системы.

Когда ФС монтируется, создается структура struct vfsmount, которая представляет конкретный экземпляр смонтированной ФС:

```
1 struct vfsmount {
2     struct dentry *mnt_root; /* root of the mounted tree */
3     struct super_block *mnt_sb; /* pointer to superblock */
4     int mnt_flags;
5     struct mnt_idmap *mnt_idmap;
6};
```

Листинг 30. Структура vfsmount

mnt_root — корень подмонтированного дерева, mnt_sb — указатель на суперблок, mnt_flags — флаги монтирования.

6.8.2 Команда mount

Команда mount имеет следующий синтаксис:

```
1 mount <устройство> <точка монтирования>
```

Ключи: -t <тип ФС> -o <опции ФС>; то есть полная форма:

```
1 mount -t type device dir
```

Точка монтирования — любой каталог; в самой ФС нужно создать корневой каталог (root) — именно он монтируется. Ключ -t необязателен: система сама может определить тип ФС. После монтирования каталоги и файлы ФС доступны. Обычно монтирование требует прав суперпользователя.

С помощью команды mount [-l] [-t type] можно получить список смонтированных файловых систем типа type. Размонтируется ФС командой umount <точка монтирования>.

6.8.3 Функции монтирования и их параметры

В ядре определено несколько функций mount(); все они имеют модификатор extern и возвращают значение типа struct dentry*. В версии ядра 6.17.13, в отличие от предыдущих подверсий, определено только две:

```
1 extern struct dentry *mount_bdev(struct file_system_type *fs_type, int flags,
2                                 const char *dev_name, void *data, int (*fill_super)
3                                 (struct super_block *, void *, int));
4 extern struct dentry *mount_nodev(struct file_system_type *fs_type, int flags,
5                                 void *data, int (*fill_super)
6                                 (struct super_block *, void *, int));
```

Параметры функций монтирования:

- fs_type — тип файловой системы;
- flags — флаги монтирования;
- dev_name — имя устройства (только у mount_bdev; внешнее отличие двух функций именно в этом параметре);
- data — произвольные данные монтирования;
- fill_super — указатель на функцию заполнения суперблока.

Различаются функции существенно кодом и областью применения: — mount_bdev — для монтирования ФС, находящейся на базовом устройстве; — mount_nodev — для монтирования ФС, не связанной ни с каким устройством.

Функциям монтирования передается функция fill_super — она выполняет основные действия для монтирования: инициализирует поля struct super_block (размер блока, смещение, магическое число, super_operations). В системе существует готовая функция simple_fill_super, которую находим в коде ядра и используем как основу для собственной fill_super.

104 / 257

Когда файловая система отмонтируется, всё должно быть удалено — этим занимается функция kill_sb из file_system_type. В ядре для этого определены функции:

```
1 void generic_shutdown_super(struct super_block *sb);
2 void kill_block_super(struct super_block *sb);
3 void kill_anon_super(struct super_block *sb);
4 void kill_litter_super(struct super_block *sb);
```

Общую работу по удалению суперблока выполняет generic_shutdown_super.

6.8.4 Точка монтирования — корневой каталог и inode
Функции монтирования возвращают объект dentry, которым должен быть root: точкой монтирования является корневой каталог ФС. Поэтому, чтобы смонтировать ФС, в ней нужно создать корневой каталог (root) — а struct super_block это поле s_root.

Чтобы создать root, надо проинициализировать inode, выделяя для него память функцией new_inode() (внутри неё виден аллокатор — inode аллоцируется в slab-кеше):

```
1 struct inode *new_inode(struct super_block *sb) {
2     struct inode *inode = new_inode_pseudo(sb); // -> alloc_inode(sb); выделение
3     no slab
4     if (inode)
5         inode_sb_list_add(inode);
6     return inode;
7 }
```

Листинг 31. Функция new_inode (coap.)

Из созданного inode функцией d_make_root делается корневой dentry, который записывается в поле s_root суперблока. Так точка монтирования (корневой каталог) оказывается связана с корнем inode.

6.9 Пример (лаб. раб.)

В лабораторной работе создается собственная (виртуальная) файловая система в загружаемом модуле ядра; по отношению к VFS выполняется два действия:

- 1) **Регистрация** ФС: объявляем свой struct file_system_type (в поле — myfs_type; минимально owner, name, плюс поля mount и kill_sb); в init(myfs_init) вызываем register_filesystem, а exit(myfs_exit) — unregister_filesystem.
- 2) **Монтирование** ФС: в своей функции mount(myfs_mount) вызываем mount_nodev (ФС не связана с устройством), передавая ей свою fill_super (в коде myfs_fill_sb, написана на основе simple_fill_super). Именно myfs_fill_sb инициализирует поля struct super_block (размер блока, магическое число,

свой block super_operations), через new_inode создает inode корневого каталога (с готовыми <simple-операциями каталога) и функцией d_make_root делает из него root, записывая его в s_root. Освобождение суперблока поручено готовой функции ядра kill_litter_super (поле kill_sb); чтобы при размонтировании в super_block сообщали, а полям набора super_operations задается собственный put_super(myfs_put_super).

После загрузки модуля ФС монтируется командой mount и становится видна в дереве каталогов (может быть смонтирована несколько раз). Результат показывают ls -la /, cat /proc/mounts (смонтированные ФС) и cat /proc/filesystems (зарегистрированный тип ФС).

```
1 #include <linux/module.h>
2 #include <linux/kernel.h>
3 #include <linux/init.h>
4 #include <linux/fs.h>
5 #include <asm/atomic.h>
6 #include <asm/uaccess.h>
7
8 MODULE_LICENSE("GPL");
9 MODULE_AUTHOR("2012in kskita");
10
11 #define MYFS_MAGIC_NUMBER 0x13131313
12 static void myfs_put_super(struct super_block *sb) {
13     printk("myfs: put_super\n");
14 }
15 static struct super_operations const myfs_super_ops = {
16     .put_super = myfs_put_super,
17     .stats = &simple_stats,
18     .drop_inode = generic_delete_inode
19 };
20 static int myfs_fill_sb(struct super_block *sb, void *data, int silent) {
21     struct inode *root;
22     struct dentry *root;
23     sb->s_blocksize = PAGE_SIZE;
24     sb->s_blocksize_bits = PAGE_SHIFT;
25     sb->s_magic = MYFS_MAGIC_NUMBER;
26     sb->s_op = &myfs_super_ops;
27     sb->s_time_gran = 1;
28     inode = new_inode(sb);
29     if (!inode) {
30         printk(KERN_ERR "myfs: new_inode error\n");
31         return -ENOMEM;
32     }
```

106 / 257

```
33 inode->i_ino = 1;
34 inode->i_mode = S_IFDIR | 0755;
35 simple_inode_init(&inode);
36
37 inode->i_op = &simple_dir_operations;
38 inode->i_fop = &simple_dir_operations;
39
40 root = d_make_root(inode);
41 printk("myfs: d_make_root\n");
42 if (!root) {
43     printk(KERN_ERR "myfs: d_make_root error\n");
44     iput(inode);
45     return -ENOMEM;
46 }
47 sb->s_root = root;
48 printk(KERN_INFO "myfs: VFS root created\n");
49 printk(KERN_INFO "myfs: super_block init");
50 return 0;
51 }
52 static struct dentry *myfs_mount(struct file_system_type *type, int flags, const
char *dev, void *data) {
53     struct dentry *const entry = mount_nodev(type, flags, data, myfs_fill_sb);
54     if (!IS_ERR(entry)) {
55         printk(KERN_ERR "myfs: mount error\n");
56     } else
57         printk(KERN_INFO "myfs: mount\n");
58     return entry;
59 }
60 static struct file_system_type myfs_type = {
61     .owner = THIS_MODULE,
62     .name = "myfs",
63     .mount = myfs_mount,
64     .kill_sb = kill_litter_super
65 };
66 static int __init myfs_init(void) {
67     int ret;
68
69     printk(KERN_INFO "myfs: register_filesystem call\n");
70     ret = register_filesystem(&myfs_type);
71     if (ret) {
72         printk(KERN_ERR "myfs: register_filesystem error\n");
73     }
74     return ret;
75 }
76 static int __init myfs_init(void) {
77     return 0;
78 }
```

107 / 257

```
77 }
78 static void __exit myfs_exit(void) {
79     int ret;
80     ret = unregister_filesystem(&myfs_type);
81     if (ret)
82         printk(KERN_ERR "myfs: unregister_filesystem error\n");
83     else
84         printk("myfs: unregister_filesystem\n");
85     printk(KERN_INFO "myfs: unloading\n");
86 }
87 module_init(myfs_init);
88 module_exit(myfs_exit);
```

108 / 257

7

7 Виртуальная файловая система /proc: структура proc_dir_entry, создание файла proc, директории и символические ссылки; точки входа и их регистрация, интерфейсы передачи данных из ядра в пространство пользователя — реализация и особенности. Примеры из лабораторных работ.

7.1 Виртуальная файловая система /proc

Виртуальная файловая подсистема /proc (process information pseudo-file-system) позволяет в режиме пользователя получить большой объем информации о ресурсе процесса. Виртуальная — то есть не существующая на самом деле (не хранящаяся во вторичной памяти), создающаяся "на лету" по мере обращения к данным. По сути интерфейс /proc позволяет получить информацию из структур ядра, не обращаясь к ним напрямую, а потому работает в режиме пользователя. Большинство файлов доступны только для чтения, однако некоторые доступны для записи, что позволяет изменять переменные ядра.

ФС /proc является немонтируемой. Под этим подразумевается, что она не связана ни с каким внешним носителем (она видна в файловых системах, подключенных к корневому каталогу, но не связана ни с каким внешним устройством. Поэтому она и называется псевдофайловой системой).

Для каждого процесса в /proc определена директория идентификатора процесса (/proc/<pid>), она будет создана налету. В ней хранятся файлы, директории и символические ссылки, используя которые можно получить разную информацию о конкретном процессе. Вся информация доступна из usermode:

Элемент	Тип	Содержание
cmdline	Файл	holds the complete command line
cwd	Символическая ссылка	Указывает на рабочую директорию процесса
environ	Файл	Содержит список переменных окружения процесса
exe	Символическая ссылка	Указывает на образ процесса (на его файл)
		Символическая ссылка, содержащая фактический путь к выполняемой команде.
comm	Файл	Имя команды, связанное с процессом. Это именно то имя файла, которое приведет к выполнению нашего кода.

111 / 257

Элемент	Тип	Содержание
		Замечание 7.1 Разные потоки в одном и том же процессе могут иметь разные значения, доступные через <code>proc->pid/task->ctid/cmm.task</code> — <code>file</code>, в котором находится поддиректория потоков.
stat	Файл	Содержит информацию о процессе (<code>vsize</code> , <code>start</code> , <code>lbrk</code> и др.)
fd	Директория	Содержит ссылки на открытые процессом файлы Замечание 7.2 В системе различаются файлы и открытые файлы. Открытые файлы описываются структурой <code>struct file</code>. Простые файлы предоставляются в системе деревом каталогов, а описываются физические файлы <code>struct inode</code>. Один и тот же файл может быть открыт много раз, прочтен разными процессами. В таблице открытых файлов на каждый такой открытый файл будет дескриптор <code>struct file</code>
root	soft link	Это символическая ссылка, которая указывает на корень файловой системы процесса. Замечание 7.3 Процессы, которые выполняются на нашем компьютере, могут принадлежать разным файловым системам. Пока мы имеем маленького файла не указали в командной строке, пока о не начал выполняться, он лежит во второй памяти.
maps	Файл	Это адресное пространство того процесса, <code>pid</code> которого мы указали. В зависимости от <code>pid</code> информации будет отличаться, но есть неизменные составляющие:
pagemap	файл	Это 64-х разрядное значение для каждой строки процесса. При этом нас интересует фактически несколько бит: 1) 63 бит — <code>present</code> (in RAM) 2) 62 бит — <code>swap space</code>, область свопинга 3) 61 — <code>shared anonymous page</code> 4) 55 бит — <code>dirty</code>. Он указывается в дескрипторе страницы.

112 / 257

```
3 .proc_read = device_read,
4 .proc_write = device_write,
5 .proc_release = device_release,
6 );
```

Регистрация точек входа происходит при инициализации полей структуры `proc_ops` (до регистрации эти функции должны быть объявлены и описаны), а затем проинициализированная структура передаётся функции создания виртуального файла `proc_create`.

7.5 Интерфейсы передачи данных из ядра в пространство пользователя — реализация и особенности

В своих функциях `read`, `write` мы должны вызывать специальные функции ядра для передачи данных из пространства пользователя в пространство ядра и обратно: `copy_to_user` копирует данные из буфера ядра в буфер пользователя, `copy_from_user` — из буфера пользователя в буфер ядра.

Специальные функции нужны, потому что ядро и загружаемый модуль ядра, становящийся частью ядра, оперируют физическими адресами: ядро загружено в физическую память резидентно, а процесс имеет только виртуальные адреса пространства. Процесс получает ресурс «физическая страница» только по мере необходимости. При этом ядро, чтобы передать данные, должно обратиться к буферу пространства пользователя. Вероятна ситуация, когда буфер (страница), к которому обращается ядро, выгружен — в результате возникнет странное прерывание, которое должно обработать ядро (должен быть выполнен код ядра). Ядро обратилось, стало источником прерывания, оно же и должно его обработать; если не предпринять специальных мер, возникнет паника ядра.

```
1 #ifdef INLINE_COPY_TO_USER
2
3 static inline __must_check unsigned long
4 copy_to_user(void __user *to, const void *from, unsigned long n)
5 {
6     might_fault();
7     if (should_fail_usercopy())
8         return n;
9     if (access_ok(to, n)) {
10         instrument_copy_to_user(to, from, n);
11         n = raw_copy_to_user(to, from, n);
12     }
13     return n;
14 }
```

117 / 257

Элемент	Тип	Содержание
		С 54 по 0-ю если страница присутствует в физической памяти, то эти биты являются номером фрейма, то есть номером физической страницы.
task	Директория	Содержит поддиректорию потоков
interrupts	Файл	Предоставляет таблицу о количестве прерываний на каждом из процессоров в следующем виде: 1) Первая колонка: номер прерывания 2) Колонки CPUs: счётчики прерываний на каждом из процессоров 3) Следующая колонка: вид прерывания: 3.1) IO-APIC-edge — прерывание по фронту на контроллер I/O APIC. 3.2) IO-APIC-fasteoi — прерывание по уровню на контроллер I/O APIC. 3.3) PCI-MSI-edge — MSI прерывание. 3.4) XT-PIC-XT-PIC — прерывание на PIC контроллер. 4) Последняя колонка: устройство, ассоциированное с данным прерыванием.

Таблица 5. Содержание директории `/proc/<pid>`

7.2 Структура `proc_dir_entry`: функции для работы с элементами `proc`

Файлы и поддиректория файловой системы `/proc` описаны структурой `proc_dir_entry`.

```
1 struct proc_dir_entry {
2     atomic_t in_use;
3     // ...
4     struct list_head pde_openers;
5     // ...
6     const struct inode_operations *proc_ops;
7     union {
8         const struct proc_ops *proc_ops;
9         const struct file_operations *proc_dir_ops;
10    };
11
12    const struct dentry_operations *proc_dops;
13    union {
14        const struct seq_operations *seq_ops;
```

113 / 257

```
15 #else
16 #endif
17
18 extern __must_check unsigned long
19 copy_to_user(void __user *to, const void *from, unsigned long n);
20
21 #endif
```

Листинг 34. Определение функции `copy_to_user`

```
1 static inline __must_check unsigned long
2 copy_from_user(void __user *to, const void *from, unsigned long n)
3 {
4     ...
5 }
```

Листинг 35. Определение функции `copy_from_user`

Для `copy_to_user` указателя `to` — это указатель на буфер пользователя, `from` — указатель на буфер ядра, `n` — количество байт; для `copy_from_user` — обратная ситуация. Обе функции возвращают количество байт, которые не могут быть скопированы: в случае успешного выполнения будет возвращён 0.

7.6 Примеры из лабораторных работ

Этой теме посвящены 2 лабораторные работы, обе связаны с файлами VFS `proc`: работа ведётся в ядре, то есть пишется загружаемый модуль ядра.

7.6.1 IP «Фортунки»

Лабораторные относятся к теме передачи данных из пространства пользователя в пространство ядра и наоборот. В модуле 6 точек входа: `open`, `read`, `write`, `release`, `init`, `exit`. Функции регистрируются на виртуальном файле и вызываются автоматически процессами `echo` (запись строки-фортун) и `cat` (чтение); каждая функция пишет в `syslog` через `printk`, что она прочитала или записала. В `init` создаётся файл, директория и символическая ссылка (в `proc_create_data` передаётся созданная директория), а перед этим выделяется память для буфера, чтобы было куда записывать данные. В `read/write` данные передаются через `copy_to_user/copy_from_user`.

```
1 #define COOKIE_POT_SIZE 4096
2 #define FILENAME "fortune_file"
3 #define DIRNAME "fortune_dir"
4 #define SYMLINK "fortune_link"
5
6 static struct proc_dir_entry *fortune_file;
7 static struct proc_dir_entry *fortune_dir;
8 static struct proc_dir_entry *fortune_link;
```

118 / 257

```
15 int (*single_show)(struct seq_file *, void *);
16 };
17
18 proc_write_t write;
19 void *data;
20 unsigned int state_size;
21 unsigned int low_ino;
22 #ifdef __linux__
23 // ...
24 loff_t size;
25 struct proc_dir_entry *parent;
26 // ...
27 char *name;
28 // ...
29 #ifdef __linux__
30 char *inline_name[];
31 );
```

Листинг 32. Структура `struct proc_dir_entry`

- `atomic_t` — неделимая переменная, счётчик.
- `const struct inode_operations *proc_ops` — у файла `proc` должен быть `inode` и данные быть определены `inode_operations`.

— Первый `union` — интерфейс регистрации функций. До шестой версии ядра существовала только `struct file_operations "proc_dir_ops`. В качестве альтернативы определена структура `const struct proc_ops "proc_ops` — в ней практически такие же функции. Объясняется такое решение тем, что функции `struct file_operations` вызываются интенсивно: все драйверы вызывают функции, определённые в этой структуре. Чтобы не нагружать эти функции дополнительными задачами, для работы с VFS `proc`, которая является информационной (используется для получения разработчиками информации), введена отдельная структура.

— Второй `union` — здесь видна информация о двух `seq`-интерфейсах взаимодействия `user-mode`:

- `const struct seq_operations *seq_ops` — полный интерфейс, полный набор функций;
- `int (*single_show)(struct seq_file *, void *)` — упрощённый интерфейс.

— `proc_write_t write` — в самых старых версиях структуры было определено 2 функции: `read` и `write`, сейчас осталена только функция `write`. Передача данных из `user-mode` в `kernel-mode` — это `write`, обратно — `read`.

114 / 257

```
9
10 static char *cookie_pot = NULL;
11 char buffer(COOKIE_POT_SIZE);
12
13 static int fortune_open(struct inode *inode, struct file *file)
14 {
15     printk(KERN_INFO "fortune: open\n");
16     return 0;
17 }
18
19 static int fortune_release(struct inode *inode, struct file *file)
20 {
21     printk(KERN_INFO "fortune: release\n");
22     return 0;
23 }
24
25 static ssize_t fortune_write(struct file *file, const char __user *buf, size_t len, loff_t *ppos)
26 {
27     printk(KERN_INFO "fortune: write\n");
28     ...
29     if (copy_from_user(&cookie_pot[write_index], buf, len) != 0)
30     {
31         printk(KERN_ERR "fortune: copy_from_user error\n");
32         return -EFAULT;
33     }
34     ...
35     return len;
36 }
37
38 static ssize_t fortune_read(struct file *file, char __user *buf, size_t len, loff_t *ppos)
39 {
40     printk(KERN_INFO "fortune: read\n");
41     ...
42     read_len = sprintf(buffer, COOKIE_POT_SIZE, "%s\n", &cookie_pot[read_index]);
43     if (copy_to_user(buf, buffer, read_len) != 0)
44     {
45         printk(KERN_ERR "fortune: copy_to_user error\n");
46         return -EFAULT;
47     }
48     ...
49     return read_len;
50 }
```

Листинг 36. Точки входа `open`, `release`, `write`, `read` модуля «фортунки»

119 / 257

- `void *data` — приватные данные для работы с файлами `proc`.
- `unsigned int low_ino` — `inode` нижнего уровня (мы с ним не работаем). Поскольку `proc` — виртуальная файловая система, данные не хранятся во второй памяти, всё создаётся на лету в оперативной памяти, но `inode` у любого файла, в том числе и виртуального, всё равно имеется.
- `struct proc_dir_entry *parent` — указатель на родителя: поддерживается иерархия файловой системы `proc`.
- `loff_t size` — размер. О `type loff_t`: введено огромное количество типов, детализированных по назначению (ярчайший пример — `pid_t`, хотя мы понимаем, что это `int`). Это сделано для дополнительного контроля соответствия типов при компиляции.

7.3 Создание файла `proc`, директории и символические ссылки

Для создания файла виртуальной файловой системы `proc` в ядре имеется функция `proc_create_data`, для создания каталогов, используется функция `proc_mkdir`, символические ссылки создаются — с помощью `proc_symlink` (третий параметр — путь, на который указывает ссылка). Удаляется элемент `proc` (файл, директория или ссылка) функцией `remove_proc_entry`:

```
1 // создание файла
2 extern struct proc_dir_entry *proc_create_data
3 const char *name,
4 umode_t mode,
5 struct proc_dir_entry *parent,
6 const struct file_operations *proc_ops,
7 void *data);
8
9 struct proc_dir_entry *proc_create(
10 const char *name,
11 umode_t mode,
12 struct proc_dir_entry *parent,
13 const struct file_operations *proc_ops);
14
15 // создание директории
16 extern struct proc_dir_entry *proc_mkdir(
17 const char *name,
18 struct proc_dir_entry *parent);
19
20 // создание символической ссылки
21 extern struct proc_dir_entry *proc_symlink(
22 const char *name,
```

115 / 257

```
1 static int __init md_init(void) {
2     printk(KERN_INFO "fortune: init\n");
3     if (cookie_pot == vmalloc(COOKIE_POT_SIZE)) == NULL {
4         printk(KERN_ERR "fortune: memory error\n");
5         return -ENOMEM;
6     }
7     memset(cookie_pot, 0, COOKIE_POT_SIZE);
8     if (!fortune_file = proc_create(FILENAME, 0640, NULL, &fops)) == NULL {
9         ...
10    }
11    if (!fortune_dir = proc_mkdir(DIRNAME, NULL)) == NULL {
12        ...
13    }
14    if (!fortune_link = proc_symlink(SYMLINK, NULL, FILENAME)) == NULL {
15        ...
16    }
17    printk(KERN_INFO "fortune: loaded\n");
18    return 0;
19 }
20
21 static void __exit md_exit(void) {
22     printk(KERN_INFO "fortune: exit\n");
23     remove_proc_entry(FILENAME, NULL);
24     remove_proc_entry(DIRNAME, NULL);
25     remove_proc_entry(SYMLINK, NULL);
26     vfree(cookie_pot);
27 }
28
29 module_init(md_init);
30 module_exit(md_exit);
```

Листинг 37. Создание файла, директории и символической ссылки в `init` и их удаление в `exit`

120 / 257

```
23 struct proc_dir_entry *parent,
24 const char *);
25
26 // удаление
27 remove_proc_entry(
28 const char *name,
29 struct proc_dir_entry *parent);
```

Параметры:

- `name` — имя файла в `proc`;
- `mode` — права доступа (`read/write` для `user`, `group`, `others`);
- `parent` — указатель на родительскую директорию, то есть поддерживается иерархия файловой системы `proc`: чтобы файл создавался в поддиректории `proc`, нужно до `proc_create_data` вызвать `proc_mkdir` и передать эту директорию; если в параметре `parent` указать `NULL`, файл будет создаваться в корневом каталоге `proc`;
- `proc_ops` — указатель на проинициализированную структуру `proc_ops` с нашими функциями.

7.4 Точки входа и их регистрация

В загружаемом модуле ядра, работающем с файлом `proc`, 6 точек входа: `open`, `read`, `write`, `release`, `init`, `exit`.

Свои функции регистрируются инициализацией полей структуры `struct proc_ops` (для старых ядр — `struct file_operations`):

```
1 struct proc_ops {
2     ...
3     int (*proc_open)(struct inode *, struct file *);
4     ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
5     ssize_t (*proc_read_iter)(struct kiocb *, struct iov_iter *);
6     ssize_t (*proc_write)(struct file *, const char __user *, size_t, loff_t *);
7     loff_t (*proc_seek)(struct file *, loff_t, int);
8     int (*proc_release)(struct inode *, struct file *);
9     ...
10 };
```

Листинг 33. Структура `proc_ops`

Для стандарта C99 начиная с версии GCC 2.5 принят следующий синтаксис (назначение инициализаторы):

```
1 static const struct proc_ops device_ops = {
2     .proc_open = device_open,
```

116 / 257

```
1 static int __init md_init(void) {
2     printk(KERN_INFO "fortune: init\n");
3     if (cookie_pot == vmalloc(COOKIE_POT_SIZE)) == NULL {
4         printk(KERN_ERR "fortune: memory error\n");
5         return -ENOMEM;
6     }
7     memset(cookie_pot, 0, COOKIE_POT_SIZE);
8     if (!fortune_file = proc_create(FILENAME, 0640, NULL, &fops)) == NULL {
9         ...
10    }
11    if (!fortune_dir = proc_mkdir(DIRNAME, NULL)) == NULL {
12        ...
13    }
14    if (!fortune_link = proc_symlink(SYMLINK, NULL, FILENAME)) == NULL {
15        ...
16    }
17    printk(KERN_INFO "fortune: loaded\n");
18    return 0;
19 }
20
21 static void __exit md_exit(void) {
22     printk(KERN_INFO "fortune: exit\n");
23     remove_proc_entry(FILENAME, NULL);
24     remove_proc_entry(DIRNAME, NULL);
25     remove_proc_entry(SYMLINK, NULL);
26     vfree(cookie_pot);
27 }
28
29 module_init(md_init);
30 module_exit(md_exit);
```

```
1 static int __init md_init(void) {
2     printk(KERN_INFO "fortune: init\n");
3     if (cookie_pot == vmalloc(COOKIE_POT_SIZE)) == NULL {
4         printk(KERN_ERR "fortune: memory error\n");
5         return -ENOMEM;
6     }
7     memset(cookie_pot, 0, COOKIE_POT_SIZE);
8     if (!fortune_file = proc_create(FILENAME, 0640, NULL, &fops)) == NULL {
9         ...
10    }
11    if (!fortune_dir = proc_mkdir(DIRNAME, NULL)) == NULL {
12        ...
13    }
14    if (!fortune_link = proc_symlink(SYMLINK, NULL, FILENAME)) == NULL {
15        ...
16    }
17    printk(KERN_INFO "fortune: loaded\n");
18    return 0;
19 }
20
21 static void __exit md_exit(void) {
22     printk(KERN_INFO "fortune: exit\n");
23     remove_proc_entry(FILENAME, NULL);
24     remove_proc_entry(DIRNAME, NULL);
25     remove_proc_entry(SYMLINK, NULL);
26     vfree(cookie_pot);
27 }
28
29 module_init(md_init);
30 module_exit(md_exit);
```

Листинг 37. Создание файла, директории и символической ссылки в `init` и их удаление в `exit`

120 / 257

```
1 static int __init md_init(void) {
2     printk(KERN_INFO "fortune: init\n");
3     if (cookie_pot == vmalloc(COOKIE_POT_SIZE)) == NULL {
4         printk(KERN_ERR "fortune: memory error\n");
5         return -ENOMEM;
6     }
7     memset(cookie_pot, 0, COOKIE_POT_SIZE);
8     if (!fortune_file = proc_create(FILENAME, 0640, NULL, &fops)) == NULL {
9         ...
10    }
11    if (!fortune_dir = proc_mkdir(DIRNAME, NULL)) == NULL {
12        ...
13    }
14    if (!fortune_link = proc_symlink(SYMLINK, NULL, FILENAME)) == NULL {
15        ...
16    }
17    printk(KERN_INFO "fortune: loaded\n");
18    return 0;
19 }
20
21 static void __exit md_exit(void) {
22     printk(KERN_INFO "fortune: exit\n");
23     remove_proc_entry(FILENAME, NULL);
24     remove_proc_entry(DIRNAME, NULL);
25     remove_proc_entry(SYMLINK, NULL);
26     vfree(cookie_pot);
27 }
28
29 module_init(md_init);
30 module_exit(md_exit);
```

```
1 static int __init md_init(void) {
2     printk(KERN_INFO "fortune: init\n");
3     if (cookie_pot == vmalloc(COOKIE_POT_SIZE)) == NULL {
4         printk(KERN_ERR "fortune: memory error\n");
5         return -ENOMEM;
6     }
7     memset(cookie_pot, 0, COOKIE_POT_SIZE);
8     if (!fortune_file = proc_create(FILENAME, 0640, NULL, &fops)) == NULL {
9         ...
10    }
11    if (!fortune_dir = proc_mkdir(DIRNAME, NULL)) == NULL {
12        ...
13    }
14    if (!fortune_link = proc_symlink(SYMLINK, NULL, FILENAME)) == NULL {
15        ...
16    }
17    printk(KERN_INFO "fortune: loaded\n");
18    return 0;
19 }
20
21 static void __exit md_exit(void) {
22     printk(KERN_INFO "fortune: exit\n");
23     remove_proc_entry(FILENAME, NULL);
24     remove_proc_entry(DIRNAME, NULL);
25     remove_proc_entry(SYMLINK, NULL);
26     vfree(cookie_pot);
27 }
28
29 module_init(md_init);
30 module_exit(md_exit);
```

Листинг 37. Создание файла, директории и символической ссылки в `init` и их удаление в `exit`

121 / 257

8 Средства взаимодействия процессов — сокеты Беркли. Создание сокета — семейство, тип, протокол. Системный вызов `sys_socket()` и `struct socket`. Состояния сокета. Адресация сокетов и ее особенности для разных типов сокетов. Модель клиент-сервер. Сетевые сокеты — сетевой стек, аппаратный и сетевой порядок байтов. Примеры реализации взаимодействия процессов по модели клиент-сервер с использованием сокетов и мультиплексирования (tab. раб.).

8.1 Сокеты

Сокет — абстракция точки соединения. Являются универсальным средством меж-процессного взаимодействия: могут служить как для взаимодействия процессов на отдельно стоящей машине (память, процессор — всё в одном месте), так и для взаимодействия в распределённых системах (много машин, соединённых кабелями, у каждой машины своя память). Были разработаны Unix BSD, поэтому часто называются сокетами Беркли или сокетами BSD.

Определение 8.1 (Сокет)

Сокет — абстракция точки соединения.

Абстракция от особенностей взаимодействия. Средство универсальное: на конкретной машине особенности определяются тем, что это процессор и память, а в распределённой системе — тем, что память распределённая.

В компьютере обращение к данным всегда выполняется по адресу, но на отдельно стоящей машине один способ обращения, а в распределённых системах — другой, поскольку нужно иметь адреса хостов.

Разработаны парные сокеты — отдельный системный вызов `socketpair()`. Парные сокеты сделаны как **альтернатива** `pipe`, но в отличие от последних, которые обеспечивают симметричную связь, сокеты имеют **дуплексную связь**.

8.2 Создание сокета — семейство, тип, протокол

```
1 int socket(int family, int type, int protocol);
```

Системный вызов `socket()` создаёт конечную точку взаимодействия и возвращает файловый дескриптор, который ссылается на эту конечную точку при успешном вызове возвращается дескриптор с наименьшим номером, который в данный момент ещё не открыт для процесса.

Параметры системного вызова `socket()`:

- 1) `family` — пространство (семейство) адресов. В компьютере всё адресуется. Раньше чаще использовалось ключевое слово `domain`, сейчас — `family`.

123 / 257

В наших системах 2 адресных пространства: физической памяти и портов ввода/вывода. Порт — это адрес, но он локальный.

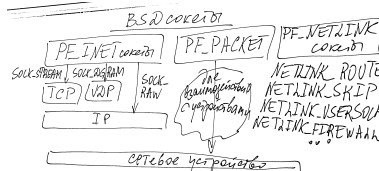


Рис. 23. BSD сокеты

8.5 Модель клиент-сервер

Когда клиент запрашивает соединение с сервером, сервер может либо принять, либо отклонить это соединение. Если соединение принято, то сервер устанавливает и поддерживает соединение с клиентом по определённому протоколу. Клиент не предоставляет общий доступ ни к одному из своих ресурсов, но запрашивает данные или службу у сервера.

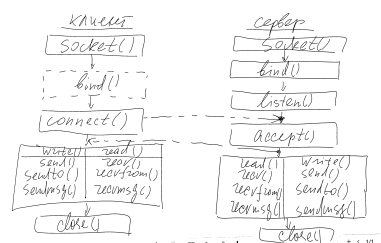


Рис. 24. Клиент-сервер на сокетах

Системный вызов `socket()` создаёт сокет и возвращает файловый дескриптор сокета. Затем на стороне сервера в обязательном порядке вызывается системный вызов `bind()`, который связывает файловый дескриптор сокета с адресом, то есть до `bind()` необходимо инициализировать поля структуры `struct sockaddr_in`. После этого на стороне сервера вызывается системный вызов `listen()`, который переводит серверный процесс в режим пассивного ожидания соединения. Когда соединение произошло, то есть клиент на своей стороне выполнил системный вызов `connect()`, ядро выполняет предварительную регистрацию соединения и выполняет системный вызов `accept()`. После этого начинается обмен данными.

Системный вызов bind(). Назначает сокету `sockfd` адрес, указанный `addr`; параметр `addrlen` — размер в байтах структуры адреса, на которую указывает `addr`. Для сетевых сокетов адрес состоит из IP-адреса сетевого интерфейса локальной системы и номера порта. Клиенты могут не вызывать `bind()`, так как их адрес часто никакой роли не играет — в таком случае адрес назначается им автоматически.

```
1 #include <sys/socket.h>
2 int bind(int sockfd, struct sockaddr *addr, int addrlen);
```

Листинг 42. Объявление функции bind()

127 / 257

- 1.1) `AF_UNIX` — сокеты для взаимодействия параллельных процессов на отдельно стоящих машинах. Для него используется тип `SOCK_DGRAM`.

- 1.2) `AF_INET` — имеет тип `SOCK_DGRAM`.

- 1.3) `AF_INET6`

- 1.4) `AF_IPX`

- 1.5) `AF_UNSPEC`

- 2) `type` — тип сокета. В зависимости от того, где используется сокет, у него будет соответствующий тип.

- 2.1) `SOCK_STREAM` — определяет ориентированное по потоку надёжное упорядоченное полнодуплексное логическое соединение двух сокетов (между двумя сокетами).

- 2.2) `SOCK_DGRAM` — определяет ненадёжную службу дейтаграмм (datagram) без установления логического соединения, где пакеты могут передаваться без сохранения порядка. Это широкоиспользуемая передача данных.

- 2.3) `SOCK_RAW` — прямой (сырой, низкоуровневый) сокет.

- 3) `protocol` — выбирается для определённого типа сокета. Обычно параметр устанавливается равным 0, тогда протокол задётся по умолчанию. Если возможно несколько протоколов, то это нужно указывать специальным образом. Протоколы обозначаются символическими константами с префиксами `IPPROTO_*`, например `IPPROTO_TCP`.
— По умолчанию для сокетов `AF_INET` типа `SOCK_STREAM` будет установлен протокол TCP, для этих же сокетов типа `SOCK_DGRAM` — протокол UDP.

Таким образом, сокеты `AF_UNIX` используются для взаимодействия параллельных процессов на отдельно стоящей машине, а сокеты `AF_INET` — для взаимодействия процессов в распределённой системе (в сети).

8.3 Системный вызов sys_socket() и struct socket

Сокеты описываются как открытые файлы: они не хранятся во вторичной памяти, это файлы специального типа.

Открытый файл описывает структуру `struct file`, а в самой структуре есть указатель на структуру `struct inode`, которая описывает физический файл в любом варианте, то есть хранящийся во вторичной или оперативной памяти.

Для работы с сокетами ядро Linux предоставляет системный вызов `sys_socketcall`. Этот вызов является общей точкой входа для различных операций с сокетами.

В качестве одного из аргументов передаётся номер операции: создание сокета, привязка адреса, установление соединения, ожидание соединений, приём или

124 / 257

Системный вызов listen(). На стороне сервера переводит сервер в режим пассивного прослушивания. Сервер вызывает `listen()` для того, чтобы проинформировать операционную систему, что на соquete нужно принять соединение. Это имеет смысл только для ориентированных на соединение протоколов (TCP). Параметр `backlog` определяет максимальную длину, до которой может вырасти очередь ожидающих соединений для `sockfd`.

```
1 #include <sys/socket.h>
2 int listen(int sockfd, int backlog);
```

Листинг 43. Объявление функции listen()

Системный вызов connect(). Вызывается на стороне клиента, чтобы создать активное соединение. Клиент должен знать адрес сервера (`serv_addr` — адрес сервера, нужно обратить на это внимание). Для протокола TCP активное соединение устанавливается по указанному адресу. Для протокола UDP `connect()` может использоваться для указания адреса назначения для всех передаваемых последствии пакетов.

```
1 #include <sys/socket.h>
2 int connect(int sockfd, struct sockaddr *serv_addr, int addrlen);
```

Листинг 44. Объявление функции connect()

Системный вызов accept(). Выполняется на стороне сервера для принятия соединения при условии, что ранее поступил запрос на соединение. В противном случае `accept()` будет блокирован до поступления запроса на соединение.

```
1 #include <sys/socket.h>
2 int accept(int sockfd, void *addr, int *addrlen);
```

Листинг 45. Объявление функции accept()

Важно

Когда соединение принимается, то есть фактически выполняется системный вызов `accept()`, создаётся копия исходного сокета, при этом исходный сокет останется в состоянии `LISTEN`, а созданная копия будет находиться в состоянии `CONNECTED`.

Системный вызов `accept()` возвращает файловый дескриптор созданной копии сокета, то есть нового сокета. Такое дублирование сокетов в ходе принятия соединений даёт серверу возможность продолжать принимать новые соединения без необходимости предварительно закрывать предыдущие.

129 / 257

передача данных. Внутри ядра по этому номеру выбирается соответствующая функция, например `socket`, `bind`, `connect`, `listen`, `accept`, `send`, `recv` и другие.

Перед выполнением операции ядро копирует аргументы из пользовательского пространства в пространство ядра. Напрямую обращаться к памяти процесса нельзя, поэтому используются специальные функции вроде `copy_from_user` и `copy_to_user`.

`sys_socketcall()` приводит к вызову `sys_socket()`:

```
1 asmlinkage long sys_socket(int family, int type, int protocol)
2 {
3     int retval;
4     struct socket *sock;
5     ...
6     retval = socket_create(family, type, protocol, &sock);
7     ...
8     return retval;
9 }
```

Листинг 38. Системный вызов sys_socket()

В ядре сокет описывает структуру `struct socket`:

```
1 struct socket
2 {
3     socket_state state;
4     short type;
5     unsigned long flags;
6     const struct proto_ops *ops; // protocol operations
7     struct file *file;
8     struct sock *sk;
9     wait_queue_head_t wait;
10 };
```

Листинг 39. Структура socket

У сокета может быть 5 состояний (state):

- 1) `SS_FREE` — свободный сокет.
- 2) `SS_UNCONNECTED` — неосоединённый сокет.
- 3) `SS_CONNECTING` — сокет находится в процессе соединения.
- 4) `SS_CONNECTED` — соединённый сокет.
- 5) `SS_DISCONNECTING` — сокет в процессе разъединения.

125 / 257

Сервер блокируется на системном вызове `accept()` до тех пор, пока не поступит запрос на соединение от клиента.

Для протокола TCP определено тройное рукопожатие (`clnt.SYN`, `srv.SYN-ACK`, `clnt.ACK`): сначала клиент обращается к серверу с запросом на установление соединения; получив первое ответное сообщение, клиент так же отправляет подтверждение получения, и только после этого соединение устанавливается. В настоящее время тройное рукопожатие стало огромной проблемой — DDoS-атаки.

8.6 Сетевые сокеты — сетевой стек, аппаратный и сетевой порядок байтов



Рис. 25. BSD сокеты

Когда создавался интернет, был изменён порядок байтов. Порядок следования байтов на разных архитектурах различен, различают аппаратный и сетевой порядок байтов:

- `big_endian` — network byte order (сетевой порядок); старший значащий байт целого числа имеет наименьший адрес. В таком формате передаются данные по сети.
- `little_endian` — host byte order (порядок хоста); соответственно наоборот. В таком формате данные хранятся, например, на процессорах Intel.

Функции для работы с порядком байтов:

- 1) `htons()` — host to network short:

```
1 #include <arpa/inet.h>
```

130 / 257

8.4 Адресация сокетов и ее особенности для разных типов сокетов

Сокеты BSD поддерживают множество протоколов. Так как сокеты адресуются и обращение к ним тоже выполняется по адресу, на сокетах была определена общая структура `struct sockaddr`:

```
1 struct sockaddr
2 {
3     sa_family_t sa_family; // AF_INET
4     char sa_data[14];
5 };
```

Листинг 40. Структура sockaddr

Ничего не конкретизировано, просто 14 байт. Структура определена в таком виде, потому что разработчики понимали, что в разных системах нужна разная адресация. Поле `sa_family` определяет семейство адресов, но точный формат адреса не определён.

Особенности для сокетов AF_UNIX. Для сокетов `AF_UNIX` в поле `sa_data` помещается имя файла, поскольку эти сокеты взаимодействуют через пространство файловых имён. Адрес сокета `AF_UNIX` — имя файла. Сокет сервера можно показать в файловой подсистеме: командой `ls -la` выводится файл сокета с его `inode`, при этом тип файла обозначается буквой `s`.

Особенности для сокетов AF_INET. Для сетей адрес в виде структуры `struct sockaddr` не подходит, мы должны указать сетевой адрес и номер порта, поэтому объявлена структура `struct sockaddr_in` для IP версии 4. Для IP версии 6 также объявлена структура `struct sockaddr_in6`.

```
1 struct sockaddr_in
2 {
3     uint16_t sin_family; // AF_INET
4     uint16_t sin_port; // Номер порта
5     struct in_addr sin_addr; // Сетевой адрес
6     unsigned char sin_zero[sizeof(struct sockaddr) - sizeof(sa_family_t) - sizeof(uint16_t) - sizeof(struct in_addr)];
7 };
```

Листинг 41. Структура sockaddr_in

126 / 257

```
2 uint16_t htons(uint16_t hostint16);
3 uint32_t htonl(uint32_t hostint32);
```

- 2) `ntohs()` — network to host:

```
1 #include <arpa/inet.h>
2 uint16_t ntohs(uint16_t netint16);
3 uint32_t ntohl(uint32_t netint32);
```

Смысл этих функций в том, что существует разная последовательность байтов, фактически меняется формат передаваемых значений. В результате никакая работа будет невозможна. Эти функции нужно вызывать в обязательном порядке, даже в том случае, если мы уверены, что порядок байтов в сети и на хосте совпадает: не все аппаратные платформы имеют прямой порядок байтов. Если порядок байтов совпадает, то функция не выполняется.

8.7 Примеры реализации взаимодействия процессов по модели клиент-сервер с использованием сокетов и мультиплексирования (tab. раб.)

8.7.1 Сервер производственно-потребление из лабораторной работы
Реализуется классическая задача производственно-потребление. У нас 2 типа клиентов: клиент-производитель и клиент-потребитель. Клиент посылает серверу приказ, что он P (производитель) или C (консумент, потребитель), то есть что он хочет сделать — положить данные в кольцевой буфер или считать данные из буфера.

Сервер в мультиплексированном однопоточном: создаётся массив `events`, дальше пробегает по нему, обсуживаясь соединения (по схеме из предыдущего подраздела). Когда мы переходим к обработке по `else`, для нас важно, производитель это или потребитель — какую функцию вызывать (практически как у потоков в многопоточной версии).

Синхронизация:

- Не нужно обеспечивать взаимное исключение (сервер однопоточный, блокировок нет), а синхронизация нужна: считанные семафоры выключить не получится, иначе буфер будет переполнен. Как определять, что буфер заполнен? — Всё равно вводить счётчик.
- В обеих версиях (многопоточной и с мультиплексированием) должны быть семафоры, но в мультиплексоре не должно быть бинарного.

131 / 257

— Считаемые семафоры, чтобы исключить переход в режим ядра, можно было бы заменить счётчиками: переход в режим ядра — затратное действие. Но счётчики не позволяют нам выполнить блокировку.

— Если буфер заполнен ($N = \text{max}$), производитель ничего положить в буфер не может — значит, мы можем послать ему, что он должен ждать. Если мы используем семафор, то сервер будет заблокирован на семафоре, значит, функция не отправит клиенту сообщение, что всё в порядке. Что должны получить производитель от сервера? — Он должен получить ОК, что его данные положены. Пока ответ не получен, клиент заблокирован на запросе `recv` `from`, и только после получения он сможет выполнить следующую итерацию.

*** Claude**

В однопоточном сервере с мультиплексированием блокировка на считанном семафоре означала бы весь сервер, исключая обслуживание потребителей, которые могли бы освободить место в буфере — получились бы `deadlock`. Поэтому проверку семафора в такой версии нужно делать неблокирующей (например, `sem_trywait`) и при заполненном буфере отвечать клиенту, что нужно повторить попытку, о чём и говорится в комментарии («послать, что он должен ждать»).

```
1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <netinet/in.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <sys/epoll.h>
7 #include <sys/ipc.h>
8 #include <sys/sem.h>
9 #include <sys/socket.h>
10 #include <sys/stat.h>
11 #include <time.h>
12 #include <unistd.h>
13
14 #define SERVER_PORT 9878
15 #define SERVER_BACKLOG 5
16 #define BUF_SIZE 52
17 #define MAX_EVENTS 8
18
19 #define OP_PRODUCER 'P'
20 #define OP_CONSUMER 'C'
21
22 #define SEM_KEY 200
```

132 / 257

```
23 #define SEM_STORAGE_EMPTY 0
24 #define SEM_STORAGE_FULL 1
25 #define SEM_LOCK -1
26 #define SEM_UNLOCK 1
27
28 static char ring_buffer[BUF_SIZE];
29 static int write_idx = 0;
30 static int read_idx = 0;
31
32 static int semaphore_id;
33 static int listen_fd;
34 static int epoll_fd;
35 static int should_exit = 0;
36
37 struct sembuf start_write = {SEM_STORAGE_EMPTY, SEM_LOCK, IPC_NOWAIT | SEM_UNDO};
38 struct sembuf start_read = {SEM_STORAGE_FULL, SEM_UNLOCK, SEM_UNDO};
39 struct sembuf stop_read = {SEM_STORAGE_FULL, SEM_LOCK, IPC_NOWAIT | SEM_UNDO};
40 struct sembuf start_write = {SEM_STORAGE_EMPTY, SEM_UNLOCK, SEM_UNDO};
41
42 void producer(int fd) {
43     char letter;
44     if (recv(fd, &letter, 1, 0) <= 0) {
45         return;
46     }
47     if (semop(semaphore_id, &start_write, 1) == -1) {
48         send(fd, "WAIT", 5, 0);
49         printf("[P %d] FULL, rejected '%c'\n", fd, letter);
50         return;
51     }
52     ring_buffer[write_idx] = letter;
53     write_idx = (write_idx + 1) % BUF_SIZE;
54     semop(semaphore_id, &start_read, 1);
55     send(fd, "OK", 3, 0);
56     printf("[P %d] put '%c'\n", fd, letter);
57 }
58 void consumer(int fd) {
59     if (semop(semaphore_id, &stop_read, 1) == -1) {
60         send(fd, "WAIT", 5, 0);
61         printf("[C %d] EMPTY\n", fd);
62         return;
63     }
64     char resp[2];
65     resp[0] = ring_buffer[read_idx];
66     resp[1] = '\0';
```

133 / 257

```
67     read_idx = (read_idx + 1) % BUF_SIZE;
68     semop(semaphore_id, &start_write, 1);
69     send(fd, resp, 2, 0);
70     printf("[C %d] got '%c'\n", fd, resp[0]);
71 }
72
73 void client_service(int fd, struct timespec* time_start) {
74     char op;
75     if (recv(fd, &op, 1, 0) <= 0) {
76         close(fd);
77         return;
78     }
79     if (op == OP_PRODUCER) {
80         producer(fd);
81     } else if (op == OP_CONSUMER) {
82         consumer(fd);
83     }
84     close(fd);
85 }
86 int main(void) {
87     struct sockaddr_in server_addr = {0}, client_addr;
88     socklen_t client_len = sizeof(client_addr);
89     if (!listen_fd = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
90         perror("socket");
91         exit(1);
92     }
93     server_addr.sin_family = AF_INET;
94     server_addr.sin_addr.s_addr = INADDR_ANY;
95     server_addr.sin_port = htons(SERVER_PORT);
96     if (!bind(listen_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) == -1) {
97         perror("bind");
98         exit(1);
99     }
100     if (!listen(listen_fd, SERVER_BACKLOG) == -1) {
101         perror("listen");
102         exit(1);
103     }
104     int permissions = S_IRWXU | S_IRWXG | S_IRWXO;
105     semaphore_id = semget(SEM_KEY, 2, IPC_CREAT | permissions);
106     if (semaphore_id == -1) {
107         perror("semget");
108         exit(1);
109     }
110     if (semctl(semaphore_id, SEM_STORAGE_EMPTY, SETVAL, BUF_SIZE) == -1) {
```

134 / 257

```
111     perror("semctl");
112     exit(1);
113 }
114 if (semctl(semaphore_id, SEM_STORAGE_FULL, SETVAL, 0) == -1) {
115     perror("semctl");
116     exit(1);
117 }
118 epoll_fd = epoll_create(0);
119 if (epoll_fd == -1) {
120     perror("epoll_create");
121     exit(1);
122 }
123 struct epoll_event ev;
124 ev.events = EPOLLIN;
125 ev.data.fd = listen_fd;
126 if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, listen_fd, &ev) == -1) {
127     perror("epoll_ctl");
128     exit(1);
129 }
130 struct epoll_event events[MAX_EVENTS];
131 while (!should_exit) {
132     int nfd = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
133     if (nfd == -1) {
134         perror("epoll_wait");
135         break;
136     }
137     for (int i = 0; i < nfd; i++) {
138         if (events[i].data.fd == listen_fd) {
139             int conn_fd = accept(listen_fd, (struct sockaddr*)&client_addr,
140                                 &client_len);
141             if (conn_fd == -1) {
142                 perror("accept");
143                 continue;
144             }
145             ev.events = EPOLLIN;
146             ev.data.fd = conn_fd;
147             if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, conn_fd, &ev) == -1) {
148                 perror("epoll_ctl");
149                 close(conn_fd);
150                 continue;
151             }
152             client_service(events[i].data.fd, &time_start);
153         }
154     }
```

135 / 257

```
155     }
156     close(epoll_fd);
157     if (semctl(semaphore_id, 0, IPC_RMID) == -1) {
158         perror("semctl");
159         exit(1);
160     }
161     return EXIT_SUCCESS;
162 }
```

Клиент в цикле подключается к серверу, отправляет признак P/C (производитель дополнительно шлёт символ данных) и обрабатывает ответ сервера: ОК — запрос выполнен, WAIT — буфер полон/пуст, попытку нужно повторить.

```
1 #include <arpa/inet.h>
2 #include <netinet/in.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <sys/socket.h>
7 #include <unistd.h>
8
9 #define SERVER_IP "127.0.0.1"
10 #define SERVER_PORT 9878
11 #define MSG_SIZE 8
12 #define OP_PRODUCER 'P'
13 #define OP_CONSUMER 'C'
14
15 static int loop_flag = 1;
16 int connect_to_server(void) {
17     int fd = socket(AF_INET, SOCK_STREAM, 0);
18     if (fd == -1) {
19         perror("socket");
20         return -1;
21     }
22     struct sockaddr_in server_addr;
23     memset(&server_addr, 0, sizeof(server_addr));
24     server_addr.sin_family = AF_INET;
25     server_addr.sin_port = htons(SERVER_PORT);
26     inet_aton(SERVER_IP, &server_addr.sin_addr);
27     if (!connect(fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) == -1) {
28         perror("connect");
29         close(fd);
30         return -1;
31     }
```

136 / 257

```
32     return fd;
33 }
34 int producer(int fd, char* letter) {
35     char response[MSG_SIZE] = {0};
36     if (recv(fd, response, MSG_SIZE, 0) <= 0) {
37         perror("recv"); return -1;
38     }
39     if (strcmp(response, "WAIT") == 0) {
40         printf("P: buffer full\n"); return 1;
41     }
42     printf("P: '%c' -> %s\n", *letter, response);
43     *letter = (*letter == 'z') ? 'a' : *letter + 1;
44     return 0;
45 }
46 int consumer(int fd) {
47     char response[MSG_SIZE] = {0};
48     if (recv(fd, response, MSG_SIZE, 0) <= 0) {
49         perror("recv"); return -1;
50     }
51     if (strcmp(response, "WAIT") == 0) {
52         printf("C: buffer empty\n"); return 1;
53     }
54     printf("C: got '%c'\n", response[0]);
55     return 0;
56 }
57 int main(void) {
58     char letter = 'a';
59     for (int i = 0; i < ITERATIONS && loop_flag; i++) {
60         char op = ((i % 3) == 2) ? OP_PRODUCER : OP_CONSUMER;
61         int fd = connect_to_server();
62         if (fd == -1) return EXIT_FAILURE;
63         int rc;
64         if (op == OP_PRODUCER) {
65             char msg[2] = {op, letter};
66             if (send(fd, msg, 2, 0) == -1) {
67                 perror("send");
68                 close(fd);
69                 return EXIT_FAILURE;
70             }
71             rc = producer(fd, &letter);
72         } else {
73             if (send(fd, &op, 1, 0) == -1) {
74                 perror("send");
75                 close(fd);
```

137 / 257

```
76         return EXIT_FAILURE;
77     }
78     rc = consumer(fd);
79 }
80 close(fd);
81 if (rc == -1) return EXIT_FAILURE;
82 }
83 return EXIT_SUCCESS;
84 }
```

138 / 257

```
76         return EXIT_FAILURE;
77     }
78     rc = consumer(fd);
79 }
80 close(fd);
81 if (rc == -1) return EXIT_FAILURE;
82 }
83 return EXIT_SUCCESS;
84 }
```

139 / 257

```
76         return EXIT_FAILURE;
77     }
78     rc = consumer(fd);
79 }
80 close(fd);
81 if (rc == -1) return EXIT_FAILURE;
82 }
83 return EXIT_SUCCESS;
84 }
```

9

9 Файловая система: процесс и файловые структуры связанные с процессом. Файлы и открытые файлы, связь структур, представляющих открытые файлы на разных уровнях. Системный вызов `open` и библиотечная функция `fcntl`: параметры и флаги, определяемые на функции `open`. Различия системного вызова `open` и ядра Linux. Пример: файл открывается два раза системным вызовом `open` для записи и в него последовательно записывается строка «аааааааааа» по первому дескриптору и затем строка «биии» по второму дескриптору, затем файл закрывается два раза. Показать, что будет записано в файл и почему результаты.

9.1 Файловая система: процесс и файловые структуры, связанные с процессом

Файл — именованная совокупность данных, может быть бессмысленная, хранящаяся во вторичной памяти. Понятие файла связано с долговременным хранением информации.

Файловая система — это порядок, определяющий способ организации хранения, именования и доступа к данным на вторичных носителях информации.

Файловая система используется для управления хранением данных и доступа к ним. ФС должна обеспечить доступ к хранимой в системе на вторичных дисках информации.

Процесс — программа в стадии выполнения. Каждый процесс в системе описывается структурой `task_struct`. С каждым процессом в системе связаны: корневая файловая система, текущий рабочий каталог, список открытых им файлов. Для этого в `task_struct` есть два поля:

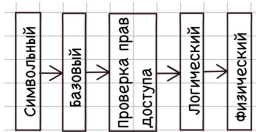
```
1 struct task_struct
2 {
3     ...
4     struct fs_struct *fs;
5     struct files_struct *files;
6     ...
7 }
```

Листинг 46. Структура `task_struct`

— `struct fs_struct` — это файловая система, которой принадлежит файл. Процесс — программа в стадии выполнения: до того, как она была запущена на выполнение, она была файлом, а файл принадлежит какой-то ФС. Процесс содержит информацию об этой файловой системе.

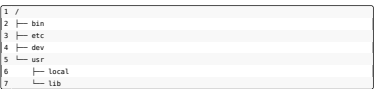
— `struct files_struct` — структура, инициализируемая для каждого процесса. В ней есть поле `struct file *fd_array[]` — таблица открытых файлов процесса. Каждый процесс имеет собственную таблицу.

141 / 257



- **Символьный уровень** — именование файлов: у файлов **имена** (массив символов), собранные по каталогам; корневой каталог в UNIX/Linux — `/`. Имя не является идентификатором файла, а лишь позволяет удобно с ним работать; идентификатором файла является `inode`; определены жёсткие (`hard links`) и символические (`soft links`) ссылки.
- **Базовый уровень** — идентификация файла: это `inode`; идентификатором является номер `inode`, который описывает физический файл.
- **Проверка прав доступа** — права отдельно для пользователя, группы и остальных (команда `chmod`).
- **Логический уровень** — процесс открывает файл и выполняет над ним операции: файл считает, что начинается с нулевого смещения (`offset`), каждая операция чтения/записи смещает указатель; файл байт-ориентированный, размера не имеет. Здесь определяется адресация данных, преобразуемая в физические адреса на диске.
- **Физический уровень** — уровень физической памяти; логический и физический уровни связывает драйвер дискового устройства, переводящий смещения в физические адреса.

Для UNIX характерно существование корневого каталога /, от которого строится дерево каталогов:



152 / 257

```
14 struct list_head s_inodes; // all inodes
15 ;
```

Листинг 55. Структура `super_block` (сокр.).

- `list_head` — двусвязные списки в Linux;
- `s_type` — указатель на тип ФС (`file_system_type`);
- `s_op` — указатель на операции суперблока (`super_operations`);
- `s_magic` — магическое число ФС;
- `s_root` — **точка монтирования**, корневой каталог смонтированной ФС;
- `s_umount` — семифор читателя-писателя для монтирования;
- `s_mounts` — список монтирования данной ФС (одна и та же ФС может быть смонтирована много раз);
- `s_bdev` — связанное блочное устройство;
- `s_inodes` — список всех `inode` данной ФС.

⚠ **Осторожно**

Вместо первого рисунка суперблока, который Рязанова не показывала (показывала), вероятно, хочет видеть второй.

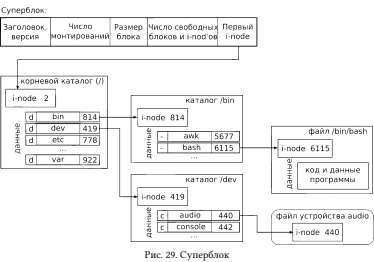


Рис. 29. Суперблок

157 / 257

Для Unix был предложен интерфейс VFS/vnode (virtual file system, virtual node). Linux отказался от него, оставив только VFS. VFS — интерфейс, через который ОС получает доступ к большому числу разных ФС, сократив до минимума количество системных вызовов: любая работа с файлом любой ФС идёт через один и тот же интерфейс. Основа такой работы — монтирование: прежде чем ФС станет доступна (станут видны её каталоги и файлы), она должна быть смонтирована.

В основе интерфейса VFS лежат 4 структуры (это ядерные структуры):

- 1) `struct super_block`;
- 2) `struct inode`;
- 3) `struct dentry`;
- 4) `struct file`.

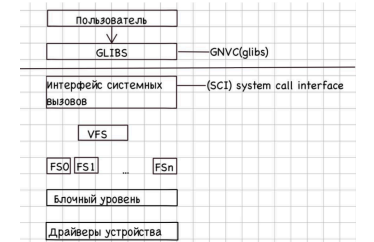


Рис. 27. Virtual File System

10.4 Inode и dentry: пример доступа к файлу `mbox`

- 1) **дисковый inode** — описывает физический файл, содержит адресную информацию, то есть адреса блоков, в которых хранится информация, записанная в данный файл (на диске адреса и на диске информация);

153 / 257

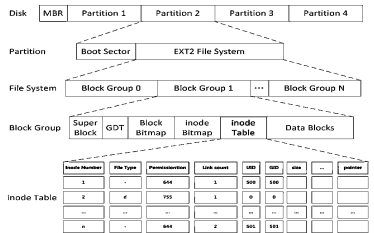


Рис. 30. Суперблок

Таким образом, суперблок связан указателями с остальными структурами интерфейса VFS: с типом ФС (`s_type`), набором операций суперблока (`s_op`), операциями над `dentry` (`s_d_op`), со списком всех `inode` (`s_inodes`) и с корневым каталогом ФС (`s_root`). Любой физический файл доступен, только если ФС смонтирована, поэтому в `inode` есть указатель на суперблок.

10.7 Структура `struct super_operations`

Для каждой ФС определяется свой набор операций над объектами суперблока. Поскольку разработчики Linux ставили задачу обеспечить интерфейс для большого числа ФС, перечень операций широк:

```
1 struct super_operations {
2 struct inode *(*alloc_inode)(struct super_block *sb);
3 void (*destroy_inode)(struct inode *);
4 void (*free_inode)(struct inode *);
5 void (*dirty_inode)(struct inode *, int flags);
6 int (*write_inode)(struct inode *, struct writeback_control *wbc);
7 int (*drop_inode)(struct inode *);
8 ...
9 void (*put_super)(struct super_block *);
10 ...
11 int (*freeze_fs)(struct super_block *);
```

158 / 257

- 2) **inode в ядре** содержит информацию о файле для ускорения доступа (копия в памяти, дополнительные поля: логический номер устройства, номер индекса, ссылки для хеш-очереди).

Каталоги в UNIX/Linux обрабатываются как файлы. Участок пути от / до следующего / — элемент пути, для него в VFS введена концепция `dentry` (directory entry). Структура `struct dentry` не соответствует информации из вторичной памяти: объекты `dentry` создаются на лету при операциях с каталогами. Объектами `dentry` являются все элементы пути, включая обычные файлы (в пути `/bin/vi` — `/`, `bin`, `vi`).

Структура `inode` директории состоит из имени, сопоставленных с номерами `inode`:

inode number	3470036
..	3470017
folder1	3470031
file1	3470043
folder2	3470023
file2	3470024
file3	3470065

Таблица 6. Структура `inode` каталога

Используется слово `number`, а не `index`, потому что `index` — это всегда смещение.

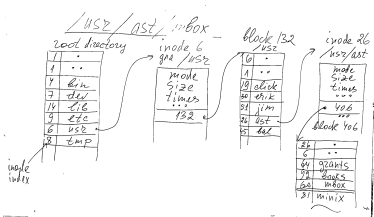


Рис. 28. Шаги доступа к файлу `/usr/ast/mbox`

154 / 257

```
12 ...
13 int (*remount_fs)(struct super_block *, int *, char *);
14 void (*umount_begin)(struct super_block *);
15 int (*show_options)(struct seq_file *, struct dentry *);
16 ...
17 int (*show_path)(struct seq_file *, struct dentry *);
18 ...
19 ;
```

Листинг 56. `struct super_operations`

Основное количество перечисляемых операций — это операции с `inode`:

- `alloc_inode` — создаёт и инициализирует новый объект `inode` для конкретной смонтированной ФС (выделяет адресное пространство);
- `destroy_inode` — удаляет `inode` из связанного списка `inode`;
- `free_inode` — информация с метаданными (с номером `inode`) не удаляется, и данный `inode` может быть повторно использован;
- `dirty_inode` — вызывается, чтобы переместить `inode` в другой список, если в него были введены изменения;
- `write_inode` — записывает `inode` на диск (только `alloc` недостаточно);
- `drop_inode` — вызывается интерфейсом VFS, когда количество ссылок на `inode` равно 0;
- `put_super` — вызывается VFS при размонтировании ФС; данный суперблок изымается из связанного списка монтирования;
- `remount_fs` — вызывается VFS для монтирования суперблока с другими параметрами;
- `umount_begin` — вызывается VFS для прерывания монтирования; используется сетевыми ФС, например NFS (Network File System).

10.8 Структуры `struct inode`, `struct inode_operations`

Номер `inode` называется метаданными.

```
1 struct inode {
2 umode_t i_mode;
3 ...
4 unsigned int i_flags;
5 ...
6 const struct inode_operations *i_op;
7 struct super_block *i_sb;
8 struct address_space *i_mapping;
9 ...
10 void *i_security;
11 unsigned long i_ino;
```

159 / 257

Поиск файла всегда начинается с корневого каталога (в случае текущей рабочей директории часть пути уже пройдена). Доступ к файлу `/usr/ast/mbox` выполняется по шагам:

- 1) **корневой индексный дескриптор** уже находится в кэше (он закеширован во время монтирования). В таблице корневого каталога / имени `usr` сопоставлен номер `inode` 6;
- 2) обращаемся к `inode` 6 (по `/usr`), из его полей (`inode`, `size`, `times`, ...) и адреса блока получаем номер блока с содержимым каталога — блок 132;
- 3) в блоке 132 (содержимое `/usr`) имени `ast` сопоставлен номер `inode` 26;
- 4) обращаемся к `inode` 26 (это `/usr/ast`): из него получаем адрес блока с содержимым каталога — блок 406;
- 5) в блоке 406 (содержимое `/usr/ast`) имени `mbox` сопоставлен номер `inode` 60 — это искомый файл.

Содержимое индексного дескриптора представляет собой список, на основе которого можно перейти к соответствующему блоку, по информации блока — к соответствующему индексному дескриптору, и так далее. Сама структура `dentry` номера индексного дескриптора не содержит — он загружается отдельно.

10.5 Кэш `inode` и `dentry`

Проход по элементам пути требует много времени (имена сравниваются как строки), поэтому элементы пути кешируются. Действительный объект `dentry` может находиться в одном из трёх состояний:

- 1) **used** — используемый объект `dentry`; у него есть связанный `inode` (поле `inode`), а счётчик `d_count` больше 0. Такой объект не может быть просто удалён;
- 2) **unused** — неиспользуемый объект; ему соответствует доступный `inode`, но `d_count` == 0. Такие объекты образуют двусвязный список LRU (least recently used), отсортированный по времени последнего обращения;
- 3) **negative** — объект, связанный с несуществующим номером `inode`. Такие элементы пути сохраняются в расчёте на повторное обращение и не удаляются сразу, что ускоряет обработку.

Соответственно, кэш `dentry` (dcache) состоит из трёх частей:

- 1) список используемых объектов `dentry` (used);
- 2) список неиспользуемых объектов `dentry` (unused, LRU);
- 3) хеш-таблица и хеш-функция, позволяющие быстро преобразовать заданный путь в объект `dentry`.

Кеширование каталогов и файловых индексов сокращает время обращения: после первого обращения к файлу высока вероятность, что находящийся в кэше элемент

155 / 257

```
12 unsigned int i_nlink; // количество жёстких ссылок
13 loff_t i_size;
14 ...
15 u8 i_bkbits;
16 ...
17 struct rw_semaphore i_rwsem;
18 ...
19 struct hlist_head i_hash;
20 ...
21 struct list_head i_lru; /* inode LRU list */
22 struct list_head i_sb_list;
23 struct list_head i_wb_list; /* backing dev writeback list */
24 union {
25 struct hlist_head i_dentry;
26 ...
27 };
28 ...
29 atomic_t i_readcount; /* struct files open RD */
30 union {
31 const struct file_operations *i_fop; /* former ->i_op-default_file_ops */
32 ...
33 };
34 ...
35 union {
36 struct pipe_inode_info *i_pipe;
37 struct cdev *i_cdev;
38 char *i_link;
39 ...
40 ...
41 ;
```

Листинг 57. `struct inode`

- `i_mode`, `i_flags` — права доступа и флаги; флаги играют роль ограничений, указывая, что можно делать, а что нельзя.
- `i_op` — указатель на таблицу операций с индексом (`inode_operations`).
- `i_sb` — указатель на связанный суперблок: файловая система монтируется на конкретное физическое устройство, и любой файл относится к конкретной ФС.
- `i_ino` — номер `inode`.
- `i_nlink` — количество жёстких ссылок.
- `i_size` — размер файла; поскольку файлы связаны с блочными устройствами, в `inode` есть информация о размере блока (`i_bkbits`).
- `i_rwsem` — read-write семифор.

160 / 257

каталога и связанные с ними `inode` будут там в течение некоторого времени. Например, если выполнить `ls file` для несуществующего файла, повторный вызов не потребует повторного обращения к диску — для этого и существует состояние `negative`.

Кэш `inode` представляет собой:

- 1) **глобальный хеш-массив** `inode_hashtable`, в котором каждый `inode` хешируется по значению указателя на суперблок и по номеру `inode`;
- 2) **глобальный список** `inode_in_use` — содержит допустимые `inode`, у которых `i_count` > 0 и `i_nlink` > 0 (функция `new_inode` создаёт `inode`, добавляемый в этот список);
- 3) **глобальный список** `inode_unused` — содержит допустимые `inode` с `i_count` == 0;
- 4) для каждого смонтированного суперблока список `sb->i_dirty` — содержит модифицированные («грязные») `inode` с `i_count` > 0 и `i_state` == I_DIRTY. Поддержка такого списка уменьшает накладные расходы на синхронизацию: изменённые `inode` должны быть записаны и на диск;
- 5) `slab`-кэш для объектов `inode`.

10.6 Назначение суперблока в интерфейсе VFS: монтирование файловой системы — суперблок и связанные с ним структуры; структура `struct super_block` и корневого каталога

Суперблок представляет в VFS информацию о смонтированной ФС — структура с данными, необходимыми для монтирования и доступа к файлам ФС: метаданными для обращения к файлам, хранящимся «вразброс» (адресация соответствующих участков диска). В каждой ФС ровно один суперблок, он располагается в начале раздела (если резервная копия). При монтировании суперблок считывается в память ядра и находится там до размонтирования.

```
1 struct super_block {
2 struct list_head s_list;
3 dev_t s_dev;
4 unsigned long s_blocksize;
5 loff_t s_maxbytes; // Max file size
6 struct file_system_type *s_type;
7 const struct super_operations *s_op;
8 unsigned long s_flags;
9 unsigned long s_magic;
10 struct dentry *s_root; // точка монтирования
11 struct rw_semaphore s_umount;
12 struct list_head s_mounts;
13 struct block_device *s_bdev;
```

156 / 257

- списки (`i_lru`, `i_hash`, `i_sb_list`, ...) — обеспечивают быстрый доступ к файлам, так как обращение к внешнему устройству — медленное действие с накладными расходами.
- `i_readcount` — счётчик количества обращений к файлу для чтения (через `struct file`).

Все обсуждаемые структуры между собой связаны указателями, причём связь строится относительно процесса, потому что процессы открывают файлы (открытие файла выполняет, например, текстовый редактор, являющийся процессом).

В VFS каталоги обрабатываются как файлы, но VFS часто приходится выполнять спецификации для каталогов операции — прежде всего поиск элемента пути по имени (`lookup`), исключивший преобразование каждого элемента пути, проверку корректности и переход к следующему элементу:

```
1 struct inode_operations {
2 struct dentry * (*lookup)(struct inode *, struct dentry *, unsigned int);
3 ...
4 int (*permission)(structmnt_idmap *, struct inode *, int);
5 ...
6 int (*readlink)(struct dentry *, char __user *, int);
7 int (*create)(structmnt_idmap *, struct inode *, struct dentry *, umode_t, bool);
8 int (*link)(struct dentry *, struct inode *, struct dentry *);
9 int (*unlink)(struct inode *, struct dentry *);
10 int (*symlink)(structmnt_idmap *, struct inode *, struct dentry *, const char *);
11 struct dentry * (*mkdir)(structmnt_idmap *, struct inode *, struct dentry *, umode_t);
12 int (*rmdir)(struct dentry *);
13 int (*rmdir2)(structmnt_idmap *, struct inode *, struct dentry *, umode_t, dev_t);
14 int (*rename)(structmnt_idmap *, struct inode *, struct dentry *, struct dentry *, struct dentry *, unsigned int);
15 ...
16 int (*atomic_open)(struct inode *, struct dentry *, struct file *, unsigned open_flag, umode_t, create_mode);
17 ...
18 ;
```

Листинг 58. `struct inode_operations`

10.9 Функции `simple` и `generic`

При инициализации файловой системы можно воспользоваться предопределёнными функциями ядра. Они делятся на две группы.

Функции `generic_*` выполняют всю не зависящую от конкретной ФС работу. Например, `generic_shutdown_super` — common helper для `kill_sb`, удаляющий

161 / 257

суперблок (проходит по объектам struct super_block). На его основе построены kill_anon_super и kill_litter_super:

```
1 void kill_litter_super(struct super_block *sb)
2 {
3     if (sb->s_root)
4         d_genocide(sb->s_root);
5     kill_anon_super(sb);
6 }
7
8 void kill_anon_super(struct super_block *sb)
9 {
10     dev_t dev = sb->s_dev;
11     generic_shutdown_super(sb);
12     kill_super_notify(sb);
13     free_anon_dev(dev);
14 }
```

Листинг 59. Функции kill_litter_super и kill_anon_super

Функции simple_xxx дают готовую минимальную реализацию для простой (в частности, виртуальной, не связанной с диском) ФС:

- simple_fill_super — инициализирует поля суперблока (размер блока, магическое число, операции суперблока), создаёт корневой inode функцией new_inode, делает из него корневой каталог через d_make_root и заполняет s_root;
- simple_dir_operations, simple_dir_inode_operations — готовые наборы операций для работы с каталогами, чтобы не реализовывать их вручную.

10.10 Точка монтирования

В файловой системе UNIX/Linux определена важная операция монтирования. Монтирование — это последовательность действий, в результате которой файловая система (и все её файлы) становится доступной. Только смонтированные ФС предоставляют доступ к своим каталогам и файлам.

Точкой монтирования является каталог, к которому подмонтируется ФС (это может быть любой каталог). Когда ФС монтируется в существующее дерево каталогов, все файлы и директории монтируемой ФС становятся файлами и поддиректориями точки монтирования. Если в точке монтирования было какое-то содержимое, оно становится незвидимым до тех пор, пока ФС не будет размонтирована, потому что в качестве точки монтирования лучше использовать пустые каталоги.

Чтобы смонтировать ФС, в ней нужно создать корневой каталог (root) — именно он монтируется в struct super_block это поле s_root. Объекты dentry могут включать точки монтирования (в пути /mnt/cdrom/тоо точка монтирования — cdrom).

162 / 257

Команда mount имеет следующий синтаксис:

```
1 mount -t type device dir
```

где dir — точка монтирования. Флаг -t обязателен. Команда mount [-l] [-t type] выводит смонтированные ФС типа type. Монтировать ФС обычно можно только одна структура file_system_type, независимо от того, сколько экземпляров этой ФС смонтировано.

10.11 Функции монтирования

Файловую систему описывает структура struct file_system_type. Это тип ФС, которая будет монтироваться; данная структура объявлена в ядре, и инициализируя её поля, мы определяем собственный тип ФС. Для каждого типа ФС существует только одна структура file_system_type, независимо от того, сколько экземпляров этой ФС смонтировано.

```
1 struct file_system_type {
2     const char *name; /* Название файловой системы */
3     int fs_flags; /* Флаги типа файловой системы */
4     #define FS_REQUIRES_DEV 1
5     #define FS_USERMOUNT 8
6     struct dentry * (*mount) (struct file_system_type *, int, const char *, void *);
7     void (*kill_sb) (struct super_block *); /* прекращение доступа к суперблоку */
8     struct module *owner; /* Могуль, владеющей файловой системой */
9     struct file_system_type *next; /* Связующая файловая система */
10     struct list_head fs_supers; /* Список объектов типа суперблок */
11     ...
12 };
```

Листинг 60. Структура file_system_type

Минимальный набор полей, которые нужно проинициализировать для **регистрации** ФС, — это owner и name. Зарегистрировать ФС можно системным вызовом register_filesystem, для deregистрации используется unregister_filesystem:

```
1 extern int register_filesystem(struct file_system_type *);
2 extern int unregister_filesystem(struct file_system_type *);
```

Для **монтирования** этого недостаточно: нужно написать собственную функцию mount и зарегистрировать её в поле mount структуры file_system_type. В этой функции выполняется одна из четырёх функций монтирования ядра (все имеют модификатор extern и возвращают struct dentry):

- mount_bdev — для монтирования ФС, находящейся на блочном устройстве (требуется флаг FS_REQUIRES_DEV);

163 / 257

- mount_single — для монтирования ФС, точки монтирования которой разделяют один-единственный экземпляр ФС;
- mount_nodem — для монтирования ФС, не связанной ни с каким устройством;
- mount_subtree.

```
1 extern struct dentry *mount_nodev(struct file_system_type *fs_type, int flags,
2 void *data, int (*fill_super) (struct super_block *, void *, int));
```

При монтировании создается структура struct vfsmount, представляющая конкретный экземпляр ФС:

```
1 struct vfsmount {
2     struct dentry *mnt_root; /* root of the mounted tree */
3     struct super_block *mnt_sb; /* pointer to superblock */
4     int mnt_flags;
5     struct mt_idmap *mnt_idmap;
6 };
```

Листинг 61. Структура vfsmount

10.12 Пример создания файловой системы, её регистрации и монтирование (заб. раб)

В лабораторной работе создается соборенная (виртуальная) файловая система в загружаемом модуле ядра; по отношению к VFS выполняются два действия:

- 1) **Регистрация ФС:** объявляем свой struct file_system_type (в коде — myfs_type; минимально owner, name, плюс поля mount и kill_sb); в init (myfs_init) вызываем register_filesystem, в exit (myfs_exit) — unregister_filesystem.
- 2) **Монтирование ФС:** в своей функции mount (myfs_mount) вызываем mount_nodev (ФС не связана с устройством), передавая ей свое fill_super (в коде myfs_fill_sb, написанна на основе simple_fill_super). Именно myfs_fill_sb инициализирует поля struct super_block (размер блока, магическое число, свой набор super_operations), через new_inode создаёт inode корневого каталога (с типовыми simple-операциями каталога) и функцией d_make_root делает из него root, записывая его в s_root. Освобождение суперблока поручено готовой функции ядра kill_litter_super (поле kill_sb): чтобы при размонтировании в syslog попадало сообщение, в своем наборе super_operations задается собственный put_super (myfs_put_super).

После загрузки модуля ФС монтируется командой mount и становится видна в дереве каталогов (может быть смонтирована несколько раз). Результат показывать ls -ial, cat /proc/mounts (монтированные ФС) и cat /proc/filesystems (зарегистрированный тип ФС).

164 / 257

```
1 #include <linux/module.h>
2 #include <linux/kernel.h>
3 #include <linux/init.h>
4 #include <linux/fs.h>
5 #include <asm/uaccess.h>
6
7
8 MODULE_LICENSE("GPL");
9 MODULE_AUTHOR("Zhizhin Nikita");
10
11 #define MYFS_MAGIC_NUMBER 0x13331333
12 static void myfs_put_super(struct super_block *sb) {
13     printk("myfs: put_super\n");
14 }
15 static struct super_operations const myfs_super_ops = {
16     .put_super = myfs_put_super,
17     .statsfs = simple_statsfs,
18     .drop_inode = generic_delete_inode
19 };
20 static int myfs_fill_sb(struct super_block *sb, void *data, int silent) {
21     struct inode *inode;
22     struct dentry *root;
23     sb->s_blocksize = PAGE_SIZE;
24     sb->s_blocksize_bits = PAGE_SHIFT;
25     sb->s_magic = MYFS_MAGIC_NUMBER;
26     sb->s_op = &myfs_super_ops;
27     sb->s_time_gran = 1;
28     inode = new_inode(sb);
29     if (inode) {
30         printk(KERN_INFO "myfs: new_inode error\n");
31         return -ENOMEM;
32     }
33     inode->i_ino = 1;
34     inode->i_mode = S_IFDIR | 0755;
35     simple_inode_init_ts(inode);
36     inode->i_op = &simple_dir_inode_operations;
37     inode->i_fop = &simple_dir_operations;
38     root = d_make_root(inode);
41     printk("myfs: d_make_root\n");
42     if (root) {
43         printk(KERN_ERR "myfs: d_make_root error\n");
44         iput(inode);
45     }
```

165 / 257

```
45     return -ENOMEM;
46 }
47 sb->s_root = root;
48 printk(KERN_INFO "myfs: VFS root created\n");
49 printk(KERN_INFO "myfs: super_block init");
50 return 0;
51 }
52 static struct dentry *myfs_mount(struct file_system_type *type, int flags, const char *dev, void *data) {
53     struct dentry *cont entry = mount_nodev(type, flags, data, myfs_fill_sb);
54     if (IS_ERR(cont entry))
55         printk(KERN_ERR "myfs: mount error\n");
56     else
57         printk(KERN_INFO "myfs: mount\n");
58     return entry;
59 }
60 static struct file_system_type myfs_type = {
61     .owner = THIS_MODULE,
62     .name = "myfs",
63     .mount = myfs_mount,
64     .kill_sb = kill_litter_super
65 };
66 static int __init myfs_init(void) {
67     int ret;
68
69     printk(KERN_INFO "myfs: register_filesystem call\n");
70     ret = register_filesystem(&myfs_type);
71     if (ret) {
72         printk(KERN_ERR "myfs: register_filesystem error\n");
73         return ret;
74     }
75     printk(KERN_INFO "myfs: loaded\n");
76     return 0;
77 }
78 static void __exit myfs_exit(void) {
79     int ret;
80     ret = unregister_filesystem(&myfs_type);
81     if (ret)
82         printk(KERN_ERR "myfs: unregister_filesystem error\n");
83     else
84         printk("myfs: unregister_filesystem\n");
85     printk(KERN_INFO "myfs: unloaded\n");
86 }
87 module_init(myfs_init);
88 module_exit(myfs_exit);
```

166 / 257

11

11.1 Аппаратные прерывания в Linux: запрос прерывания и линии IRQ. Простейшая схема аппаратной поддержки прерываний (концептуальная трёхшинная архитектура системы). Быстрые и медленные прерывания, флаги, пример быстрого прерывания. Нижняя и верхняя половины обработчиков прерывания, функция регистрации прерывания и её параметры. Нижняя половина: softirq, определение, массив softirq, yes, типы soft_irq, функции open_softirq(), raise_softirq(). Поток ядра ksoftirqd. Пример обработки приходящих пакетов NET_RX_SOFTIRQ.

11.1.1 Аппаратные прерывания в Linux: запрос прерывания и линии IRQ

Взаимодействие компьютера с внешними устройствами выполняется с помощью аппаратных прерываний — это концияция параллелизации действий, когда управление внешними устройствами берут на себя специальные устройства — контроллеры. Как правило, контроллеры входят в состав внешних устройств, а адаптеры располагаются на материнской плате.

Так как внешними устройствами управляет специальный программруемый контроллер, возникла необходимость в информировании процессора о завершении операции ввода/вывода:

- 1) контроллер устройства генерирует сигнал прерывания — запрос прерывания (Interrupt Request);
- 2) в простейшей схеме этот сигнал поступает на вход контроллера прерываний PIC;
- 3) входы контроллера прерываний — это линии IRQ.

Примеры линий: IRQ0 — системный таймер, IRQ1 — прерывание от клавиатуры, IRQ12 — прерывание от мыши.

Сетевая карта при поступлении пакета формирует аппаратное прерывание — оно должно обрабатываться. Драйвер внешнего устройства всегда имеет в качестве одной из точек входа **обработчик прерывания**. Он вызывается, когда происходит прерывание от данного устройства.

169 / 257

11.1.2 Простейшая схема аппаратной поддержки прерываний (концептуальная трёхшинная архитектура системы)

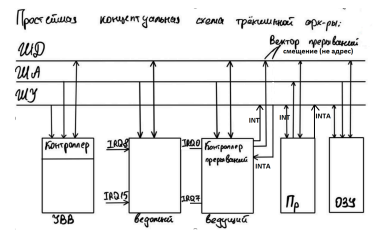


Рис. 31. Простейшая концептуальная схема

В компьютерах передаются сигналы трёх типов: данные (данные и команды), адреса, сигналы управления — отсюда три шины: шина данных, шина адреса, шина управления.

Обработка запроса прерывания по этой схеме:

- 1) На линии IRQ приходит сигнал от внешнего устройства.
- 2) Когда на контроллер прерываний приходит прерывание, он формирует сигнал INT, который по шине управления поступает на выделенную ножку процессора.
- 3) В конце цикла выполнения команды процессор проверяет наличие сигнала на выделенной ножке.
- 4) Если сигнал пришёл — процессор посылает контроллеру сигнал INTA (подтверждение).
- 5) Получив INTA, контроллер по шине данных выставляет вектор прерывания (базовый вектор + номер линии IRQ).
- 6) Процессор считывает вектор прерывания, переводит его смещение по таблице и считывает адрес обработчика.

170 / 257

11.1.3 Быстрые и медленные прерывания, пример быстрого прерывания

В ОС Linux различают быстрые и медленные прерывания:

- 1) **Быстрые:** Обработка быстрых прерываний выполняется от начала до конца. В современных системах осталось единственное быстрое прерывание — прерывание от таймера, вытеснить его невозможно. При этом выполняется на процессоре 0 и условно этот процессор считается главным.
- 2) **Медленные:** Все остальные аппаратные прерывания, то есть прерывания от устройств ввода/вывода (внешних устройств), делятся на две части: 2.1) top half (верхняя половина): 2.1.1) interrupt (hardware — аппаратное) 2.2) bottom half (нижняя половина): 2.2.1) softirq 2.2.2) tasklet 2.2.3) workqueue

Пример быстрого прерывания — прерывание от системного таймера. Главные задачи его обработки:

- 1) Инициализировать планировщик. Ставит в очередь на выполнение очень важные задачи: — Планировщик — Диспетчер — scheduler
- 2) Инициализировать диспетчеризацию.

Он не может выполняться долго. Если он выполняется долго, это приведёт к снижению **отзывчивости** системы.

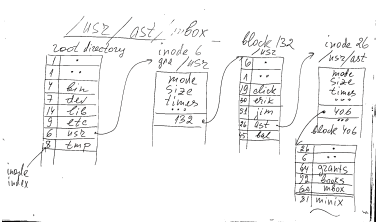
11.1.4 Нижняя и верхняя половины обработчиков прерывания, функция регистрации прерывания и её параметры

Аппаратные прерывания (top half) выполняются на высоких уровнях приоритета, причем системы, с которыми мы работаем, не поддерживают вложенных прерываний, которые были написаны для IBM/360. Для выделения аппаратных прерываний установлены следующие ограничения:

- если на процессоре выполняется обработчик аппаратного прерывания, то на этом процессоре запрещены все прерывания;
- на остальных процессорах запрещены прерывания по линии IRQ, на которой установлен выполняющийся обработчик прерывания.

171 / 257

167 / 257



- Доступ к файлу `/usr/ast/mbx` (в пути 4 объекта `dentry`: `/`, `usr`, `ast`, `mbx`):
- Путь начинается с корневого каталога (`root` `directory`), его содержимое известно системе.
 - В корневом каталоге ищется элемент пути `usr` — ему сопоставлен `inode` с номером 6.
 - Поли `struct inode` для данного каталога (`mode`, `size`, `times`, ...) позволяют идти дальше: нас интересует адресная часть — в `inode` 6 указано, что информация каталога находится в блоке 132. Обращаемся к диску: блок 132 — это содержимое директории `/usr`.
 - В блоке 132 ищется элемент `ast` — ему сопоставлен `inode` с номером 26 (это структура для `/usr/ast`). В этой структуре поля `data` и те же, они не меняются в зависимости от того, чей это `inode`.
 - Из `inode` 26 берётся адрес блока 406 — это содержимое директории `/usr/ast`.
 - В блоке 406 ищется `mbx` — ему сопоставлен `inode` с номером 66; из этого `inode` берётся адрес блока, в которых хранятся данные файла `mbx`.

Пример показывает, как много действий (обращений к диску) нужно выполнить для обращения к конкретному файлу, поэтому кеширование каталогов и файловых индексов позволяет сократить это время обращения.

12.11 Кеширование в системе: `dentry` кэш и кэш `inode`
Структуры `dentry` хранятся в кеше. Когда происходит первое обращение к элементу пути, создаётся объект `dentry` и он помещается в кеш. Когда необходимо обратиться

к файлу, система проходит по элементам пути и ищет их в `dentry`-кеше; если в нём `dentry` не найден, то обращение идёт к диску (см. пример с `vbox` выше).

12.11.1 Состояния `dentry` и `dentry`-кеш
Действительный объект `dentry` может быть в одном из трёх состояний (`used`, `unused`, `negative`); им соответствуют три части `dentry`-кеша:

- Список используемых** (`used`): у `dentry` есть `inode` (`d_inode`), счётчик `d_count` > 0; объект не удаляется, пока `d_count` не 0.
- Список неиспользуемых** (`unused`) — двунаправленный LRU-список (`dentry_lru`), сортируется по времени последнего обращения; при нехватке памяти удаляются «старые» объекты.
- Хеш-таблица** (`dentry_hashtable`) и **функция кеширования** быстро преобразуют путь в `dentry`; кеширование — `d_hash()`, поиск — `d_lookup()`.

Состояние `negative` — элемент пути с несуществующим `inode`: кеширует факт отсутствия файла и не удаётся сразу, чтобы повторное обращение (например, повторный `ls` несуществующего файла) не шло к диску.

В примере с `vbox` получение корневого индексного дескриптора упрощается: он кеширован уже при монтировании корневого каталога. Сама `dentry` не содержит номера индексного дескриптора — он загружается отдельно.

Доступ к файлам выполняется «в пространстве»: программы обращаются к разным файлам одного каталога, поэтому кеширование всех элементов пути одного файла сокращает время доступа и к соседним файлам этого каталога.

В примере с `vbox` получение корневого индексного дескриптора упрощается: он кеширован уже при монтировании корневого каталога. Сама `dentry` не содержит номера индексного дескриптора — он загружается отдельно.

12.11.2 Кэш `inode`
Кэш `inode`:

- Хеш-массив** `inode_hashtable` — `inode` кешируется по указателю на суперблок и номеру `inode`.
- Список `inode_in_use`** — допустимые `inode` с `i_count` > 0 и `i_nlink` > 0 (туда `new_inode` добавляет созданный `inode`).
- Список `inode_unused`** — допустимые `inode` с `i_count` == 0.

Для каждого суперблока — список `sb->s_dirty` — модифицированные («грязные», `I_DIRTY`) `inode` с `i_count` > 0; уменьшает накладные расходы на синхронизацию с диском.

Кеши взаимодружатся: `inode`, на которые ссылаются неиспользуемые объекты `dentry`, не удаляются, пока на них ссылается `dentry`-кеш.

12.12 Кэш SLAB, функции для работы с кэшем SLAB
Slab — «буферок». Ядро оперирует ограниченным числом объектов (структур), для каждого нужно выделить память и проинициализировать поля, что занимает время.

Если удалить объект после использования, для следующего такого же объекта придётся проделать ту же работу заново.

Идея: не удалять объекты, а сохранять их в slab для повторного использования. Под разные структуры нужны разные кэши — это и реализует Slab-кеш. Каждый кеш хранит объекты одного размера (больше или меньше страницы); есть slab и для часто используемых размеров буферов (16, 32, 64, ...). Все кэши видны через `cat /proc/slabinfo`.

Важно
Для каждого типа данных создаётся свой SLAB кеш. Не один на всех, а для каждого.

Побочный эффект slabов — сокращение фрагментации памяти.
С версии ядра 2.6.23 SLAB переименован в SLUB (есть ещё SLOB). Это только код ядра: из пользовательского пространства — память выделяется страницами или slabами уже в ядре.

Имеются структуры `kmem_cache`; каждый кеш представляется такой структурой, а кэши связаны друг с другом в список `cache_chain` (цепочка кэшей):

Каждый slab может находиться в одном из трёх состояний:

— **slab full** — полностью распределён в таком slabе нет свободных объектов, они уже распределены и используются;

— **slab partial** — частично распределён: есть как выделенные, так и свободные объекты;

— **slab empty** — пустой: нет ни одного распределённого объекта, все объекты свободны для последующего использования.

Выделение: новый объект берётся из частично распределённого slabа, иначе из пустого; если пустых нет, slab создается по мере необходимости. Побочный эффект — уменьшение фрагментации памяти.

12.12.1 Функции для работы с кэшем SLAB

1 /* Создание кэша */
2 struct kmem_cache *kmem_cache_create(const char *name, size_t size, size_t offset,
3 unsigned long flags, void (*constructor)
4 (void *));

5 /* Уничтожение кэша */
6 void kmem_cache_destroy(struct kmem_cache *cache);

7 /* Выделить объект из кэша */
8 void *kmem_cache_alloc(struct kmem_cache *cache, gfp_t flags);

9 /* Удалить объект из кэша */
10 void kmem_cache_free(struct kmem_cache *cache, void *obj);

11 /* Проверить, есть ли объект в кэше */
12 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

13 /* Проверить, есть ли объект в кэше */
14 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

15 /* Проверить, есть ли объект в кэше */
16 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

17 /* Проверить, есть ли объект в кэше */
18 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

19 /* Проверить, есть ли объект в кэше */
20 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

21 /* Проверить, есть ли объект в кэше */
22 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

23 /* Проверить, есть ли объект в кэше */
24 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

25 /* Проверить, есть ли объект в кэше */
26 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

27 /* Проверить, есть ли объект в кэше */
28 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

29 /* Проверить, есть ли объект в кэше */
30 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

31 /* Проверить, есть ли объект в кэше */
32 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

33 /* Проверить, есть ли объект в кэше */
34 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

35 /* Проверить, есть ли объект в кэше */
36 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

37 /* Проверить, есть ли объект в кэше */
38 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

39 /* Проверить, есть ли объект в кэше */
40 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

41 /* Проверить, есть ли объект в кэше */
42 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

43 /* Проверить, есть ли объект в кэше */
44 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

45 /* Проверить, есть ли объект в кэше */
46 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

47 /* Проверить, есть ли объект в кэше */
48 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

49 /* Проверить, есть ли объект в кэше */
50 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

51 /* Проверить, есть ли объект в кэше */
52 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

53 /* Проверить, есть ли объект в кэше */
54 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

55 /* Проверить, есть ли объект в кэше */
56 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

57 /* Проверить, есть ли объект в кэше */
58 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

59 /* Проверить, есть ли объект в кэше */
60 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

61 /* Проверить, есть ли объект в кэше */
62 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

63 /* Проверить, есть ли объект в кэше */
64 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

65 /* Проверить, есть ли объект в кэше */
66 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

67 /* Проверить, есть ли объект в кэше */
68 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

69 /* Проверить, есть ли объект в кэше */
70 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

71 /* Проверить, есть ли объект в кэше */
72 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

73 /* Проверить, есть ли объект в кэше */
74 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

75 /* Проверить, есть ли объект в кэше */
76 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

77 /* Проверить, есть ли объект в кэше */
78 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

79 /* Проверить, есть ли объект в кэше */
80 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

81 /* Проверить, есть ли объект в кэше */
82 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

83 /* Проверить, есть ли объект в кэше */
84 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

85 /* Проверить, есть ли объект в кэше */
86 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

87 /* Проверить, есть ли объект в кэше */
88 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

89 /* Проверить, есть ли объект в кэше */
90 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

91 /* Проверить, есть ли объект в кэше */
92 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

93 /* Проверить, есть ли объект в кэше */
94 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

95 /* Проверить, есть ли объект в кэше */
96 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

97 /* Проверить, есть ли объект в кэше */
98 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

99 /* Проверить, есть ли объект в кэше */
100 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

101 /* Проверить, есть ли объект в кэше */
102 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

103 /* Проверить, есть ли объект в кэше */
104 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

105 /* Проверить, есть ли объект в кэше */
106 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

107 /* Проверить, есть ли объект в кэше */
108 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

109 /* Проверить, есть ли объект в кэше */
110 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

111 /* Проверить, есть ли объект в кэше */
112 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

113 /* Проверить, есть ли объект в кэше */
114 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

115 /* Проверить, есть ли объект в кэше */
116 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

117 /* Проверить, есть ли объект в кэше */
118 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

119 /* Проверить, есть ли объект в кэше */
120 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

121 /* Проверить, есть ли объект в кэше */
122 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

123 /* Проверить, есть ли объект в кэше */
124 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

125 /* Проверить, есть ли объект в кэше */
126 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

127 /* Проверить, есть ли объект в кэше */
128 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

129 /* Проверить, есть ли объект в кэше */
130 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

131 /* Проверить, есть ли объект в кэше */
132 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

133 /* Проверить, есть ли объект в кэше */
134 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

135 /* Проверить, есть ли объект в кэше */
136 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

137 /* Проверить, есть ли объект в кэше */
138 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

139 /* Проверить, есть ли объект в кэше */
140 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

141 /* Проверить, есть ли объект в кэше */
142 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

143 /* Проверить, есть ли объект в кэше */
144 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

145 /* Проверить, есть ли объект в кэше */
146 int kmem_cache_contains(struct kmem_cache *cache, void *obj);

192 / 257

193 / 257

194 / 257

195 / 257

196 / 257

9 ...
10 ;

Листинг 72. Структура `proc_ops`

Свои функции на файле `proc` регистрируют, инициализируют `struct proc_ops` и передавая её функции создания виртуального файла `proc_create_data` (или обёртке `proc_create`, передающей `NULL` в поле приватных данных):

1 struct proc_dir_entry *proc_create_data(const char *name, umode_t mode,
2 struct proc_dir_entry *parent, const struct proc_ops *proc_ops, void *data);
3 struct proc_dir_entry *proc_create(const char *name, umode_t mode,
4 struct proc_dir_entry *parent, const struct proc_ops *proc_ops);

— `proc_ops` — указатель на проинициализированную структуру; `proc_dir_entry` — дескриптор файла `proc`.

— `parent` задаёт иерархию: для файла в поддиректории её создают через `proc_mkdir`; при `parent` == `NULL` файл создаётся в корне `proc`.

12.14 Пример из лабораторной работы

12.14.1 ЛР VFS: собственная файловая система и SLAB-кеш
В ЛР создаётся собственная ФС, по отношению к VFS выполняются два действия:

1) **Регистрация ФС**: объявляем свой `struct file_system_type` (минимально `owner`, `name`, плюс функции `mount` и `kill_sb`); в `init` вызываем `register_filesystem`, в `exit` — `unregister_filesystem`.

2) **Монтирование ФС**: в своей функции `mount` вызываем `mount_nodenv` (ФС не связана с устройством), передавая ей свою `fill_super` (на основе `simple_fill_super`); она инициализирует поля `struct super_block`, через `new_inode` создаёт `inode` корневого каталога и создаёт `root` (`d_make_root`). Освобождение суперблока выполняет свои функции `kill_sb`.

После загрузки модуля ФС монтируется командой `mount` (не менее 2 раз — одна и та же ФС может быть смонтирована много раз) и показывается двумя способами: в дереве каталогов (`ls -la`) и через `cat` (`cat /proc/mounts`); зарегистрированный тип ФС — через `cat /proc/filesystems`.

Кроме того, в задании создаётся свой SLUB-кеш: объявляем свою структуру, создаём для неё кеш, через `alloc` получаем указатель на slab и выделяем объекты.

1 static int my_simple_fill_super(struct super_block *s, void *data, int silent)
2 {
3 struct inode *inode;
4 s->sb_blocksize = PAGE_SIZE;

5 s->sb_magic = MY_MAGIC_NUMBER;
6 s->sb_op = &myfs_super_ops;
7
8 inode = new_inodes(s); // inode корневого каталога
9 inode->i_ino = 3;
10 inode->i_mode = S_IFDIR | 0755;
11 inode->i_op = &my_simple_dir_inode_operations;
12 inode->i_fop = &my_simple_dir_fops; // операции на файле
13 set_nlink(inode, 2);
14 s->sb_root = d_make_root(inode); // создание root
15 return s->sb_root ? 0 : ENOMEM;
16 }

Листинг 73. Функция `fill_super` собственной ФС (`copy`)

1 static struct dentry *myfs_mount(struct file_system_type *type, int flags,
2 char const *dev, void *data)
3 {
4 return mount_nodenv(type, flags, data, my_simple_fill_super);
5 }

6 static struct file_system_type myfs_type = {
7 .owner = THIS_MODULE,
8 .name = "my_fs",
9 .mount = myfs_mount,
10 .kill_sb = my_kill_block_super,
11 };
12
13 static int __init myfs_init(void) { return register_filesystem(&myfs_type); }
14 static void __exit myfs_exit(void) { unregister_filesystem(&myfs_type); }
15 module_init(myfs_init);
16 module_exit(myfs_exit);

Листинг 74. Функции `mount`, `unregister` и `register` собственной ФС (`copy`)

1 struct myfs_inode {
2 int i_mode;
3 unsigned long i_nlink;
4 };
5
6 static struct kmem_cache *inode_cache = NULL;
7 static struct myfs_inode *inode_cached_array = NULL;
8
9 static struct dentry *myfs_mount(struct file_system_type *type, int flags,
10 char const *dev, void *data)
11 {

12 ...
13 for (i = 0; i < OBJECTS_PER_MOUNT; i++)
14 {
15 struct myfs_inode *inode = kmem_cache_alloc(inode_cache, GFP_KERNEL);
16 if (!inode)
17 {
18 for (int j = 0; j < i; j++)
19 kmem_cache_free(inode_cache, inode_cached_array[j]);
20 return ERR_PTR(-ENOMEM);
21 }
22 ...
23 }
24 ...
25 }
26
27 static int __init myfs_init(void) {
28 ...
29 // Создание slab-кэша и init
30 inode_cache = kmem_cache_create(SLAB_NAME, sizeof(struct myfs_inode), 0, 0, NULL);
31 if (!inode_cache) return -ENOMEM;
32 return -ENOMEM;
33 ...
34 }
35
36 static void __exit myfs_exit(void) {
37 ...
38 // освобождение всех объектов
39 for (i = 0; i < cached_count; i++)
40 kmem_cache_free(inode_cache, inode_cached_array[i]);
41 // удаление slab-кэша
42 kmem_cache_destroy(inode_cache);
43 }
44 }

Листинг 75. Создание и использование своего SLAB-кэша в ЛР

12.14.2 ЛР «Фортуны»: регистрация функций на файле /proc
ЛР — пересдача данных между пространством пользователя и ядром; работа с файлом `ФС` `proc` ведётся в модуле ядра. В модуле 6 точек входа: `open`, `read`, `write`, `release`, `init`, `exit`. В `read/write` вызываются `copy_to_user/copy_from_user`, так как ядро работает с физическими адресами, а процессы — с виртуальными.

1 static struct proc_ops fops = {
2 .proc_read = fortune_read,
3 .proc_write = fortune_write,

4 .proc_open = fortune_open,
5 .proc_release = fortune_release,
6 };
7
8 // собственно регистрация при создании вёрт. файла:
9 proc_info_file = proc_create(FILENAME, 0666, NULL, &fops);

Листинг 76. Регистрация функций, определённых на виртуальном файле

1 static char *buffer;
2
3 int fortune_open(struct inode *sp_inode, struct file *sp_file)
4 {
5 printk(KERN_DEBUG "open");
6 return 0;
7 }
8
9 int fortune_release(struct inode *sp_inode, struct file *sp_file)
10 {
11 printk(KERN_DEBUG "release");
12 return 0;
13 }
14
15 // buf - адрес источника в пространстве пользователя.
16 static ssize_t fortune_write(struct file *file, const char __user *buf, size_t count, loff_t *ppos)
17 {
18 ...
19 if (copy_from_user(buffer, buf, count)) {
20 return -EFAULT; // EFAULT - неправильный адрес.
21 }
22 ...
23 }
24
25 static ssize_t fortune_read(struct file *file, char __user *buf, size_t count, loff_t *ppos)
26 {
27 ...
28 len = sprintf(buf, "%s\n", buffer[read_index]);
29 copy_to_user(buf, len);
30 ...
31 }
32
33 static int __init fortune_init(void)
34 {
35 // malloc - выделить непрерывный блок памяти.

Листинг 77. Точки входа модуля «Фортуны»

36 buffer = (char *)vmalloc(buffer_SIZE);
37 ...
38 proc_entry = proc_create("fortune", 0666, NULL, &fops);
39 ...
40 ...
41
42 static void __exit fortune_exit(void)
43 {
44 ...
45 remove_proc_entry("fortune", NULL);
46 ...
47 }

Листинг 78. Точки входа модуля «Фортуны»

Листинг 79. Точки входа модуля «Фортуны»

Листинг 80. Точки входа модуля «Фортуны»

Листинг 81. Точки входа модуля «Фортуны»

Листинг 82. Точки входа модуля «Фортуны»

Листинг 83. Точки входа модуля «Фортуны»

Листинг 84. Точки входа модуля «Фортуны»

Листинг 85. Точки входа модуля «Фортуны»

Листинг 86. Точки входа модуля «Фортуны»

Листинг 87. Точки входа модуля «Фортуны»

Листинг 88. Точки входа модуля «Фортуны»

Листинг 89. Точки входа модуля «Фортуны»

Листинг 90. Точки входа модуля «Фортуны»

13.1 Файловая система: особенности файловой подсистемы Unix/Linux: иерархическая структура файловой подсистемы. Виртуальная файловая система VFS в Linux. Четыре структуры VFS — `super_block`, `inode`, `dentry`, `file` — их назначение. Адресация файлов большого размера в файловой системе `extX` и пример, показывающий доступ к файлу `/usr/ast/mbox`. Компирование `dentry` и `inode`. `Slab` кэш. Монтирование файловых систем. Команда `mount` и функции монтирования, пример из `lab` раб.

13.1 Файловая система

Файл — поименованная совокупность данных, может быть бессмысленная, хранящаяся во вторичной памяти. Понятие файла связано с долговременным хранением информации.

Файловая система — это порядок, определяющий способ организации хранения, именования и доступа к данным на вторичных носителях информации.

Файловая система используется для управления хранением данных и доступа к ним. ФС должна обеспечить доступ к хранимой в системе на вторичных дисках информации.

13.2 Особенности файловой подсистемы Unix/Linux: иерархическая структура файловой подсистемы

Файловая система — важнейшая составляющая любой вычислительной системы: без долговременного хранения данные нельзя обработать повторно (содержимое ОЗУ при выключении пропадает), поэтому разработчики UNIX начали построение системы именно с неё. Внешние ЗУ — блочные устройства, на них же вытесняются страницы при свопинге (`swapping`) и лейджинге (`paging`).

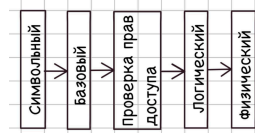
13.3 Задачи файловой системы

- 1) Долговременное хранение данных
- 2) Обеспечение доступа к хранившимся данным

13.4 Уровни иерархии ФС

Для обеспечения доступа к данным нужно предоставить пользователю достаточно удобный способ доступа. Уровни файловой системы:

- 1) Символьный
- 2) Базовый
- 3) Проверка прав доступа
- 4) Логический
- 5) Физический



- **Символьный уровень** — именование файлов: у файлов **имена** (массив символов), собранные по каталогам; корневой каталог в UNIX/Linux — `/`. Имя не является идентификатором файла, а лишь позволяет удобно с ним работать; идентификатором файла является `inode`; определены жесткие (`hard links`) и символические (`soft links`) ссылки.
- **Базовый уровень** — идентификация файла: это `inode`; идентификатором является номер `inode`, который описывает физический файл.
- **Проверка прав доступа** — права отдельно для пользователя, группы и остальных (команда `chmod`).
- **Логический уровень** — процесс открывает файл и выполняет над ним операции: файл считает, что начинается с нулевого смещения (`offset`), каждая операция чтения/записи сменяет указатель: файл байт-ориентированный, размера не имеет. Здесь определяется адресация данных, преобразуемая в физические адреса на диске.
- **Физический уровень** — уровень физической памяти; логический и физический уровни связывает драйвер дискового устройства, переводящий смещения в физические адреса.

13.4.1 Иерархическая структура каталогов

Файловые системы знакомых нам ОС являются иерархическими — это негласное правило, определённое именно в UNIX. Для Unix характерно существование корневой каталог `/`, от которого ветвится дерево каталогов:



```
12 const struct file_operations *f_op;
13 union { struct cdev *i_cdev; char *i_link; ... }; // для спецфайлов
14 ;
```

Листинг 79. struct inode (cop.)

- `i_ino` — номер `inode` (идентификатор файла); `i_link` — количество жестких ссылок; `i_size` — размер файла.
- `i_op`, `i_fop` — операции на `inode` и на файле; `i_sb` — указатель на суперблок (физический файл доступен только если ФС смонтирована).
- Флаги (`i_flags`) играют роль ограничений: указывают допустимые операции над файлом и над самим `inode`.

13.6.3 dentry — элемент пути (directory entry)

В VFS каталоги обрабатываются как файлы и хранятся во вторичной памяти. Элемент пути (`for / до следующего /`) — это `dentry`. Например, в `/bin/vi` объекты `dentry` — `/`, `bin`, `vi` (первые два — каталоги, последний — файл). Объектами `dentry` являются все элементы пути, включая файлы и точки монтирования; VFS создаёт их «на лету» при операциях с каталогами. Концепция `dentry` введена потому, что проход по элементам пути — долгое действие, связанное со сравнением строк, а VFS часто выполняет специфические для каталога операции (поиск элемента пути по имени с проверкой и переходом к следующему).

`struct dentry` не соответствует никакой информации из вторичной памяти: она нигде не хранится и создается «на лету» на основе данных директорий на диске. Поэтому у `dentry` нет флагов измёнчивости.

```
1 struct dentry {
2     /* RCU lookup touched fields */
3     unsigned int i_flags; /* protected by d_lock */
4     ...
5     struct hlist_bl_node d_hash; /* lookup hash list */
6     struct dentry *d_parent; /* parent directory */
7     struct dentry *d_name;
8     struct inode *d_inode; /* Where the name belongs to - NULL is
9     ...
10    const struct dentry_operations *d_op;
11    struct super_block *d_sb; /* The root of the dentry tree */
12    ...
13    struct list_head d_lru; /* LRU list */
14    ...
15    struct list_head d_subdirs; /* our children */
16    ...
```

13.5 Виртуальная файловая система VFS в Linux

Для Unix был предложен интерфейс `VFS/vnode` (virtual file system, virtual node). Linux отказался от `vnode`, оставив только `VFS` — интерфейс, через который ОС получает доступ к большому числу разных ФС, скрывая до минимума количество системных вызовов: любая работа с файлом, любой ФС идёт через один и тот же интерфейс. Основа такой работы — монтирование: прежде чем ФС станет доступна (станут видны её каталоги и файлы), она должна быть смонтирована.

В основе интерфейса VFS лежат 4 структуры (это ядерные структуры):

- 1) `struct super_block`;
- 2) `struct inode`;
- 3) `struct dentry`;
- 4) `struct file`.

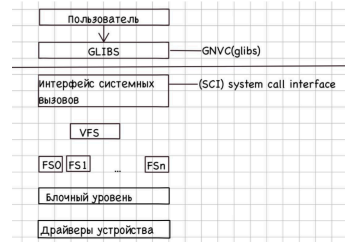


Рис. 40. Virtual File System

13.6 Четыре структуры VFS — super_block, inode, dentry, file — их назначение

Все четыре структуры взаимодействуют между собой указателями, и для каждого файла создаются их экземпляры. Кратко их назначение: `super_block` описывает смонтированную ФС, `inode` — физический файл, `dentry` — элемент пути (имя), `file` — открытый файл.

13.6.1 super_block — описывает смонтированную ФС

Суперблок представляет в VFS информацию о смонтированной ФС — структура с данными, необходимыми для монтирования и доступа к файлам ФС: метаданные для обращения к файлам, хранившиеся «вразброс» (адресация соответствующих участков диска). В каждой ФС ровно один суперблок, он располагается в начале раздела (есть резервная копия). При монтировании суперблок считается в память ядра и находится там до размонтирования.

```
1 struct super_block {
2     struct list_head s_list;
3     dev_t s_dev;
4     unsigned long s_blocksize;
5     loff_t s_maxbytes; // Max file size
6     struct file_system_type *s_type;
7     const struct super_operations *s_op;
8     unsigned long s_flags;
9     unsigned long s_magic;
10    struct dentry *s_root; // точка монтирования
11    struct rw_semaphore s_mounts;
12    struct list_head s_mounts;
13    struct block_device *s_bdev;
14    struct list_head s_inodes; // all inodes
15 ;
```

Листинг 78. Структура `super_block` (cop.)

- `s_blocksize`, `s_maxbytes` — размер блока и максимальный размер файла;
- `s_magic` — магическое число ФС.
- `s_type`, `s_op` — тип ФС и функции на суперблоке (`super_operations`); `s_root` — точка монтирования (корневой каталог); `s_inodes` — список всех `inode`.
- `s_mounts` — список монтированных данной ФС (одна и та же ФС может быть смонтирована много раз).

⚠ Осторожно

Вместо первого рисунка суперблока, который Рязанова не показывала (показывала), вероятно, хочет видеть второй.

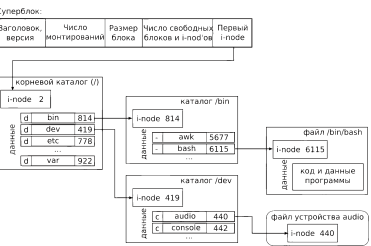


Рис. 41. Суперблок

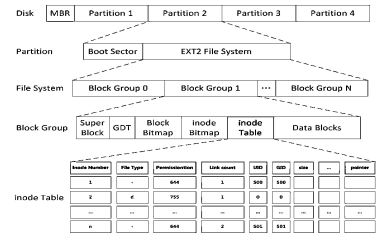


Рис. 42. Суперблок

13.6.2 inode — дескриптор физического файла

`inode` — Дескриптор файла содержит множество полей, но идентификатором является номер `inode` (`i_ino`, метаданные). `inode` описывает физический файл — реально хранимый информации. Есть 2 вида `struct inode`:

- 1) **дисковый `inode`** — описывает физический файл, содержит адресную информацию, то есть адреса блоков, в которых хранится информация, записанная в данный файл (на диске адреса и на диске информации);
- 2) **`inode` в ядре** содержит информацию о файле для ускорения доступа (копия в памяти, дополнительные поля: логический номер устройства, номер индекса, ссылки для хеш-операций).

```
1 struct inode {
2     union { struct dentry *dentry; struct super_block *sb; }
3     ...
4     const struct inode_operations *i_op;
5     struct super_block *i_sb;
6     unsigned long i_ino;
7     unsigned int i_nlink; // Количество жестких ссылок
8     loff_t i_size;
9     unsigned int i_blocks;
10    struct rw_semaphore i_rsem;
11    ... // список/хеш: i_lru, i_sb_list, i_hash, i_dentry, ...
```

17 ;

Листинг 80. struct dentry

- `d_parent` — объект `dentry` родительского каталога; `d_name` — имя, `d_inode` — связанный `inode` (если он NULL, элемент пути — `negative`);
- `d_op` — допустимые действия над элементом пути; `d_sb` — суперблок: любой каталог и файл относятся к конкретной смонтированной ФС.

13.6.4 file — дескриптор открытого файла

Различают файлы и открытые файлы: просто файлы представлены деревом каталогов, физический файл описывает `struct inode`. Открытый файл — файл, с которым работает процесс; его описывает `struct file`. Один файл может быть открыт много раз (в том числе разными процессами): сколько открытий, столько дескрипторов открытого файла, каждый со своим смещением.

```
1 struct file {
2     ...
3     spinlock_t f_lock; // монополия доступа
4     mode_t f_mode; // режим доступа
5     atomic_long_t f_count; // счетчик ссылок на файл
6     loff_t f_pos; // логическая позиция в файле
7     ...
8     struct path f_path; // связь с dentry и ФС
9     struct inode *f_inode; // физический файл
10    const struct file_operations *f_op; // операции на открытом файле
11    ...
12 ;
```

Листинг 81. Структура `file` (cop.)

- `f_lock` — монополия доступа к файлу; `f_mode` — режим доступа; `f_count` — счетчик ссылок на файл.
- `f_pos` — логическая позиция в файле: любой файл считает, что начинается с нулевого смещения, каждая операция чтения/записи изменяет это поле на соответствующее количество байт.
- `f_path` — связь с элементом пути (`dentry`) и смонтированной ФС; `f_inode` — указатель на `inode` (физический файл); `f_op` — операции, определенные на открытом файле.

Связь структур строится относительно процесса (именно процессы открывают файлы): через `task_struct` по файловому дескриптору находится `struct file`; его поле `f_path.f_inode` ведет к `dentry` и `inode`, а из `inode` (поле `i_sb`) и со стороны монтирования — к `super_block` смонтированной ФС.

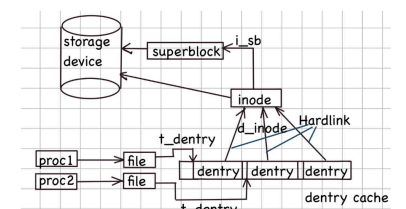


Рис. 43. Связь структур

13.7 Адресация файлов большого размера в файловой системе extX и пример, показывающий доступ к файлу /usr/ast/mbox

Адресное пространство диска делится на разделы, один раздел занимает одна ФС.

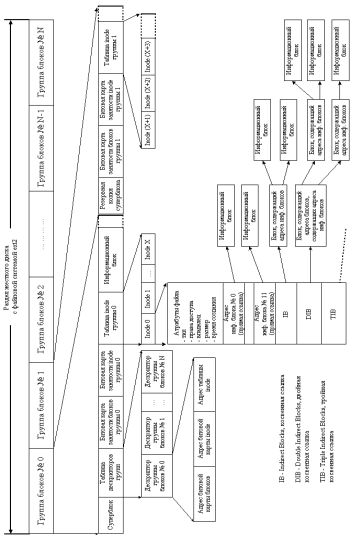


Рис. 44. Обобщенная структура схемы файловой системы ext2

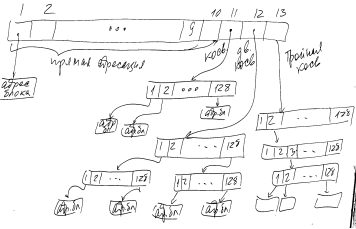


Рис. 45. Адресация файла на жёстком диске

Логический и физический уровни ФС связывает драйвер дискового устройства, переводящий смещения в логическом представлении файла в физические адреса: struct inode содержит адреса блоков с данными файла. Адресуемая единица файла – блок.

Каждый блок по 512 байт. С помощью третьего уровня (тройной косвенной адресации) появляется возможность обращаться к 128³ = 2097152 блокам. Таким образом, в файловую систему Unix была заложена возможность создавать и работать с очень большими файлами.

Файловая система ext2 (Second Extended File System) была написана так, чтобы обеспечить доступ к хранению очень больших файлов:

- в дисковом inode 12 блоками адресуется прямо – inode непосредственно содержит адреса 12 блоков данных;
- далее используются косвенная, двойная косвенная и тройная косвенная адресация: блок, содержащий адреса блоков информации, имеет размер 4 Кб (страница), и при размере указателя 4 байта в нём помещается 1024 адреса (1024 · 4 байта = 4 Кб);

– чтобы обратиться к блоку данных через косвенную адресацию, нужно многократно обращаться к диску за получением информации; это не делается аппаратно, а выполняется драйвером, поэтому время кратно увеличивается.

13.7.1 Пример доступа к файлу /usr/ast/nbox

Каталоги хранятся на диске аналогично файлам. Содержимое каталога – таблица, сопоставляющая имена файлов и подкаталогов с номерами их inode:

inode number	3470036
..	3470036
..	3470017
folder1	3470031
file1	3470043

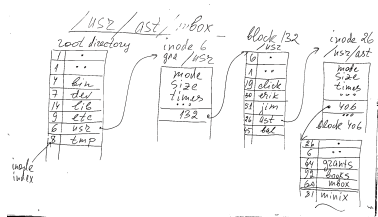
– В таблице видим имена файлов и папок (folder), видим номера inode.

– number – более правильное слово, потому что index – это всегда смещение.

– Структура inode директории состоит из имен, сопоставленных с inode.

Таблица 8. Структура inode каталога

Поиск файла всегда начинается с корневого каталога (для текущей директории её часть пути уже пройдена).



Доступ к файлу /usr/ast/nbox (в пути 4 объекта дентри: /, usr, ast, nbox):

- 1) Путь начинается с корневого каталога (root directory); его содержимое известно системе.
- 2) В корневом каталоге ищется элемент пути usr – ему сопоставлен inode с номером 6.

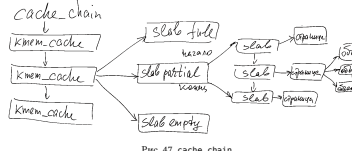


Рис. 47. cache_chain

Каждый slab может находиться в одном из трёх состояний:

- **slab full** – полностью распределённый: нет свободных объектов, все распределены и используются;
- **slab partial** – частично распределённый: есть как выделенные, так и свободные объекты;
- **slab empty** – пустой: нет ни одного распределённого объекта, все объекты свободны для последующего использования.

Выделение: новый объект берётся из частично распределённого слэба, иначе из пустого; если пустых нет, слэбы создаются по мере необходимости. Побочный эффект – уменьшение фрагментации памяти.

```
1 /* Создание кэша */
2 struct kmem_cache *kmem_cache_create(const char *name, size_t size, size_t offset,
3 unsigned long flags, void (*constructor)
4 (void *));
5
6 /* Уничтожение кэша */
7 void kmem_cache_destroy(struct kmem_cache *cache);
8
9 /* Выделить объект из кэша */
10 void *kmem_cache_alloc(struct kmem_cache *cache, gfp_t flags);
11
12 /* Удальить объект из кэша */
13 void kmem_cache_free(struct kmem_cache *cache, void *obj);
```

Листинг 82. Функции для работы с кешем SLAB

Кеш SLAB напрямую используется подсистемой VFS: объекты дентри создаются и уничтожаются вызовами kmem_cache_alloc() и kmem_cache_free(), а внутри new_inode виден аллокатор – inode аллоцируется в slab-кеше.

13.9 Монтирование файловых систем. Команда mount и функции монтирования, пример из лаб раб

Дерево файлов и каталогов в Linux формируется из ветвей, соответствующих разным носителям (разным ФС), но это одно большое дерево; некоторые каталоги могут быть точками монтирования.

Монтирование – система действий, делающая ФС (и все её файлы) доступной. Точка монтирования – каталог, к которому подмонтируется ФС: файлы и директории монтируемой ФС становятся файлами и поддиректориями точки монтирования, а её прежние файлы скрываются до размонтирования (потому лучше использовать пустые каталоги в качестве точки монтирования). Доступ к файлам ФС возможен только после монтирования. В Unix/Linux всегда есть корневая ФС (root fs), её монтирование – часть инициализации системы.

Когда ФС монтируется, создается структура struct vfsmount, которая представляет конкретный экземпляр смонтированной ФС.

```
1 struct vfsmount {
2 struct dentry *mnt_root; /* root of the mounted tree */
3 struct super_block *mnt_sb; /* pointer to superblock */
4 int mnt_flags;
5 struct mt_idmap *mnt_idmap;
6 };
```

Листинг 83. Структура vfsmount

mnt_root – корень подмонтированного дерева, mnt_sb – указатель на суперблок, mnt_flags – флаги монтирования.

13.9.1 Команда mount

Команда mount имеет следующий синтаксис:

```
1 mount <устройство> <точка монтирования>
```

Ключи: <-t тип ФС> -o <опции ФС>, то есть полная форма:

```
1 mount -t type device dir
```

Точка монтирования – любой каталог; в самой ФС нужно создать корневой каталог (root) – именно он монтируется. Ключ -t необязателен: система сама может определить тип ФС. После монтирования каталоги и файлы ФС доступны. Обычно монтирование требует прав суперпользователя.

С помощью команды

```
1 mount [-A] [-t type]
```

можно получить список смонтированных файловых систем типа tree. Размонтируется ФС командой umount <точка монтирования>.

13.9.2 Функции монтирования

В ядре определено несколько функций mount(); все они имеют модификатор extern и возвращают значение типа struct dentry*. В версии ядра 6.17.13 определено только две:

```
1 extern struct dentry *mount_bdev(struct file_system_type *fs_type, int flags,
2 const char *dev_name, void *data, int (*fill_super)
3 (struct super_block *, void *, int));
4 extern struct dentry *mount_nodev(struct file_system_type *fs_type, int flags,
5 void *data, int (*fill_super)
6 (struct super_block *, void *, int));
```

Параметры: тип файловой системы (fs_type), флаги монтирования (flags), имя устройства (dev_name, только у mount_bdev), произвольные данные монтирования (data) и указатель на функцию fill_super. Более отличие – в параметре const char *dev_name, которого нет у mount_nodev.

- mount_bdev – для монтирования ФС, находящейся на блочном устройстве;
- mount_nodev – для монтирования ФС, не связанной ни с каким устройством.

Функцией монтирования передаётся функция fill_super – она выполняет основные действия для монтирования: инициализирует поля struct super_block (размер блока, смещение, магическое число, super_operations). Функции монтирования возвращают объект дентри, которому должен быть root: для ФС нужно создать корневой каталог, а для этого – inode. В системе существует готовая функция simple_fill_super, которую находим в коде ядра и используем как основу для собственной fill_super.

Для создания root надо проинициализировать inode, выделив для него память функцией new_inode() (внутри неё виден аллокатор – inode аллоцируется в slab-кеше):

```
1 struct inode *new_inode(struct super_block *sb) {
2 struct inode *inode = new_inode_pseudo(sb); // -> alloc_inode(); выделение
3 из slab
4 if (inode)
5 inode->i_list.add(inode);
6 return inode;
```

- 3) Поля struct inode для данного каталога (mode, size, times, ...) позволяют идти дальше: нас интересует адресная часть – в inode 6 указано, что информация каталога находится в блоке 132. Обращаемся к диску: блок 132 – это содержимое директории /usr.
- 4) В блоке 132 ищется элемент ast – ему сопоставлен inode с номером 26 (это структура для /usr/ast). В этой структуре поля один и те же, они не меняются в зависимости от того, ней это inode.
- 5) Из inode 26 берётся адрес блока 406 – это содержимое директории /usr/ast.
- 6) В блоке 406 ищется nbox – ему сопоставлен inode с номером 60; из этого inode берётся адреса блоков, в которых хранятся данные файла nbox.

Пример показывает, как много действий (обращений к диску) нужно выполнить для обращения к конкретному файлу, поэтому кеширование каталогов и файловых индексов позволяет сократить это время обращения.

13.8 Кеширование dentry и inode. Slab кэш

13.8.1 Кэш dentry

Структуры dentry хранятся в кеше. Когда происходит первое обращение к элементу пути, создается объект дентри и помещается в кеш. При обращении к файлу система проходит по элементам пути и ищет их в dentry-кеше; если dentry не найден, обращение идёт к диску (см. пример с nbox выше).

Действительный объект дентри может быть в одном из трёх состояний (used, unused, negative); им соответствуют три части dentry-кеша:

- 1) **Список используемых (used)**: у dentry есть inode (d_inode), счётчик d_count > 0; объект не удаляется, пока d_count не 0.
- 2) **Список неиспользуемых (unused)** – двусторонний LRU-список (dentry_lru), сортируется по времени последнего обращения; при нехватке памяти удаляются «старые» объекты.
- 3) **Кеш-таблица (dentry_hashtable)** и **функция кеширования** быстро преобразуют путь в dentry; кеширование – d_hash(), поиск – d_lookup().

Доступ к файлам выполняется «в пространстве» программы обращаются к разным файлам одного каталога, потому что кеширование всех элементов пути одного файла сокращает время доступа к к соседним файлам этого каталога. В примере с nbox получение корневого индексного дескриптора упрощается: он кеширован уже при

монтировании корневого каталога. Сама dentry не содержит номера индексного дескриптора – он загружается отдельно.

- 13.8.2 Кэш inode**
- Кеш inode:
- 1) **Кеш-массив inode_hashtable** – inode кешируется по указателю на суперблок и номеру inode.
 - 2) **Список inode_in_use** – допустимые inode с i_count > 0 и i_nlink > 0 (туда new_inode добавляет созданный inode).
 - 3) **Список inode_unused** – допустимые inode с i_count == 0.
 - 4) **Для каждого суперблока – список sb->s_dirty** – модифицированные («грязные», I_DIRTY) inode с i_count > 0; уменьшает накладные расходы на синхронизацию с диском.
 - 5) inode_cached – slab-кеш.

Кеш взаимодействуют: inode, на которые ссылаются неиспользуемые объекты dentry, не удаляются, пока на них ссылается dentry-кеш.

13.8.3 Slab кэш

Slab – «буфер». Ядро оперирует ограниченным числом объектов (структур), для каждого нужно выделить память и проинициализировать поля, что занимает время. Если удалить объект после использования, для следующего такого же объекта придётся продумать ту же работу заново.

Идея: не удалять объекты, а сохранить их в slab для повторного использования. Под разные структуры нужны разные кешы – это и реализует Slab-кеш. Каждый кеш хранит объекты одного размера (больше или меньше страницы); есть слабы и для часто используемых размеров буферов (16, 32, 64, ...). Все кешы видны через cat /proc/slabinfo.

Важно

Для каждого типа данных создается свой SLAB кеш. Не один на всех, а для каждого.

Побочный эффект слаб – сокращение фрагментации памяти.

С версии ядра 2.6.23 SLAB переименован в SLUB (это ещё SLOB). Это тоже код ядра: из ядра слаб недоступен – память выделяется страницами или слабыми узлами в ядре.

Имеются структуры kmem_cache; каждый кеш представляется такой структурой, а кеш связаны друг с другом в списке cache_chain (цепочка кешей):

Листинг 84. Функция new_inode (copr.)

Когда файловая система монтируется, всё должно быть удалено – этим занимается функция kill_sb на file_system_type. В ядре для этого определены функции:

```
1 void generic_shutdown_super(struct super_block *sb);
2 void kill_block_super(struct super_block *sb);
3 void kill_anon_super(struct super_block *sb);
4 void kill_litter_super(struct super_block *sb);
```

Общую работу по удалению суперблока выполняет generic_shutdown_super.

13.9.3 Пример из лабораторной работы (JP VFS)

В JP создается собственная (виртуальная) ФС в загружаемом модуле ядра; по необходимости к VFS выполняются два действия:

- 1) **Регистрация ФС**: объявляем свой struct file_system_type (минимально owner, name, плюс функции mount и kill_sb); в init вызываем register_filesystem, а в exit – unregister_filesystem.
- 2) **Монтирование ФС**: в своей функции mount вызываем mount_nodev (ФС не связана с устройством), передаем ей свою fill_super (на основе simple_fill_super); она инициализирует поля struct super_block, alloc, через new_inode создаёт inode корневого каталога и создаёт root (d_make_root). Освобождение суперблока выполняем свои функциями kill_sb.

После загрузки модуля ФС монтируется кешем (не менее 2 раз – одна и та же ФС может быть смонтирована много раз) и показывается двумя способами: в дереве каталогов (ls -lat) и через proc (cat /proc/mounts; зарегистрированный тип ФС – через cat /proc/filesystems). Кроме того, в задании создается свой SLUB-кеш: объявляем свою структуру, создаём для неё кеш, через alloc получаем указатель на slab и выделяем объекты.

```
1 static int my_simple_fill_super(struct super_block *sb, void *data, int silent)
2 {
3 struct inode *inode;
4 s->blocksize = PAGE_SIZE;
5 s->magic = MY_MAGIC_NUMBER;
6 s->op = myfs_super_ops;
7
8 inode = new_inode(1); // inode корневого каталога
9 inode->i_ino = 1;
10 inode->i_mode = S_IFDIR | 0755;
11 inode->i_op = my_simple_dir_inode_operations;
12 inode->i_fop = my_simple_dir_operations; // операции на файле
13 set_nlink(inode, 2);
```

```
14 s->root = d_make_root(inode); // создание root
15 return s->root ? 0 : -ENOMEM;
16 }
```

Листинг 85. Функция fill_super собственной ФС (copr.)

```
1 static struct vfsmount *myfs_mount(struct file_system_type *type, int flags,
2 const char *dev, void *data)
3 {
4 return mount_nodev(type, flags, data, my_simple_fill_super);
5 }
6
7 static struct file_system_type myfs_type = {
8 .owner = THIS_MODULE,
9 .name = "myfs",
10 .mount = myfs_mount,
11 .kill_sb = my_kill_block_super,
12 };
13
14 static int __init my_init(void) { return register_filesystem(&myfs_type); }
15 static void __exit my_exit(void) { unregister_filesystem(&myfs_type); }
16 module_init(my_init);
17 module_exit(my_exit);
```

Листинг 86. Функции mount, регистрации и deregистрации собственной ФС (copr.)

```
1 struct myfs_inode {
2 int i_mode;
3 unsigned long i_ino;
4 };
5
6 static struct kmem_cache *inode_cache = NULL;
7 static struct myfs_inode** inode_cached_array = NULL;
8
9 static struct dentry *myfs_mount(struct file_system_type *type, int flags,
10 const char *dev, void *data)
11 {
12 ...
13 for (i = 0; i < OBJECTS_PER_MOUNT; i++)
14 {
15 struct myfs_inode *inode = kmem_cache_alloc(inode_cache, GFP_KERNEL);
16 if (!inode)
17 for (int j = 0; j < i; j++)
18 kmem_cache_free(inode_cache, inode_cached_array[j]);
19 return ERR_PTR(-ENOMEM);
20 }
```

```
21 }
22 ...
23 }
24 ...
25 }
26
27 static int __init myfs_init(void) {
28
29     // Создание slab-кэша и init
30     inode_cache = kmem_cache_create(SLAB_NAME, sizeof(struct myfs_inode), 0, 0, NULL);
31     if (inode_cache == NULL)
32         return -ENOMEM;
33     ...
34 }
35
36 static void __exit myfs_exit(void) {
37     ...
38     // освобождение всех объектов
39     for (i = 0; i < cached_count; i++)
40         kmem_cache_free(inode_cache, inode_cached_array[i]);
41     // удаление slab-кэша
42     kmem_cache_destroy(inode_cache);
43     ...
44 }
```

Листинг 87. Создание и использование своего SLAB-кэша и LRU

222 / 257

Элемент	Тип	Содержание
		4) 55 бит — dirty. Он указывается в дескрипторе страницы. С 54 по 0: если страница присутствует в физической памяти, то эти биты являются номером фрейма, то есть номером физической страницы.
task	Директория	Содержит поддиректории потоков
interrupts	Файл	Предоставляет таблицу о количестве прерываний на каждом из процессоров в следующем виде: 1) Первая колонка: номер прерывания 2) Колонки CPUx: счётчики прерываний на каждом из процессоров 3) Следующая колонка: вид прерывания: 3.1) IO-APIC-edge — прерывание по фронту на контроллер I/O APIC. 3.2) IO-APIC-fastedge — прерывание по уровню на контроллер I/O APIC. 3.3) PCI-MSI-edge — MSI прерывание. 3.4) XT-PIC-XT-PIC — прерывание на PIC контроллер. 4) Последняя колонка: устройство, ассоциированное с данным прерыванием.

Таблица 9. Содержание директории /proc/cpuid-

14.1.1 Ссылка self
/proc/self — символическая ссылка, существующая для каждого процесса и указывающая на его собственную директорию /proc/cpid-; позволяет процессу обращаться к своим данным, не вызывая getpid().

14.1.2 Информация об окружении
Окружение несёт файл environ (для текущего процесса — /proc/self/environ): в нём исходное окружение, установленное при запуске (execve()), переменные разделены нулевыми символами ('\0'), на строки файл не разбит. Изменения окружения после execve() (например, putenv()) в нём не отражаются.

14.1.3 Информация о состоянии процесса
Информацию о состоянии процесса несёт файл stat — наикруп, подобранную разработчиками информацию о процессе: его идентификатор pid, состояние state,

227 / 257

223 / 257

ния команды comm, идентификатор родителя ppid, приоритет prio, размер виртуального адресного пространства vsize, адрес начала кучи start_brk и т.д.

14.1.4 Информация о прерываниях
Файл /proc/interrupts несёт информацию о прерываниях на каждом CPU с указанием их источника. Формат записи:
— номер прерывания;
— колонки CPUx — счётчики прерываний на каждом процессоре;
— вид прерывания (IO-APIC-edge/-level, PCI-MSI-edge, XT-PIC и т.д.);
— устройство, ассоциированное с прерыванием.

Кроме /proc/interrupts, информацию о системе в целом дают, например, /proc/cpuinfo, /proc/vmstat, /proc/softirqs.

14.2 Структура proc_dir_entry: функции для работы с элементами /proc
Все файлы и поддиректории файловой системы proc описываются структурой struct proc_dir_entry — она фактически является дескриптором файла proc. Поскольку proc — виртуальная ФС, данные не хранятся во вторичной памяти, но inode у любого файла, в том числе виртуального, всё равно имеется.

```
1 struct proc_dir_entry {
2     // ...
3     const struct inode_operations *proc_ops;
4     union {
5         const struct proc_ops *proc_ops;
6         const struct file_operations *proc_dir_ops;
7     };
8     union {
9         const struct seq_operations *seq_ops;
10        int (*single_show)(struct seq_file *, void *);
11    };
12    proc_write_t write;
13    void *data;
14    struct proc_dir_entry *parent;
15    char *name;
16    // ...
17};
```

Листинг 88. Структура struct proc_dir_entry

— proc_ops — у файла proc есть inode и определённые на нём inode_operations.
— union(1); до 6-й версии ядра было только file_operations *proc_dir_ops; рядом введена proc_ops с почти теми же функциями (зачем — см. регистрацию).

228 / 257

— union(2): два интерфейса с user-mode — полный seq_operations *seq_ops и упрощённый single_show.
— write — передача из user-mode в kernel-mode (обратно — read); data — приватные данные файла: parent — древовидная структура proc.

Функции для работы с элементами /proc:
— proc_mkdir — создаёт поддиректорию в /proc;
— proc_create_data (и её обёртка proc_create) — создаёт виртуальный файл и регистрирует на нём свои функции (подробнее — в подразделе про регистрацию);
— remove_proc_entry — удаляет элемент /proc (вызывается при выгрузке модуля).

14.3 Структура, перечисляющая функции, определённые на файлах

В struct file (дескрипторе открытого файла) поле f_op указывает на структуру file_operations, в которой перечислены функции, определённые на открытом файле:

```
1 struct file_operations {
2     struct module *owner;
3     ...
4     ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
5     ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
6     int (*open) (struct inode *, struct file *);
7     int (*release) (struct inode *, struct file *);
8     ...
9};
```

Листинг 89. Структура file_operations

Важно
Здесь перечисляются функции ядра. struct file — структура ядра. Разработчики называют функции ядра аналогично функциям стандартных библиотек, но это функции ядра. Поэтому, например, в функции read присутствует указатель user — на буфер пользователя.

— Для struct file_operations существуют поле owner (владелец).
В перечне функций kernel close отсутствует — закрытие соответствует release.

14.4 Использование структуры file_operations для регистрации собственных функций работы с файлами

Регистрация функций выполняется с помощью функции инициализации структуры назначенными инициализаторами (синтаксис C99):

229 / 257

14.4.1 Файловая подсистема /proc — назначение, особенности, файлы, поддиректории, ссылка self, информация об окружении, состоянии процесса, прерываниях
Структура proc_dir_entry: функции для работы с элементами /proc. Структура, перечисляющая функции, определённые на файлах. Использование структуры file_operations для регистрации собственных функций работы с файлами. Передача данных из пространства пользователя в пространство ядра и из ядра в пространство пользователя. Обоснование необходимости этих функций. Функция printk() — назначение и особенности. Пример программы «Фортуны» из лаб. работы.

14.1.1 Файловая подсистема /proc: назначение, особенности, файлы, поддиректории, ссылка self, информация об окружении, состоянии процесса, прерываниях

Виртуальная файловая подсистема /proc (process information pseudo-file-system) позволяет в режиме пользователя получить большой объем информации о ресурсах процесса. Виртуальная — то есть не существующая на самом деле (не хранящаяся во вторичной памяти), а создающаяся «на лету» по мере обращения к данным. По сути интерфейс /proc позволяет получить информацию из структур ядра, не обращаясь к ним напрямую, а потому работая в режиме пользователя. Большинство файлов доступны только для чтения, однако некоторые доступны для записи, что позволяет изменять переменные ядра.

Файловая система /proc является немониторимой. Под этим подразумевается, что она не связана ни с каким внешним посетителем (она видна в файловых системах, подключенных к корневому каталогу, но не связана ни с каким внешним устройством. Поэтому она и называется псевдофайловой системой).

Для каждого процесса в /proc определена директория идентификатора процесса (/proc/cpid-), она будет создана налету. В ней хранятся файлы, директории и символические ссылки, используя которые можно получить развёрнутую информацию о конкретном процессе. Вся информация доступна из user-mode:

Элемент	Тип	Содержание
cmdline	Файл	holds the complete command line Содержит командную строку процесса
cwd	Символическая ссылка	Указывает на рабочую директорию процесса
environ	Файл	Содержит список переменных окружения процесса
exe	Символическая ссылка	Указывает на образ процесса (на его файл) Символическая ссылка, содержащая фактический путь к выполняемой команде.

225 / 257

```
1 struct file_operations fops = {
2     .read = device_read,
3     .write = device_write,
4     .open = device_open,
5     .release = device_release
6 };
```

struct file_operations перенести нельзя, а её функции вызывают все драйверы. Чтобы её разгрузить, для работы с ФС proc введена struct proc_ops, функции которой повторяют функции file_operations:

```
1 struct proc_ops {
2     ...
3     int (*proc_open)(struct inode *, struct file *);
4     ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
5     ssize_t (*proc_write)(struct file *, const char __user *, size_t, loff_t *);
6     int (*proc_release)(struct inode *, struct file *);
7     ...
8};
```

Листинг 90. Структура proc_ops

Свои функции на файле proc регистрируют, инициализировав struct proc_ops и передавая её функции создания виртуального файла proc_create_data (или обёртку proc_create, передающей NULL в поле приватных данных):

```
1 struct proc_dir_entry *proc_create_data(const char *name, umode_t mode,
2 struct proc_dir_entry *parent, const struct proc_ops *proc_ops, void *data);
3 struct proc_dir_entry *proc_create(const char *name, umode_t mode,
4 struct proc_dir_entry *parent, const struct proc_ops *proc_ops);
```

— proc_ops — указатель на пронициализированную структуру; proc_dir_entry — дескриптор файла proc.
— parent задаёт иерархию: для файла в поддиректории её предварительно создают через proc_mkdir; при parent == NULL файл создаётся в корне proc.

14.5 Передача данных из пространства пользователя в пространство пользователя и из ядра в пространство пользователя. Обоснование необходимости этих функций

В своих функциях read, write нужно вызывать пространство пользователя ядра для передачи данных из пространства пользователя в пространство ядра и обратно — это copy_from_user и copy_to_user.

230 / 257

Элемент	Тип	Содержание
comm	Файл	Имя команды, связанное с процессом. Это именно то имя файла, которое приведёт к выполнению нашего кода. Замечание 14.1 Разные потоки в одном и том же процессе могут иметь разные значения, доступные через /proc/cpid-/task/<tid>/comm. task — файл, в котором находится поддиректория потоков.
stat	Файл	Содержит информацию о процессе (vszie, start_brk и др.)
fd	Директория	Содержит ссылки на открытые процессом файлы Замечание 14.2 В системе различаются файлы и открытые файлы. Открытые файлы описываются структурой struct file. Простые файлы представляются в системе деревом каталогов, а описываются бизнес-файлом struct inode. Одна и та же файл может быть открыт много раз, приёмом разными процессами. В таблице открытых файлов на каждой такой открытой файл будет дескриптор struct file
root	soft link	Это символическая ссылка, которая указывает на корень файловой системы процесса. Замечание 14.3 Процессы, которые выполняются на нашем компьютере, могут принадлежать разным файловым системам. Пока мы имеем исполняемый файл, не указав в командной строке, пока он не начал выполняться, он может на вторичной памяти.
maps	Файл	Это адресное пространство того процесса, pid которого мы указали. В зависимости от pid информации будет отличаться, но есть неизменные составляющие:
pagewar	файл	Это 64-х разрядное значение для каждой строки процесса. При этом нас интересуют фактически несколько бит: 1) 63 бит — present (in RAM) 2) 62 бит — swap space, область свопинга 3) 61 — shared anonymous page

226 / 257

Специальные функции нужны, потому что ядро (и модуль ядра как его часть) оперирует физическими адресами: оно резидентно в физической памяти и не выгружается, а процессы имеют лишь виртуальные адресные пространства и получают физические страницы по мере необходимости.

Чтобы передать данные из user в kernel или обратно, ядро должно обратиться к буферу пространства пользователя. Вероятна ситуация, когда буфер (страница), к которому обращается ядро, выгружен — тогда возникает странное прерывание, которое должно обработать само ядро. Получается, что ядро обратилось, само стало источником прерывания и само же должно его обработать; если не предпринять специальных мер, возникнет паника ядра. Для этого и написаны две функции:

— copy_to_user — копирует данные из буфера ядра в буфер пользователя;
— copy_from_user — копирует данные из буфера пользователя в буфер ядра.

```
1 static inline __must_check unsigned long
2 _copy_to_user(void __user *to, const void *from, unsigned long n)
3 {
4     might_fault();
5     if (access_ok(to, n))
6         n = raw_copy_to_user(to, from, n);
7     return n;
8 }
```

Листинг 91. Определение функции copy_to_user

— Указатель to (void __user *) — указатель на буфер пользователя, const from — указатель на буфер ядра. Для функции copy_from_user ситуация обратная.

```
1 static inline __must_check unsigned long
2 _copy_from_user(void __user *, const void *from, unsigned long n)
3 {
4     ...
5 }
```

Листинг 92. Определение функции copy_from_user

14.6 Функция printk() — назначение и особенности
printk (kernel) — функция ядра, не входящая ни в одну библиотеку (в ядре нельзя использовать стандартную библиотеку Cx — там свои функции). Она помещает выходные данные в кольцевой буфер (журнал) ядра; демон syslog в пространстве пользователя забирает их и переправляет на устройства согласно файлу конфигурации (/etc/syslog.conf).

231 / 257

В вызовах `printk` используются макросы уровней протоколирования (определены в `linux/kern_levels.h`):

```
1 #define KERN_EMERG  "0" /* system is unusable */
2 #define KERN_ALERT  "1" /* action must be taken immediately */
3 #define KERN_CRIT    "2" /* critical conditions */
4 #define KERN_ERR     "3" /* error conditions */
5 #define KERN_WARNING "4" /* warning conditions */
6 #define KERN_NOTICE  "5" /* normal but significant condition */
7 #define KERN_INFO    "6" /* informational */
8 #define KERN_DEBUG   "7" /* debug-level messages */
```

Листинг 93.

- Чем меньше значение, тем выше приоритет протоколирования.
- Чаще всего используется `KERN_INFO`.

Пример 14.4

```
1 printk(KERN_ERR "root creation failed!");
```

Важно

Для нас привычно ставить запятую. `KERN_ERR` — это строковая константа, которая объединена со строкой, и между ними запятая не ставится. Это относится к любому макросу.

Обычно сообщения журналируются в `/var/log/messages`, но могут перенаправляться и на пользовательский порт. Для просмотра кольцевого буфера ядро предоставляет утилиту `dmesg`, выводящую дамп буфера в стандартный поток.

14.7 Пример программы «Фортунки» из лаб. работы

ЛР посвящена передаче данных между пространством пользователя и ядром: работа с файлом `/С proc` ведётся в загружаемом модуле ядра. В модуле 6 точек входа: `open`, `read`, `write`, `release`, `init`, `exit`. Функции `open`/`release` вызываются, когда процедуру нужно работать с файлом (`fses` — процессами `echo` и `cat`, явно мы их не вызываем). В `read`/`write` выполняется `copy_to_user`/`copy_from_user`, так как ядро работает с физическими адресами, а процессы — с виртуальными.

```
1 #include <linux/init.h>
2 #include <linux/module.h>
3 #include <linux/kernel.h>
4 #include <linux/proc_fs.h>
```

232 / 257

15 Загружаемые модули ядра. Структура загружаемых модулей. Информация о процессах, доступная в ядре: `prio`, `priority`, `state`, `flags`. Пример вывода информации о запущенных процессах, символ `current` (`task`, `raf6`). Взаимодействие загружаемых модулей в ядре. Экспорт данных. Пример взаимодействия модулей (`task`, `raf6`). Функция `printk` — назначение и особенности. Виртуальная файловая система: рописание, примеры директорий и файлов `/proc`, файлы `map`, `page`, `map`, `task`. Регистриция функций работы с файлами `proc`. Пример приватизации пространства ядра: структура `proc_ops`. Передача данных из пространства ядра в пространство пользователя. Примеры из лабораторной работ.

15.1 Загружаемые модули ядра

Загружаемые модули ядра позволяют продвинутым пользователям включать в ядро нужную им функциональность; это механизм, специальные команды и специальная структура модуля.

Загружаемый модуль ядра — объект ядра, который может быть динамически загружен в ядро без необходимости его перекompilляции. Модуль имеет прямой доступ к структурам ядра.

Чтобы загрузить модуль в ядро, то есть выполнить команду `insmod` (`insert module`), нужны права суперпользователя.

Основные команды для работы с модулем ядра:

- `insmod` — загружает модуль; при этом вызывается функция инициализации модуля;
- `rmmod` — выгружает модуль; при этом вызывается функция завершения модуля;
- `lsmod` — выводит список загруженных модулей.

15.2 Структура загружаемых модулей

Обязательные составляющие модуля — функции `init()` и `exit()`, передаваемые макросом `module_init/module_exit`.

```
1 #include <linux/module.h>
2 #include <linux/init.h>
3
4 static int _init md_init(void)
5 {
6     printk("Module loaded!\n");
7     return 0;
8 }
9
10 static void _exit md_exit(void)
11 {
12     printk("Module unloaded!\n");
13 }
```

237 / 257

```
5 #include <linux/sem.h>
6 #include <linux/malloc.h>
7 #include <linux/string.h>
8 #include <linux/errno.h>
9
10 #define PROC_FILE_NAME "fortune"
11 #define PROC_DIR_NAME "fortune_dir"
12 #define PROC_SYM_NAME "fortune_symlink"
13
14 #define COOKIE_BUF_SIZE PAGE_SIZE
15
16 MODULE_LICENSE("GPL");
17 MODULE_AUTHOR("Zhizhin Nikita");
18
19 static char *cookie_buf;
20 static unsigned int write_index;
21 static unsigned int read_index;
22 static struct proc_dir_entry *proc_file;
23 static struct proc_dir_entry *proc_dir;
24 static struct proc_dir_entry *proc_symlink_entry;
25
26 static ssize_t fortune_write(struct file *file, const char __user *user_buf, size_t count, loff_t *ppos) {
27     size_t len;
28
29     if (count == 0) return 0;
30     if (count > COOKIE_BUF_SIZE - 1) return -ENOSPC;
31     if (write_index + count + 1 > COOKIE_BUF_SIZE) return -ENOSPC;
32     len = count;
33     if (copy_from_user(cookie_buf + write_index, user_buf, len)) {
34         return -EFAULT;
35     }
36     cookie_buf[write_index + len] = '\0';
37     printk(KERN_INFO "fortune: write %s", cookie_buf + write_index);
38     write_index = len + 1;
39     return len;
40 }
41
42 static ssize_t fortune_read(struct file *file, char __user *user_buf, size_t count, loff_t *ppos) {
43     size_t len;
44     size_t available;
45
46     if (*ppos > 0) return 0;
47     if (count == 0) return 0;
```

233 / 257

```
13 }
14
15 module_init(md_init);
16 module_exit(md_exit);
```

Листинг 94.

- `__` (два подчеркивания) — знак ядру обратить на действие пристальное внимание; современный синтаксис ядра (в старых версиях модули их нет).
- Это две точки кода модуля.

Точки входа в ядро ОС:

- 1) все системные вызовы;
- 2) все исключения;
- 3) все аппаратные прерывания.

— `module_init` регистрирует функцию инициализации (прототип `int init(void)`); она выполняется при загрузке, запрашивает ресурсы (память и т. д.) и при успехе возвращает 0.

— `module_exit` регистрирует функцию завершения (прототип `void func(void)`); выполняется при удалении модуля и освобождает ресурсы.

— `MODULE_LICENSE`, `MODULE_AUTHOR`, `MODULE_DESCRIPTION` описывают модуль (киндю командой `modinfo`). `MODULE_LICENSE` обязательна: без `MODULE_LICENSE("GPL")` модуль помечается как испорченный (`tainted`), что кликает на доступ к экспортируемым символам ядра.

Для любой функции ядра требуется обработка ошибок: при успехе возвращается отрицательное значение (иногда добавляет макрос `Linux/errno.h`), а при успешном завершении дополнительная информация не предоставляется и возвращается 0.

15.3 Информация о процессах, доступная в ядре: `prio`, `priority`, `state`, `flags`

В загружаемом модуле можно напрямую получить доступ к полям `struct task_struct` с информацией о процессе:

- `state` (в современных ядрах — `__state`) — состояние процесса, при выводе не переводится в строковую константу, печатается так, как определено в ядре.
- `flags` — флаги процесса (`PF_*`).
- `PF_NO_SETAFFINITY` — из пользовательского пространства запрещено изменять `cpu_mask`;
- `PF_FORKNOEXEC` — процесс выполнил `fork`, но не может выполнить `exec`;
- `PF_NOFREEZE` — поток не может быть заблокирован (заморожен);

238 / 257

```
48 if (write_index == 0) return 0;
49 if (read_index == write_index) read_index = 0;
50 available = write_index - read_index;
51 len = strlen(cookie_buf + read_index, available);
52 if (len == 0) {
53     read_index = 0;
54     len = strlen(cookie_buf, write_index);
55     if (len == 0) return 0;
56 }
57 if (len + count) len = count;
58 if (copy_to_user(user_buf, cookie_buf + read_index, len)) {
59     return -EFAULT;
60 }
61 printk(KERN_INFO "fortune: read %s", cookie_buf + read_index);
62 read_index += len + 1;
63 if (read_index > write_index)
64     read_index = 0;
65 *ppos += len;
66 return len;
67
68 static int fortune_open(struct inode *inode, struct file *file) {
69     printk(KERN_INFO "fortune: open");
70     return 0;
71 }
72 }
73
74 static int fortune_release(struct inode *inode, struct file *file) {
75     printk(KERN_INFO "fortune: release");
76     return 0;
77 }
78
79 static const struct proc_ops fortune_proc_ops = {
80     .proc_open = fortune_open,
81     .proc_read = fortune_read,
82     .proc_write = fortune_write,
83     .proc_release = fortune_release,
84 };
85
86 static int _init fortune_init(void) {
87     cookie_buf = vmalloc(COOKIE_BUF_SIZE);
88     if (!cookie_buf) {
89         printk(KERN_ERR "fortune: cannot allocate cookie buffer");
90         return -ENOMEM;
91     }
```

234 / 257

- `PF_NO_WORKER` — рабочий поток очереди работ, например `kworker`.
- `prio`, `static_prio`, `normal_prio`, `rt_priority` — приоритеты (`rt_priority` — реального времени). Чем меньше `prio`, тем выше приоритет; `prio` = 0 — наименьший. Отдельного поля `priority` в структуре нет.

Рядом — поле `policy` (политика планирования): 1 — FIFO (без переключения, пока процесс не закончится — выполняется «от начала до конца»), 2 — RR (переключение по кванту), 0 — по умолчанию (самый справедливый: чтобы все процессы получили процессорное время по возможности в равных количествах).

15.4 Пример вывода информации о запущенных процессах, символ `current`

Информацию о запущенных процессах выводит загружаемый модуль, обходящий кольцевой список процессов. Обход начинается со статической структуры `init_task` (указатель на первый `task_struct` в кольцевом списке) и продолжается в цикле `do-while` через `next_task`, пока снова не встретится `init_task`.

Символ `current` — определенный в ядре указатель на `task_struct` текущего процесса. После обхода `ru` все информацию выводит для текущего процесса (идея это `lsmod`, загружающий модуль), поэтому строка с `lsmod` выводится дважды: при обходе списка и в блоке `Current`.

```
1 static int _init md_init(void)
2 {
3     struct task_struct *task = &init_task;
4
5     printk(KERN_INFO "task: pid: %d, comm: %s, ppid: %d, state: %d, flags: %x,
6         prio: %d, policy: %d, root: %s, ppid: %s",
7         task->pid, task->comm, task->parent->pid,
8         task->state, task->flags, task->prio, task->policy,
9         task->fs->root, dentry->d_iname, task->fs->pwd, dentry->d_name.name);
10 }
11 while ((task = next_task(task)) != &init_task);
12 printk(KERN_INFO "Current: pid: %d, comm: %s, ppid: %d, state: %d, flags: %x,
13     prio: %d, policy: %d, root: %s, ppid: %s",
14     current->pid, current->comm, current->parent->pid,
15     current->state, current->flags, current->prio, current->policy,
16     current->fs->root, dentry->d_iname, current->fs->pwd, dentry->d_name.name);
17 return 0;
18 }
19
20 module_init(md_init);
21 module_exit(md_exit);
```

239 / 257

```
92 memset(cookie_buf, 0, COOKIE_BUF_SIZE);
93 write_index = 0;
94 read_index = 0;
95 proc_dir = proc_mkdir(PROC_DIR_NAME, NULL);
96 if (!proc_dir) {
97     printk(KERN_ERR "fortune: cannot create /proc/fs", PROC_DIR_NAME);
98     vfree(cookie_buf);
99     return -ENOMEM;
100 }
101 proc_file = proc_create(PROC_FILE_NAME, 0666, proc_dir, &fortune_proc_ops);
102 if (!proc_file) {
103     printk(KERN_ERR "fortune: cannot create /proc/fs", PROC_FILE_NAME);
104     proc_remove(proc_dir);
105     vfree(cookie_buf);
106     return -ENOMEM;
107 }
108 proc_symlink_entry = proc_symlink(PROC_SYM_NAME, NULL, "/proc/" PROC_DIR_NAME
109     "/" PROC_FILE_NAME);
110 if (!proc_symlink_entry) {
111     printk(KERN_ERR "fortune: cannot create /proc/fs", PROC_SYM_NAME);
112     proc_remove(proc_dir);
113     proc_remove(proc_file);
114     vfree(cookie_buf);
115     return -ENOMEM;
116 }
117 printk(KERN_INFO "fortune: module loaded");
118 return 0;
119 }
120 static void _exit fortune_exit(void) {
121     proc_remove(proc_symlink_entry);
122     proc_remove(proc_file);
123     proc_remove(proc_dir);
124     vfree(cookie_buf);
125     printk(KERN_INFO "fortune: module unloaded");
126 }
127
128 module_init(fortune_init);
129 module_exit(fortune_exit);
```

235 / 257

Поле `fs` — информация о файловой системе процесса; в листинге через него получают имя корня файловой системы (`fs->root`, `dentry`) и имя текущего рабочего каталога (`fs->pwd`, `dentry`).

15.5 Взаимодействие загружаемых модулей и ядре. Экспорт данных

Взаимодействие модулей — один модуль использует данные (переменные и функции), описанные в другом. Чтобы обратиться к данным другого модуля, их описывают так:

- 1) как `extern`, причём объявление кидно (через заголовочный файл) в модуле, который обращается;
- 2) имя передается в макрос `EXPORT_SYMBOL`.

Экспортируемые символы — данные и функции, доступные в другом модуле. Имена могут быть экспортированы ранее загруженными модулями, но чаще — ядром: множество экспортируемых имен ядра называют API ядра (пример вызова из него — `printk()`).

Модуль связывается с экспортируемыми именами по прямому абсолютному (физическому) адресу: ядро оперирует физическими адресами (загружено в физическую память), а процессы — виртуальными. Два следствия:

- любая перекорба ядра или экспортируемых модулей делает собранный модуль непригодным, потому модуль собирается только на месте использования;
- важен порядок загрузки: `md1` берёт абсолютный адрес из `md1`, и если `md1` не загрузил, адрес взять некуда.

Модуль может использовать только те имена, которые явно экспортированы. Локальное имя (модификатор `static`, например `md1_local()`) непригодно для связывания, а имя, не объявленное как экспортируемое (`md1_noexport()`), также не может быть использовано вне модуля.

После загрузки `lsmod` показывает, что модуль используется другим. Число ссылок на `md1` (со стороны других модулей или объектов ядра) — счётчик ссылок: разом — ссылки ссылающихся модулей. Пока счётчик не нулевой, модуль не выгружать, поэтому `sudo rmmod md1` при загруженном `md2` даёт ошибку `ERROR: Module md1 is in use by md2`.

15.6 Пример взаимодействия модулей

Возможность взаимодействия рассматривается (по статье Цихаркина) на примере трёх модулей ядра — `md1`, `md2`, `md3` — и общего заголовочного файла `md.h`. Базовый модуль `md1` экспортирует данные, остальные модули их используют.

240 / 257

15

В заголовочном файле `md.h` экспортируемые имена объявлены как `extern`:

```
1 extern char *md1_data;
2 extern char *md1_proc(void);
3 extern char *md1_noexport(void);
```

```
1 // md1.c
2 MODULE_LICENSE("GPL");
3
4 char *md1_data = "aaa";
5 extern char *md1_proc(void) { return md1_data; }
6 static char *md1_local(void) { return md1_data; } // локальная (static)
7 extern char *md1_noexport(void) { return md1_data; } // не экспортируется
8
9 EXPORT_SYMBOL(md1_data);
10 EXPORT_SYMBOL(md1_proc);
11
12 // + md1/md_exit + module_init/module_exit
```

```
1 // md2.c
2 #include "md.h"
3 MODULE_LICENSE("GPL");
4
5 static int _init md2_init(void) {
6     printk("%s data string exported from md1: %s\n", md1_data,
7         printk("%s string returned md2_proc() is: %s\n", md1_proc()));
8 // md1_local(); (1) - ошибка компиляции
9 // md1_noexport(); (2) - ошибка компоновки
10 return 0;
11 }
12 // + md2/md_exit + module_init/module_exit
```

Модули `md1` и `md2` собираются и взаимодействуют без ошибок. Ошибочные ситу-

- строка (1) с `md1_local` — ошибка компиляции (несоответствие `md1_local` и `md1.c` и `md.h`);
- строка (2) с `md1_noexport` — ошибка компоновки (`md1_noexport` не передана в `EXPORT_SYMBOL`, нет абсолютного адреса).

`md3` отличается тем, что функция инициализации возвращает -1 (ошибка):

```
1 #include "md.h"
2 MODULE_LICENSE("GPL");
3
4 static int _init md3_init(void) {
```

241 / 257

Элемент	Тип	Содержание
		Значение 15.1 Разные файлы в одном и том же процессе могут иметь разные расширения, доступные через <code>proc/<code>fsstat/<code>statfs/<code>statfs.comb</code> – файл, в котором находится подмножество потоков.</code></code></code>
stat	Файл	Содержит информацию о процессе (<code>vsiz</code>, <code>stat</code>, <code>brk</code> и др.) Содержит ссылки на открытые процессы и процессы файлов
fd	Директория	Значение 15.2 В системе различают файлы и открытые файлы. Открытые файлы описываются структурой <code>struct file</code>. Процессы представляются в системе деревьями каталогов, а описываются файнсистемные файлы <code>struct inode</code>. Один и тот же файл может быть открыт много раз, привнося различные процессы. В таблице описаны файлы на каждой такой открытой файлу будет дескриптор <code>struct file</code>
root	soft link	Это символическая ссылка, которая указывает на корень файловой системы процесса. Значение 15.3 Процессы, которые выполняются на нашем компьютере, могут привлекать разным файлам процессы. Пока мы не знаем исполняемого файла не указываем в командной строке, пока он не начал выполняться, он лежит во внутренней памяти.
maps	файл	Это адресное пространство того процесса, <code>pid</code> которого мы указали. В зависимости от <code>pid</code> информации может отличаться, но есть неизменяемые составляющие.
pageadm	файл	Это 64-х разрядное значение для каждой строки процесса. При этом нас интересует фактически несколько бит: 63 бит – <code>present</code> (in RAM) 62 бит – <code>swap</code> space, область свопинга 61 – <code>shared</code> anonymous page 55 бит – <code>dirty</code>. Он указывается в дескрипторе страницы.

Элемент	Тип	Содержание
		С 54 по 6: если страница присутствует в физической памяти, то эти биты являются номером фрейма, то есть номером физической страницы.
task	Директория	Содержит поддиректорию потоков
		Предоставляет таблицу о количестве прерываний на каждом из процессоров. В следующем виде:
		1) Первая колонка: номер прерывания
interrupts	файл	2) Колонка CPUs: счётчики прерываний на каждом из процессоров
		3) Следующая колонка: вид прерывания:
		3.1) IO-APIC-edge – прерывание по фронту на контролере I/O APIC.
		3.2) IO-APIC-fastest – прерывание по уровню на контролере I/O APIC.
		3.3) FCS-MSI-edge – MSI прерывание.
		3.4) XT-PIC-XT-PIC – прерывание на PIC контролере.
		4) Последняя колонка: устройство, ассоциированное с данным прерыванием.

```

32 len = count;
33 if (copy_from_user(cookie_buf + write_index, user_buf, len)) {
34     return -EFAULT;
35 }
36 cookie_buf[write_index + len] = '\0';
37 printk(KERN_INFO "fortune: write %s", cookie_buf + write_index);
38 write_index += len + 1;
39 return len;
40 }
41
42 static ssize_t fortune_read(struct file *file, char __user *user_buf, size_t count,
43                             loff_t *ppos) {
44     size_t len;
45     size_t available;
46
47     if (*ppos > 0) return 0;
48     if (count == 0) return 0;
49     if (write_index == 0) return 0;
50     if (read_index == write_index) read_index = 0;
51     available = write_index - read_index;
52     len = strlen(cookie_buf + read_index, available);
53     if (len == 0) {
54         read_index = 0;
55         len = strlen(cookie_buf, write_index);
56         if (len == 0) return 0;
57     }
58     if (len > count) len = count;
59     if (copy_to_user(user_buf, cookie_buf + read_index, len)) {
60         return -EFAULT;
61     }
62     printk(KERN_INFO "fortune: read %s", cookie_buf + read_index);
63     read_index = len + 1;
64     if (read_index >= write_index)
65         read_index = 0;
66     *ppos += len;
67     return len;
68 }
69
70 static int fortune_open(struct inode *inode, struct file *file) {
71     printk(KERN_INFO "fortune: open");
72     return 0;
73 }
74
75 static int fortune_release(struct inode *inode, struct file *file) {
76     printk(KERN_INFO "fortune: release");
77 }

```

```

76     return 0;
77 }
78
79 static const struct proc_ops fortune_proc_ops = {
80     .proc_open = fortune_open,
81     .proc_read = fortune_read,
82     .proc_write = fortune_write,
83     .proc_release = fortune_release,
84 };
85
86
87
88 static int __init fortune_init(void)
89 {
90     cookie_buf = malloc(COOKIE_BUF_SIZE);
91
92     if (!cookie_buf)
93         printk(KERN_ERR "fortune: cannot allocate cookie buffer");
94     return -ENOMEM;
95 }
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```
120 static void _exit_fortune_exit(void) {
121     proc_remove(proc_symlink_entry);
122     proc_remove(proc_file);
123     proc_remove(proc_dir);
124     vfree(cookie_buf);
125     printk(KERN_INFO "fortune: module unloaded");
126 }
127
128 module_init(fortune_init);
129 module_exit(fortune_exit);
```

252 / 257

Именно с флагом `O_CREAT` связан третий параметр `mode`: он задаёт права доступа создаваемого файла и должен быть указан при использовании `O_CREAT`; во всех остальных случаях `mode` игнорируется.

Для режима предусмотрены символические константы (для владельца / группы / остальных):

- `S_IRWXU` / `S_IRWXG` / `S_IRWXO` — права на чтение, запись и выполнение;
- `S_IRUSR` / `S_IRGRP` / `S_IROTH` — права на чтение;
- `S_IWUSR` / `S_IWGRP` / `S_IWOTH` — права на запись;
- `S_IXUSR` / `S_IXGRP` / `S_IXOTH` — права на выполнение.

Вместе с `O_CREAT` часто используется флаг `O_EXCL`: он заставляет систему проверить, существует ли уже создаваемый файл, чтобы не потерять ранее записанную в него информацию. Если файл с таким именем уже есть, вызов `open` с `O_CREAT` | `O_EXCL` завершается ошибкой.

Пример:

```
1 open("file.txt", O_CREAT | O_WRONLY, S_IRUSR | S_IWUSR); // права -rw.....
```

257 / 257

253 / 257

16 Открытые файлы: системный вызов `open()`: `int open(const char* pathname, int flags); int open(const char* pathname, int flags, mode_t mode);` пояснить смысла параметров. Основные флаги: `Flag CREATE`. Реализация системного вызова `open()` в системе — действия в ядре: `SYSCALL_DEFINE3(open, ...)` → `ksys_open(filename, flags, mode)` → `do_sys_open()` → `do_sys_openat2()` ... найти наименьший файловый дескриптор... Действия, если флаг `O_CREAT` установлен?

16.1 Открытые файлы

В системе различаются просто файлы и открытые файлы. Просто файлы — то, что мы видим в каталогах: они «тихо» лежат во вторичной памяти и описываются структурой `struct inode`, которая описывает физический файл. Открытый файл — файл, который открыт в результате выполнения программы, то есть открыт процессом; открытые файлы описываются структурой `struct file` — дескриптором открытого файла.

Любой процесс может открыть много файлов, причём один и тот же файл может быть открыт много раз — одним или несколькими процессами; сколько раз открыт файл, столько в таблице открытых файлов будет дескрипторов открытых файлов.

Каждый процесс имеет собственную таблицу открытых файлов (поле `struct file *fd_array[]` в структуре `files_struct`). Файловый дескриптор — индекс в этой таблице. При порождении процесса для него автоматически выделяются три дескриптора: 0 (`stdin`), 1 (`stdout`), 2 (`stderr`); первый открытый процессом файл получит дескриптор 3. Прежде чем выдать дескриптор, система выполняет поиск наименьшего свободного.

16.2 Системный вызов `open()`: смысл параметров

Чтобы получить возможность прочитать что-то из файла или записать что-то в файл, его нужно открыть системным вызовом `open()`. В пространстве пользователя есть две формы вызова:

```
1 int open(const char *pathname, int flags);
2 int open(const char *pathname, int flags, mode_t mode);
```

- `pathname` — имя открываемого файла;
- `flags` — флаги, определяющие режим доступа и поведение при открытии;
- `mode` — права доступа создаваемого файла.

Третий параметр `mode` необходим, когда `open` вызывается для создания нового файла: тогда нужно указать права доступа для трёх групп пользователей. Если же `open` вызывается для открытия существующего файла, `mode` не нужен. При создании нового файла создаётся новый `inode` (инициализируется структура `inode`).

255 / 257

и, поскольку созданный файл открывается, для него инициализируются поля `struct file`.

Функция возвращает файловый дескриптор — целое неотрицательное число, которое можно использовать в последующих системных вызовах. Это наименьший дескриптор из таблицы открытых процессом файлов, который ещё не занят. Смещение в открытом файле (`f_pos`) устанавливается в начало файла.

16.3 Основные флаги

Флаги (`flags`) в системе играют огромную роль как ограничения: они указывают, что с файлом можно делать, а что нельзя. Параметр `flags` задаёт режим доступа и поведение при открытии файла.

Флаги доступа (один из них указывается обязательно):

- `O_RDONLY` — открыть файл только на чтение;
- `O_WRONLY` — открыть файл только на запись;
- `O_RDWR` — открыть файл на чтение и запись.

Некоторые флаги создания и поведения:

- `O_CREAT` — создать файл, если его не существует (подробно рассмотрен ниже);
- `O_APPEND` — дописыв в существующий файл: перед каждым вызовом `write()` смещение файла переносится на его конец, поэтому ранее записанная информация не терится;
- `O_TRUNC` — если файл существует и открыт для записи, усекает его до нулевой длины;
- `O_NONBLOCK` — устанавливает режим неблокирования. Эта опция обычно полезна для специальных файлов типа `PIPE` и устройств типа терминалов; обычно для этих файлов `open` блокируется, пока файл не «готов», а если `O_NONBLOCK` установлен, `open` возвращается немедленно. Бит также воздействует на чтение и запись: разрешает им возвращаться немедленно с состоянием ошибки, если нет доступного ввода или вывод не может быть записан;
- `O_CLOEXEC` — установить флаг `close-on-exe` на новом файловом дескрипторе;
- `O_PATH` — получить лишь файловый дескриптор (сам файл не будет открыт);
- `O_TEMPFILE` — создать неименованный временный обычный файл.

16.4 Флаг `CREATE`

Флаг `O_CREAT` создаёт новый файл, если его не существует; если файл существует, флаг игнорируется. Создание файла означает создание нового `inode`. При этом идентификаторы владельца и группы файла берутся из действующих идентификаторов пользователя и группы процесса.

256 / 257